

Ternary Network Floats

*Троичные сетевые числа с плавающей точкой:
формат, кремний и методология измерения*

Проект t27 · Trinity S³AI

ревизия 60b2c017c · собрано 2026-08-18

источник docs/theory/TNF_ARTICLE_RU.md

Действующее правило измерения (T619a). Кремниевый вердикт обязан совпасть минимум на трёх сидах размещения, иначе он является утверждением об одном размещении, а не о спецификации. Основание: на неизменном нетлисте достигнутая частота прошла от 15.83 до 18.29 МГц, а три размещения из пяти вычисляли заданную функцию и два — нет, детерминированно.

Ternary Network Floats

Trinity S³AI: Ternary Network Floats — троичные сетевые числа

Перевод статьи на русский. Автор: Дмитрий Васильев (gHashTag). Все числа сохранены точно.

Оглавление

1. Аннотация
2. Что утверждает эта статья
3. Формат
4. Зачем лестница и почему именно эта
5. Закон общий, и он измеряет сужение
6. Точность
7. 16-битное поле
8. Железо
9. Два дефекта ширины и почему они обобщаются
10. Эквивалентность
11. Что закон говорит об улучшении лестницы
12. Почему троичность — и ровно насколько
13. Поле режима — это универсальный код
14. Перевод кремния в точность
15. Закон значения стоит дороже, чем раскладка полей
16. Есть ли оптимальный формат — и в каком смысле
17. Там, где формат и есть тракт данных
18. Лестница — это семейство, а семейство — это фронтир
19. Четыре семейства на двух осях
20. Два формата закрывают троичный узел
21. Какой рунг брать и как ответ смещается
22. Блочная ось и та её половина, что принадлежит нам

23. Тонкость стоит регистров, а не сумматоров
 24. Чего пара стоит в кремнии
 25. Что на самом деле меряет таблица пропускной способности
 26. На пятидесяти четырёх семенах утверждения расходятся
 27. Почему лестница останавливается на пяти тритах, когда мишень — fp32
 28. Инструмент разрешает семь уровней, а не двадцать восемь
 29. Троичный порядок никогда не замощает двоичное поле
 30. Рунг ровно в шестнадцать бит и чего стоит там победа
 31. Когда сверка меряет саму себя
 32. Что это решает
 33. Где это стоит в шестидесятивосьмилетнем споре
 34. Что достраивает золотое сечение
 35. Шестнадцать способов, которыми измерение может быть верным, а сравнение вокруг него — нет
 36. Ограничения
 37. Воспроизводимость
 38. Раскрытие
 39. Список литературы
-

Тракт данных, в котором нигде нет умножителя: два формата закрывают две половины троичного узла, один примитив «выбор знака» строит и радио, и трансформер при нулевом расходе DSP, а путь без умножения точен.

Дмитрий Васильев ORCID 0009-0008-4294-6159 admin@t27.ai

11 августа 2026

Аннотация

У троичного узла три места, где мог бы жить числовой формат; нужен он ровно в одном. Вес — это код: умножение на него есть выбор знака. Отсчёт приходит в родном виде АЦП. Только у аккумулятора есть

диапазон, который надо на что-то потратить, и Ternary Network Floats тратят его так: знак, порядок из E_t уравновешенно-троичных тритов, M -битная мантисса, $1 + E_t + M = N$.

Вес — это **GFternary**, двухбитный алфавит $\{-\varphi, 0, +\varphi\}$, и именно здесь решается судьба умножителя. Чтобы убрать его, а не удешевить, решётка весов должна быть *замкнута* и относительно применения веса, и относительно накопления. $\mathbb{Z}[\varphi]$ замкнута, потому что $\varphi^2 = \varphi + 1$: применение веса — это шаг Фибоначчи $(a, b) \mapsto (b, a + b)$, накопление покомпонентно, а линейный путь слоя точен. Проверено машиной в Coq.

Замкнутость сама по себе не выделяет φ ; выделяет его перебор. Шкала свободна от умножителя ровно тогда, когда она — целое алгебраическое число, у сопровождающей матрицы которого элементы лежат в $\{0, \pm 1\}$. Вторая степень допускает единственное φ , и между сдвигом и φ нет ничего. Семейство $r^d = r + 1$ достигает любой зернистости за *одно сложение* и d регистров: тонкость шага стоит регистров, а не сумматоров.

Среди двадцати четырёх форматов, несущих собственный диапазон, — тех, в которых может жить обучающий прогон, — первой строкой по пропускной способности на единицу площади стоит GFternary: 0.1584 MHz/LUT, на 10.85% впереди следующего и в $5.245\times$ впереди posit32, причём двенадцать из двадцати четырёх — наши. Первого места по самой измеряемой величине это не даёт: int8 на том же стенде даёт 0.1736 MHz/LUT, то есть на 9.60% больше, и вынесен в отдельную таблицу не по этой величине, а по роли — у него нет поля порядка, и диапазон приходит внешним обучаемым масштабом. Столбец меряет МГц на LUT, и по нему мы не первые; вдобавок ранжирование сводит вместе ширины входа от 2 до 32 бит, тогда как собственное предложение этой статьи об усечении требует сравнивать внутри одного n . Заглавное утверждение о лидерстве отозвано (эпратум от 13 августа 2026 года), и на его месте остаётся утверждение о конструкции: среди форматов одинаковой ширины входа GFternary — единственный, чья решётка замкнута относительно применения веса,

и потому единственный, у которого линейный тракт точен без ступени нормализации. На XC7A200T троичный нейрон стоит 28 LUT на вес и *ни одного DSP при любом фан-ине*.

Какой рунг брать — следует из бюджета: φ на четырёх битах, $r^5 = r^3 + 1$ на пяти, $r^6 = r + 1$ на шести, — и на шести битах при одном сумматоре мы доходим до 3.7% от fp32. На блочной оси элемент замкнут — лучший восьмиуровневый кодовый словарь обыгрывает развёрнутый в проде на 0.9%, — а масштаб не замкнут: φ -сетка на четырёх битах обыгрывает MXFP4 на двух моделях при на 0.125 бита на вес меньше.

Пять снятых утверждений отмечены ниже прямо по месту.

ОГОВОРКА W845 [не разрешено]. Число снятых утверждений названо в этой статье **четырьмя несводимыми способами**: «пять» здесь, «десять» ниже в этом же разделе, «одиннадцать» в эрратуме и «шестнадцать, два из них затем отозваны» в разделе о самопроверке. Защита через разные охваты не проходит: раздел о самопроверке прямо приписывает шестнадцать той же популяции — «приложенная к нашим собственным утверждениям, процедура дала шестнадцать отзывов», — что и «десять». **Свести их выбором числа нельзя**: попытка сделать это в W845 дала текст, противоречащий четырём другим местам, и была отклонена проверкой. Требуется перечень: пронумерованный список каждого снятия с местом, где оно сделано, и одно число, вычисленное из него. До тех пор **ни одно из четырёх чисел не следует цитировать**.

Что утверждает эта статья

Результаты ниже найдены в том порядке, в каком их находит измерительная кампания. Они складываются в один довод, а довод легче проследить, если сформулировать его сразу.

У троичного узла три места, где мог бы жить числовой формат, и нужен он лишь в одном. Вес — это код, и умножение на него есть выбор знака; отсчёт приходит в родном виде АЦП; аккумулятор — единственный

объект, несущий диапазон, который надо потратить. Значит, вопрос проектирования не в том, какой формат взять, а в том, какие два объекта замкнуть — алфавит для веса и float для аккумулятора, — и разделы от «Формат» до «16-битное поле» решают второй вопрос законом: как только диапазон назван, деление между порядком и мантиссой не выбирается, а определяется, потому что ошибка зависит только от ширины мантиссы. В обращённом виде тот же закон вычитывает поведение формата из его ошибки на круговом преобразовании и разрешает каталог из 83 форматов в четыре ступенчатые формы — и пятой нет.

TNF — это один угол квадрата, и раздел о четырёх семействах его чертит. Два правила фиксируют бюджет порядка — правило золотого сечения $E = \text{round}((N-1)/\varphi^2)$ и правило ширины $1+E+M=N$, — и два основания его кодируют, двоичное или уравновешенно-троичное. Четыре семейства — GF, GF-T, BNF и TNF, каждое отдельный формат, а не переименование другого; и на восьми битах два правила, как выясняется, называют в точности одну и ту же раскладку.

Алфавит — это место, где решается судьба умножителя, и решающее свойство алгебраическое. Раздел о замкнутости показывает: чтобы убрать умножитель — в отличие от того, чтобы его удешевить, — решётка весов должна быть замкнута относительно операции применения веса. $\mathbb{Z}[\varphi]$ замкнута, потому что $\varphi^2 = \varphi + 1$, так что применение веса — это шаг Фибоначчи, а накопление — покомпонентное целочисленное сложение; целые Фибоначчи не замкнуты, потому что $F_4 F_4 = 9$ не является числом Фибоначчи, и тракт данных, построенный на них, обязан нести ступень, возвращающую произведения в представимое множество. Эта ступень измеряется на порядок большей площадью, чем всё замкнутое накопление целиком, и конвейеризация её не спасает: каскад нормализации нельзя разрезать мельче, чем на одно сравнение-с-вычитанием на ступень, так что его нижний предел лежит выше единственного сложения на замкнутом пути. Половина довода про замкнутость проверена машиной.

Замкнутость сама по себе не выделяет φ , и раздел об иерархии говорит, что же она выделяет. Шкалу можно применить без умножителя ровно тогда, когда она — целое алгебраическое число, у сопровождающей матрицы которого элементы лежат в $\{0, \pm 1\}$; это конечное условие, а

потому перечислимое, а не дискуссионное. Первая степень даёт сдвиг; вторая даёт φ и больше ничего; третья даёт ещё три, более тонких. Между сдвигом и φ нет ничего, и φ — единственное безумножительное уточнение лестницы степеней двойки, доступное при двух регистрах. Иерархия продолжается — и перестаёт быть дешёвой на четвёртой степени, где вектор коэффициентов теряет разреженность и стоимость удваивается.

Какой рунг брать — уже эмпирический вопрос, но за ним стоит закон (раздел о законе лестницы). Лестница с отношением r и n величинами покрывает r^{n-1} , так что мелкий шаг покупается диапазоном, и оптимум сползает вниз по иерархии по мере роста кодового бюджета: сдвиг на трёх битах, φ на четырёх, пластическое число на пяти — на двух моделях с одинаковым порядком повсюду. У оптимума есть замкнутая форма через гистограмму весов, она переносится между моделями с точностью до четвёртого знака, а единственное место, где она ошибается — четырёхбитная пара, — исправляется, если взвешивать ошибку кривизной функции потерь, а не величиной веса.

Раздел о блочной границе сообщает, где всё это не выигрывает. На коротком блоке с общим масштабом оптимальный восьмиуровневый кодовый словарь обыгрывает развёрнутый в проде на 0.9% — значит, эта ось вообще не про формат элемента, и она принадлежит MXFP4. Мы излагаем это как результат конкурента и приводим границу, из-за которой четвёртая наша попытка была бы неоправданной.

Два раздела — про стоимость, а не про род. Раздел о конвертации делает площадь и точность соизмеримыми, так что поле порядка переменной длины можно оценить в мантисных битах кремния; раздел о законе значения затем обнаруживает, что для логарифмического формата раскладка полей — меньшая половина этой стоимости, и что мы всё это время измеряли не ту половину.

Десять наших собственных утверждений были опровергнуты теми самыми измерениями, которые должны были их подтвердить, и каждое снято там, где было сделано, а не тихо выброшено. Шестое, в разделе о корректной постановке, объясняет сразу четыре опубликованные цифры расхождения, потому что у них оказалась одна общая причина: не то, что сравнивали, а само сравнение. Последние четыре нашла независимая проверка, а не мы: оговорка, которую эта

статья вычислила, а не измерила; фраза, защищавшая наше же первое место; ранжирование, ни разу не проверенное на фоне разрешающей способности собственного инструмента; и четыре рунга лестницы, у которых порядки алиасят. Раздел о самопроверке обобщает форму этих ошибок, потому что почти всегда измерение было верным, а сравнение вокруг него — нет.

Формат

$$\text{TNF16} = [s \mid E=4\text{уравновешенно-троичных} \\ \text{трита} \mid M=11\text{бит}], v = (-1)^s (1 + \frac{M}{2^9}) 2^e, e = \sum i t_i 3^i \in [-40, 40].$$

Три свойства следуют из раскладки, а не из подгонки.

Никакого декодирования режима. Главная статья расходов в сужающемся (tapered) формате — режимное поле переменной длины: его надо найти, посчитать его длину и бочкообразным сдвигом поставить значащую часть на место. Эта цена платится на любой фабрике, троичной в том числе. Фиксированные поля превращают её в срез битов.

Порядок уже троичный. Четыре уравновешенно-троичных трита дают $3^4=81$ ступень порядка. На троичной фабрике сложение порядков нативно — ни двоичного переноса, ни перевода между основаниями.

Точность не сужается. Девять битов мантиссы при любой величине — там, где сужающийся формат ужимается к краям. Это и есть механизм, стоящий за результатом по точности (см. раздел «Точность»): выигрыш появляется там, где формат-соперник потратил мантиссу на диапазон.

Зачем лестница и почему именно эта

У формата, который используют как эталон, требования не те, что у рабочей лошадки. Рабочая лошадка должна быть быстрой и точной там, где лежат данные. Эталон обязан быть *предсказуемым*: по его параметрам вы должны знать его ошибку, не измеряя её, и эта ошибка не должна съезжать, когда вы смотрите на другую часть диапазона. Линейка, у которой деления к одному концу растут, — не линейка.

Таблица ниже проверяет оба свойства на этой лестнице.

Таблица (закон точности и плоскостность), измерено в точных рациональных. «Плоскостность» — отношение худшей полосы величин к лучшей: 1.000 означало бы, что ошибка вообще не зависит от величины.

рунг	M	$2^{-(M+1)}$	измерено	отношение	плоскостность
TNF8	4	$3.12e-2$	$1.22e-2$	0.39	1.000
TNF16	9	$9.77e-4$	$3.60e-4$	0.37	1.070
TNF32	25	$1.49e-8$	$5.49e-9$	0.37	1.070
TNF64	52	$1.11e-16$	$4.22e-17$	0.38	1.050
TNF128	115	$1.20e-35$	$4.44e-36$	0.37	1.070
TNF256	242	$7.07e-74$	$2.69e-74$	0.38	1.050
TNF512	497	$1.22e-150$	$4.51e-151$	0.37	1.070
TNF1024	1006	$7.29e-304$	$2.77e-304$	0.38	1.050

Точность выражается замкнутой формулой, и формула эта выведена, а не подогнана. Обозначим $u=2^{-(M+1)}$ — единичное округление. Внутри бинады абсолютная ошибка округления при round-to-nearest равномерна на $[-u2^e, +u2^e]$, тогда как значение равно $v=s2^e$ со значащей частью $s \in [1, 2)$; поэтому относительная ошибка равна $|U|/s$, где $|U|$ равномерно на $[0, u]$ и не зависит от e . Отсюда:

Теорема 1 (закон точности). [доказано] Для формата с постоянной шириной значащей части в M бит при round-to-nearest, где значащая часть s и порядок e независимы,

$$\mathbb{E}[|\text{отн. ошибка}|] = \frac{1}{2} \mathbb{E}[1/s] \cdot 2^{-(M+1)},$$

независимо от e .

Порядок выпадает полностью — это и есть плоскостность следующего абзаца, доказанная, а не подмеченная. Константа не универсальна: она равна $\frac{1}{2} \ln 2 = 0.3466$ для значащей части, равномерной на $[1, 2)$, и $\frac{1}{2} (2 \ln 2)^{-1} = 0.3607$ при логарифмическом (бенфордовом) распределении значащей части. У нашей нагрузки $\mathbb{E}[1/s] = 0.7721$, что

предсказывает 0.3861; измеренные цифры в таблице дают в среднем 0.3756 по восьми рунгам, с разбросом 0.369–0.390, который это значение накрывает.

Практическое следствие — ровно то, что нужно эталону: по M и распределению значащей части рабочей нагрузки ошибка ещё не построенного рунга известна до того, как его построят.

Ошибка не съезжает с величиной и не может съехать. Вывод выше убирает e из выражения, так что независимость от величины — свойство раскладки, а не удачное измерение. Эмпирически плоскостность держится между 1.05 и 1.07 — то есть максимум 7% разницы между ближайшей и дальней частями рабочей нагрузки, покрывающей 76 двоичных декад. Это то самое свойство, которое покупают фиксированные поля, и то, которого у сужающегося формата быть не может: тратить биты мантиссы на диапазон — это ровно то, что делает точность зависящей от того, где вы находитесь.

Чего здесь не утверждается. Не утверждается, что TNF — самый точный формат при данной ширине: раздел о 16-битном поле показывает `posit16` и `binary16` впереди вблизи единицы, а он же показывает двоичные форматы впереди без оговорок на 32 битах. Эталону не нужно выигрывать эти сравнения. Ему нужно быть тем, относительно чего вы их меряете.

Закон общий, и он измеряет сужение

Предыдущий раздел вывел ошибку формата с фиксированными полями. В этом выводе нет ничего специфичного для TNF, так что он должен выполняться для любого формата с неизменной шириной значащей части — и должен заметно ломаться на том, у которого ширина меняется. Это превращает его из описания в инструмент, и здесь мы им пользуемся именно так.

Определение 1 (эффективная мантисса). Для формата и полосы величин B положим

$$M_{\text{eff}}(B) = -\log_2 \left(\frac{2\mathbb{E}[|\text{отн. ошибка}| | B]}{\mathbb{E}[1/s]} \right) - 1,$$

то есть ширину мантиссы, которая понадобилась бы формату с фиксированными полями, чтобы дать ошибку, наблюдаемую в B .

Теорема 2 (диагностика). [доказано] M_{eff} постоянна по полосам тогда и только тогда, когда формат держит постоянную ширину значащей части на этих полосах и не исчерпывает свой диапазон. Для сужающегося формата M_{eff} убывает с $|e|$, а наклон $dM_{eff}/d|e|$ — это темп сужения в битах значащей части на бинаду.

Таблица ниже применяет её. Измерение ведётся при равномерной значащей части на $[1, 2)$, так что $\mathbb{E}[1/s] = \ln 2$ точно — рабочая нагрузка тем самым выведена из вопроса.

Таблица (эффективная мантисса по полосам величин и подогнанный темп сужения). Заявленное M восстанавливается с точностью до 0.01 бита для каждого формата с фиксированными полями, что подтверждает закон; сужающиеся форматы отделяются чисто.

формат	заявл. M	$\ e\ _{0-6}$	6–12	12–20	20–30	бит/бинада
TNF16	9	8.99	8.99	9.00	9.01	+0.001
GF16	9	8.99	8.99	9.00	9.01	+0.001
bfloat16	7	7.04	7.04	7.00	7.04	–0.000
binary16	10	10.01	10.01	7.41	0.52	–
posit16	–	10.72	9.34	7.48	5.17	–0.254
takum16	–	9.62	8.17	7.34	7.04	–0.113

Закон восстанавливает то, что форматы о себе заявляют. TNF16 и GF16 возвращают 8.99–9.01 против заявленной 9; bfloat16 возвращает 7.00–7.04 против заявленной 7. Закон, который восстанавливает собственный параметр формата с точностью до сотой доли бита, не имея ничего, кроме ошибки на круговом преобразовании, делает нечто большее, чем подгонку кривой.

Сужение становится числом. posit16 теряет 0.261 бита значащей части на бинаду ($R^2=0.999$), а takum16, судя по всему, теряет 0.113 на тридцати бинадах этой развёртки — но см. раздел о функциональной форме сужения: у takum сужение логарифмическое, так что эта цифра

— артефакт окна, а не темп. Это и есть та величина, которую сужающийся формат меняет на диапазон, и, насколько нам известно, её обычно не сообщают одним числом. Она же прямо объясняет таблицы точности: posit16 стартует на 1.7 бита впереди TNF16 и заканчивает на 3.8 позади.

Исчерпание диапазона — другой отказ, и инструмент их разделяет. binary16 — формат с фиксированными полями, однако его M_{eff} обваливается с 10.01 до 0.52. Это не сужение: это уход в субнормали, а затем переполнение. Постоянный наклон означает сужение; обрыв означает предел диапазона. В таблице точности эти два выглядят одинаково, а здесь — нет.

Следствие 1. [доказано] Положение формата на компромиссе «диапазон — точность» можно записать парой: постоянная M_{eff} которую он держит, и число бинад, на которых он её держит. Для лестницы TNF это $(M, 2 \cdot 3^{E_i-1})$ по построению; для сужающегося формата не постоянно ни то ни другое — вот почему лестница получается лучшей линейкой, чем сужение.

Диагностика на масштабе каталога

Применение теоремы 2 к каждому формату, для которого нашёлся опубликованный оракул, — 51 штука, включая IEEE, bfloat, posit, takum, LNS, MXFP, VAX, шестнадцатеричный IBM, Cray, PDP-11, x87 и Microsoft binary, плюс обе лестницы GoldenFloat, — дало три наблюдения, которых мы не ждали.

Заявленные ширины восстанавливаются на трёх порядках величины по ширине. Лестница GF возвращает 8.97 на GF16 против заявленной 9, 78.03 на GF128 против 78 и 631.99 на GF1024 против 632. То же верно для double-double (106.68), quad-double (215.90) и 80-битного формата x87 (63.04). Соотношение, восстанавливающее заявленный параметр из одной лишь ошибки кругового преобразования, на ширинах от 8 до 1006 бит, делает что-то иное, нежели описывает формат, на котором его вывели.

Темп сужения — константа семейства, но темп есть только у posit. posit16 теряет 0.261 бита значащей части на бинаду, posit32 теряет 0.247 — против $2^{-es}=0.25$, которых требует теорема 3 ниже; posit64, оракул которого несёт нестандартное $es=3$, теряет 0.123 против требуемых 0.125. Три формата, три точных подтверждения. Ранее мы сообщали сопоставимый побинадный темп для takum, и это была категориальная ошибка с нашей стороны: ступенчатость takum логарифмическая, так что подгонка прямой даёт число, зависящее от подогнанного интервала и ничего не описывающее. Правильная его константа — один бит на удвоение $|e|$, и против неё четыре рунга takum и три рунга tekum измеряются в 0.83–1.09.

Исчерпание диапазона отличимо от сужения, а оба — от «ни того ни другого». Различие не факультативно: наш собственный первый проход поставил полосы измерения в фиксированные позиции, а не внутрь измеренного диапазона каждого формата, и прочитал насыщение любого узкого формата как сужение — классифицировав TNF16, формат с фиксированными полями, как геометрически сужающийся. Если сначала измерить пригодный диапазон каждого формата, а потом ставить полосы внутрь него, артефакт исчезает полностью. GF14 читается плоским на 8.05 по всему своему диапазону и обваливается лишь ниже своей субнормальной границы, что дефектом не является; инструмент говорит, какой из двух случаев перед вами, а одна лишь таблица точности — нет.

У сужения есть функциональная форма, и она следует из кодирования

Предыдущий подраздел сообщил темп сужения по семействам. Это было наполовину верно, и неверная половина поучительна: темп предполагает линейное сужение, а линейное оно лишь у одного из двух семейств.

Теорема 3 (сужение posit, точно). [источник в тексте] Для posit из n бит с полем порядка es режим — это серия из k одинаковых битов, несущая масштаб $useed^k = 2^{2^{es}k}$ и занимающая $k+1$ бит. Считывание k из кодировщика для 2^e подтверждает $k = \lceil |e|/2^{es} \rceil + 1$ для всех проверенных e . Значащая часть, следовательно, теряет

$$\Delta M_{\text{posit}}(e) = \lfloor \frac{|e|}{2^{es}} \rfloor \text{бит: ступенчатая функция, арифметическая по } |e|.$$

Стандарт Posit (2022) фиксирует $es=2$ при любой точности (Густафсон), так что шаг — один бит на четыре бинады при всех ширинах; вот почему потеря выглядела как константа семейства 0.25 бита на бинаду. Измеренные наклоны -0.2497 и -0.2505 у posit16 и posit32 — это та же ступенчатая функция, увиденная сквозь линейную подгонку.

Теорема 4 (сужение takum, точно). [источник в тексте] Takum тратит фиксированное 3-битное режимное поле, хранящее r , после чего характеристика занимает r бит. Считывание r из кодировщика для 2^e даёт в точности $r = \lfloor \log_2(|e|+1) \rfloor$ — одинаково на 16, 32 и 64 битах. Значащая часть, следовательно, теряет

$$\Delta M_{\text{takum}}(e) = \lfloor \log_2(|e|+1) \rfloor \text{бит: ступенчатая функция, геометрическая по } |e|.$$

Следствие 2 (два семейства различаются родом, а не степенью). [источник в тексте] Внутри своего диапазона формат с фиксированными полями не теряет ничего; posit теряет один бит значащей части на каждые 2^{es} бинад без предела; takum теряет один бит на удвоение $|e|$. Сводим ΔM :

// e //	фикс. поля	posit (es=2)	takum
30	0	7	4
100	0	25	6
1,000	0	250	9
10,000	0	2,500	13

При $|e| = 1,000$ posit'у понадобилось бы 250 бит значащей части просто чтобы оплатить свой режим — вот почему 16- и 32-битные posit туда вообще не дотягиваются. Это и есть механизм, стоящий за заявленным превосходством takum над posit (takum), выраженный количественно.

Следствие 3 (темп — не всегда верная сводка). [источник в тексте] Оба закона — ступенчатые функции, так что подогнанный наклон есть сводка, а не сама вещь, и сводка честная лишь тогда, когда ступенчатость арифметическая. В более раннем черновике этой статьи мы сообщали -0.117 бита на бинаду для takum16 на $|e| \in [0, 30)$; тот же формат на $[0, 60)$ даёт меньшую цифру — без всякой причины, кроме окна, — а гладко-логарифмическая подгонка $s=0.75$ по той же причине занижает истинный шаг в один бит на удвоение. Сначала опознайте форму, потом выбирайте статистику — и лучше читайте кодировщик, чем подгоняйте кривую по его выходу.

Следствие 4 (трёхчастная таксономия, восстановленная без знания кодирования). [доказано] Диагностика разделяет форматы по форме $M(e)$: постоянная (фиксированные поля, TNF и GF по построению), линейная по $|e|$ (posit) или логарифмическая по $|e|$ (takum). Какая форма подходит, решают данные, а не чтение спецификации, — и подогнанные коэффициенты затем восстанавливают параметры кодирования: 2^{-es} для posit и ширину характеристики для takum.

То, что литература описывает качественно — больше точности вблизи единицы, меньше на краях (Густафсон), — оказывается, имеет точную форму, восстановимую двумя способами, и эти способы согласуются: из кодировщика, чтением режимного поля, и из одной лишь ошибки кругового преобразования, по диагностике теоремы 2. Второму пути не нужен доступ к спецификации, и именно это делает его полезным на каталоге.

Четыре формы ступенчатости — и только четыре

Классификация по *форме* ступенчатости, а не по подогнанному наклону, разрешает каталог на четыре класса. Из 83 записей 11 — целочисленные форматы или форматы с фиксированной запятой без поля порядка; из оставшихся 72 восемнадцать покрывают меньше шести бинад и вообще не классифицируются по ошибке кругового преобразования, а одна требует проб шире тех, что мы генерировали.

Таблица (каталог по форме ступенчатости). Параметр — константа сужения в единицах, естественных для каждой формы. Измерено относительно собственного пригодного диапазона каждого формата.

форма	M_{eff} против $\ e\ $	число	участники и константы
постоянная	плоская	38	IEEE, bfloat, VAX, Cray, x87, GF, TNF
колебание (wobble)	периодическая, без тренда	3	шестнадцатеричный IBM, $\log_2 16 = 4$ бита
арифметическая	-2^{-es} на бинаду	5	posit16/32 0.25, posit64 0.125
геометрическая	-1 на удвоение	8	takum $\times 4$, tekum $\times 3$

Четвёртого класса в нашей схеме не было, пока каталог его не породил, — и у него есть замкнутая форма.

Теорема 5 (колебание точности). [доказано] Пусть формат масштабируется как r^e , а значащая часть квантована равномерно по интервалу масштаба, покрывающему множитель r . Тогда его относительная ошибка меняется по этому интервалу в r раз, так что M_{eff} осциллирует с амплитудой ровно $\log_2 r$ бита и не имеет тренда по $|e|$.

Доказательство. Шаг квантования постоянен по интервалу, а относительная ошибка — это шаг, делённый на значащую часть, которая пробегает множитель r . Взяв $-\log_2$ от отношения крайних значений, получаем $\log_2 r$. Поскольку интервал повторяется тождественно при каждом e , осцилляция периодична и тренда не несёт. ■

Теорема 5 воспроизводит результат, который на шестьдесят лет старше этой работы. Про шестнадцатеричный формат IBM System/360 документировано, что он даёт от $4p-3$ до $4p$ значащих бит — разброс в

три (Голдберг 1991; документация IBM HFP); целочисленный счёт — это пол непрерывной величины, так что $\log_2 16 = 4$ из теоремы и учебниковая 3 — одно и то же утверждение. Измеренные на восьми подынтервалах — что за счёт усреднения внутри каждого бина предсказывает уменьшенный разброс 2.96, а не полные 4, — `ibm_hfp32` и `ibm_hfp64` дают 3.13 и 3.04, тогда как `binary32` и `gf32` дают 0.85 и 0.93 против предсказанных 0.87. Двоичные форматы колеблются на один бит; это и есть привычное однобитное колебание всякого float с основанием 2, и это наименьшее колебание, какое допускает целочисленное основание. Значит, теорема 5 даёт второй, независимый довод в пользу двоичной шкалы TNF — наряду с 0.331 позиции из теоремы о радикасе шкалы ниже: шкала с основанием 3 колебалась бы на 1.585 бита вместо 1.

Таксономия попутно решает и вопрос, почему формат обязан выбирать между первым классом и двумя последними — а значит, почему эта работа есть лестница, а не формат.

Теорема 6 (дихотомия «диапазон — точность»). [доказано]

Зафиксируем ширину N . Если порядок формата принимает неограниченно много значений, то кодовые слова порядка имеют неограниченную длину и $M_{eff}(e) \rightarrow 0$ при $|e| \rightarrow \infty$. Следовательно, ни один формат фиксированной ширины не обладает одновременно неограниченным диапазоном и точностью, ограниченной снизу положительной константой.

Доказательство. Кодовые слова порядка образуют префиксный код над двоичным алфавитом, так что по неравенству Крафта $\sum_e 2^{-\ell(e)} \leq 1$. Если сумма содержит бесконечно много слагаемых, то $\ell(e) \rightarrow \infty$. Полезная нагрузка, доступная значащей части, равна $N - 1 - \ell(e)$, значит, она падает до нуля, а по теореме 1 туда же падает и M_{eff} ■

Следствие 5 (зачем лестница). [доказано] Теорема 6 делает постоянную точность и неограниченный диапазон несовместимыми при любой фиксированной ширине, так что формат, желающий и того и другого, обязан получать второе за

счёт изменения N . Лестница из рунгов с фиксированными полями ровно это и делает: каждый рунг держит свою точность плоской по всему своему диапазону, а лестница в целом покрывает любой требуемый диапазон. Сужающиеся семейства делают противоположный выбор — один рунг, неограниченный диапазон, точность, затухающая с темпом их режимного кода: арифметически у *posit* и геометрически у *takum*. Ни один из выборов не лучше в абстракции; дихотомия устанавливает лишь то, что доступны только эти два.

Прочитанные вместе со следствием 5, три константы сужения из таблицы таксономии превращаются в утверждение о кодах, а не о форматах. Унарный режим стоит $\ell(e) \propto |e|$ и даёт арифметическую ступенчатость; двоичный режим с префиксом длины стоит $\ell(e) \propto \log_2 |e|$ и даёт геометрическую; фиксированное поле стоит $\ell(e) = \text{const}$ и не даёт ступенчатости вовсе — ценой ограниченного носителя. Форма ступенчатости — это темп роста длины кодового слова порядка, и ничего сверх того. Это же называет и пробел в таблице: между $\ell \propto |e|$ и $\ell \propto \log_2 |e|$ не сидит ни один каталогизированный формат. Раздел о кодах строит эти коды и спрашивает, что находится в пробеле; ответ поправляет предположение, которое мы высказали до того, как их построили.

Точность

Раздел «Зачем лестница» предсказывает ошибку рунга по ширине его мантиссы, не измеряя её. Этот раздел её измеряет — против того формата, ради сравнения с которым лестница и строилась. Метрика — средняя относительная ошибка на круговом преобразовании «кодирование — декодирование», 6 000 значений в диапазоне $2^{-38} \dots 2^{38}$ со случайным знаком, разбитых по величине двоичного порядка $|e|$.

Таблица (средняя относительная ошибка кругового преобразования). Бины — степени двойки, а не декады.

величина	GF16 (φ)	TNF16	tekum16	TNF16 против tekum16
// e // < 8	$3.43e-4$	$3.56e-4$	$3.27e-4$	$0.92 \times$ (ничья)
// e // 8–20	$3.57e-4$	$3.52e-4$	$1.00e-3$	$2.84 \times$
// e // 20–38 $6.98e-3$ (478 обрезано)		$3.53e-4$	$1.95e-3$	$5.53 \times$

Рисунок (tnf_accuracy.pdf). Точность по величине двоичного порядка. 6-битный порядок GF16 обрезает 478 из 2 857 значений в дальнем бине; уравновешенно-троичный порядок TNF16 — нет.

Про ось. Бины — степени двойки. Если читать их как *декады*, дальний столбец вообще не выигрыш: порядок TNF16 достаёт до ± 40 в степенях двойки, то есть примерно до ± 12 декад, а дальше он переполняется, тогда как неограниченный режим продолжает работать — по измерению, при таком прочтении обрезаются 1 458 значений дальнего бина. Мы указываем ось явно, потому что более ранняя внутренняя записка этого не сделала, и упущение наводит читателя ровно на ту интерпретацию, при которой результат выглядит сфабрикованным.

16-битное поле

Сравнение с одним-единственным действующим форматом напрашивается на возражение, что этот формат выбрали. Таблица ниже прогоняет через ту же нагрузку и ту же метрику каждый формат 16-битного класса, для которого этот репозиторий поставляет опубликованный оракул (каталог форматов). Значения, ушедшие в переполнение, считаются отдельно, а не подмешиваются в среднее: формат не должен выглядеть точным за счёт того, что выбросил трудные случаи.

Таблица (средняя относительная ошибка кругового преобразования по величине двоичного порядка, в скобках — число переполнений). 6 000 значений, $2^{-38} \dots 2^{38}$, случайный знак, одно семя.

формат	$\ e\ < 8$	$\ e\ 8-20$	$\ e\ 20-38$
TNF16	$3.56e-4$	$3.52e-4$	$3.53e-4$
GF16 (φ)	$3.56e-4$	$3.52e-4$	$6.76e-3$ (478)
takum16 / tekum16	$3.27e-4$	$1.00e-3$	$1.95e-3$
posit16	$1.36e-4$	$8.31e-4$	$1.43e-2$
IEEE binary16	$1.76e-4$	$1.28e-3$ (299)	$1.57e-1$ (2479)
bfloat16	$1.45e-3$	$1.38e-3$	$1.41e-3$
LNS16	$8.39e-4$ (1324)	$7.63e-4$ (1808)	$6.66e-4$ (2849)

Таблица. Вся лестница, измеренная в точных рациональных числах. Начиная с TNF16 и выше ошибка плоская по величине, а её порядок примерно удваивается с каждым рунгом. TNF4 и TNF8 показаны на своих пределах: нагрузка охватывает 76 двоичных декад, а эти рунги держат 2 и 8, так что бо́льшая её часть выходит за их границы — мы это приводим, а не опускаем, потому что именно на нижних рунгах видно, чем лестница платит за диапазон.

рунг	декады	$\ e\ < 8$	$\ e\ 8-20$	$\ e\ 20-38$
TNF4	2	<i>за пределами диапазона для этой нагрузки</i>		
TNF8	8	<i>за пределами диапазона</i>		
TNF16	24	$3.45e-4$	$3.69e-4$	$3.66e-4$
TNF32	219	$5.27e-9$	$5.63e-9$	$5.58e-9$
TNF64	658	$4.33e-17$	$4.22e-17$	$4.12e-17$
TNF128	1,975	$4.25e-36$	$4.55e-36$	$4.51e-36$
TNF256	5,925	$2.76e-74$	$2.69e-74$	$2.63e-74$
TNF512	17,775	$4.32e-151$	$4.62e-151$	$4.58e-151$
TNF1024	53,326	$2.84e-304$	$2.77e-304$	$2.71e-304$

Рисунок ([tnf_ladder_acc.pdf](#)). Те же данные. Плоскость по трём полосам величины — это ровно то свойство, ради которого строят формат с фиксированным полем; заштрихованные рунги — те, которые нагрузка перекрывает с запасом.

Рисунок ([tnf_competition.pdf](#)). Те же данные. Ножницами отмечены значения, которые переполнились.

Три вывода, включая тот, что против нас.

TNF16 плоский — и единственный, кто плоский без отсечения. Его ошибка равна $3.5e-4$ на любой величине в этой нагрузке, и он ничего не выбрасывает. bfloat16 тоже плоский и без отсечения, но примерно вчетверо хуже по ошибке. Все остальные участники либо деградируют

с ростом величины, либо переполняются: `binary16` делает и то и другое, теряя три порядка и выбрасывая 2 479 значений; шестибитный порядок `GF16` отсекает 478; `LNS16` отсекает везде.

На 32 битах двоичные форматы выигрывают вчистую. Та же нагрузка на 32-битном классе: `GF32` плоский на $3.4e-7$ без отсечения, но `IEEE binary32` плоский на $2.1e-8$, а `posit32` и `takum32` ещё лучше вблизи единицы ($2.1e-9$ и $4.9e-9$). Однородность перестаёт быть решающим свойством, как только битов хватает настолько, что ничего и так не отсекается. Аргумент `TNF` — это аргумент 16-битного класса, и мы его не распространяем дальше.

Вблизи единицы мы проигрываем. `posit16` с $1.36e-4$ и `binary16` с $1.76e-4$ там точнее `TNF16` — в два с половиной и в два раза. Формат с сужением тратит биты там, где значения обычно и находятся, и вблизи единицы это правильная ставка. `TNF16` покупает вместо этого однородность, и размен окупается только на нагрузке, которая уходит из окрестности единицы.

`takum16` и `tekum16` здесь — один и тот же формат, и это проверено исчерпывающе. Оба оракула дают одинаковые числа в каждом бине, поэтому мы проверили всё 16-битное кодовое пространство: все 65,536 кодов декодируются одинаково, ноль расхождений — хотя две реализации различаются на 242 строки. Реконструкция, которую реализует наш `tekum`-оракул, *и есть* `takum16`. Поэтому всякое сравнение в этой статье следует читать как сравнение с `takum16`, и мы его так и переименовываем. Это не более слабый якорь: `takum` опубликован, у него есть публичный VHDL-кодек, и он — родитель того формата, с которым мы изначально собирались сравниваться.

Железо

Синтез — Yosys 0.65 (см. ссылку на Yosys), размещение и трассировка — `nextpnr-xilinx` (1743d0f) на Xilinx XC7A200T, аппаратные умножители отключены, чтобы цифры описывали одну лишь фабрику. Генерация битстрима — через Project X-Ray. Ни на одном шаге не участвует вендорский инструмент, и именно это делает числа проверяемыми для читателя без лицензии.

Оговорка о версиях [проверено 2026-08-18]. Стенд репозитория на эту дату несёт **Yosys 0.63** (70a11c6b) и **nextpnr-xilinx c32135b0** — ни одна из двух версий, объявленных выше, на нём не установлена. Читатель, повторяющий метод на этом репозитории сегодня, получит другой тулчейн. Числа выше либо сняты на другой машине, либо объявление требует исправления; до выяснения таблица не является воспроизводимой на нашем же стенде, и мы говорим об этом прямо, а не оставляем читателю обнаруживать это самому.

Таблица. Умножитель TNF, после трассировки, один стенд на всю линейку, чтобы строки были сопоставимы. Задержка — один такт, пропускная способность — один результат за такт.

рунг	LUT	DSP48	$F_{\max}$
TNF4	12	0	161.11 МГц
TNF8	50	0	153.23 МГц
TNF16	212	0	131.73 МГц
TNF32	1,477	0	83.27 МГц
TNF64	6,140	0	53.26 МГц
TNF128	10,029	0	33.23 МГц

ОГОВОРКА W845 к строке TNF128 [противоречие внутри статьи]. Раздел «Ограничения» ниже по тексту утверждает, что TNF128 не сходится в разводке на этом кристалле и не заявляется, и что измерено пять рунгов из девяти, а не шесть. Два места статьи говорят разное об одном и том же рунге. Строка оставлена, но как результат она не стоит, пока противоречие не снято измерением; то же касается подписи к рисунку ниже, которая предполагает, что TNF128 измерен. Рунг TNF64 в этой таблице (6,140 LUT, 53.26 МГц) и в «Ограничениях» (7,479 LUT, 48.20 МГц) — вероятно, две разные сборки при разной ширине мантиссы, но файл этого не устанавливает, и здесь это не домысливается.

Статус кремниевых вердиктов [W845, пересчёт по правилу трёх сидов]. Из пятнадцати вердиктов «спека → кристалл», на которые опирается эта статья, под правилом T619a пересняты четыре: `ternary_node` (0x a5a5323f), `e8m0` (0x a5a531bf), `gft_sadd` (0x a5a5307f) и `ternary_link` (0x a5a532bf) — все четыре совпали на

трёх размещениях и подтверждены. `phi_weights` совпал на двух сидах из трёх и по правилу вердиктом пока не является. `tnf17` не удалось прочесть: защита по идентификатору конструкции отказалась называть вердикт, когда магию несли два кабеля. Остальные десять под правилом **не проверялись**, и до проверки каждый из них — утверждение об одном размещении.

Провенанс этой таблицы [измерено, одно размещение]. Каждая строка снята при **одном** сиде размещения `nextrnr`, и сид не записан. В августе 2026 на этом же стенде была проведена развёртка по сидам на неизменном нетлисте: достигнутая $F_{\max}$ прошла от 15.83 до 18.29 МГц, разброс около 15% при полностью зафиксированной логике. Отсюда следуют две вещи, и обе — про эту таблицу.

Во-первых, **вторая значащая цифра после запятой здесь не воспроизводима**: она описывает конкретное размещение, а не рунг. Числа следует читать как порядок величины и как монотонный спад по лестнице — то и другое разброс переживает, — но не как точные частоты.

Во-вторых, всякое сравнение, чья разница меньше разброса, этой таблицей **не поддерживается**. Это касается, в частности, пары 136.44 против 131.73 МГц ниже: разница в 3.6% лежит внутри 15%.

Столбцы LUT и DSP48 к этому не относятся: они выходят из синтеза, а не из размещения, и от сида не зависят.

Таблица. Специфицированная лестница. E_t — бюджет тритов, M — ширина мантиссы, диапазон — в декадах. Рунги выше TNF128 не влезают в одну фабрику Artix-7 и относятся к масштабу оракула или ASIC; они специфицированы и покрыты эталонным оракулом, но здесь не измерялись.

рунг	E_t	M	декады	рунг	E_t	M	декады
TNF4	2	1	2.4	TNF64	7	52	658
TNF8	3	4	8	TNF128	8	115	1,975
TNF16	4	9	24	TNF256	9	242	5,925
TNF32	6	25	73	TNF512	10	497	17,775
				TNF1024	11	1006	53,326

Рисунок (`tnf_ladder.pdf`). Площадь и достигнутая частота по всей лестнице.

Для контекста, а не как контролируемое сравнение: опубликованный Altera `ALTFP_MUL` на Cyclone IV даёт 119–132 МГц при задержке 6–10 тактов, занимая 832–1041 логический элемент и 18 встроенных умножителей, а работы по `posit`-умножителям сообщают о 95–572 LUT при логической задержке 4.28–15.55 нс. Другие кристаллы и другие тулчейны; мы не выводим отсюда никакой эквивалентности, кроме наблюдения, что цифры TNF16 укладываются внутрь этого конверта сразу по всем четырём осям — на фабрике, которая не даёт формату никакого троичного преимущества.

Отдельно про конвейерный регистр: комбинационный умножитель закрывается на 81.35 МГц, двухступенчатый — на 147.32 МГц. Разрез проходит между производением значащих частей и перенормировкой — умножение 10×10 , затем проверка переноса, насыщающее сложение порядков и выборка битов.

Два дефекта ширины и почему они обобщаются

Раздел о железе сообщает, во что обходится тракт данных, когда каждая шина выведена из собственных параметров формата. Изначально измеряли не так. Два дефекта ниже — один по площади, другой по корректности — суть одна и та же ошибка, ширина, выбранная вручную; мы о них сообщаем потому, что эта ошибка не специфична для TNF.

Ширина интерфейса доминировала над арифметикой

Наша первая синтезируемая реализация объявляла каждый порт и произведение значащих частей 32-битными. Ничего 32-битного в TNF16 нет: поле мантиссы — 9, значит $1+M$ равно 10, а их произведение ровно 20; смещение порядка никогда не превышает `OFFSET_MAX` = 80, то есть 7 бит. Синтез построил умножитель 32×32 и 32-битное дерево сравнения — и выставил за это счёт (см. рисунок ниже): 1 179 LUT, или три блока DSP48, против 219 LUT и нуля, когда шины соответствуют значениям, которые по ним идут. Арифметика при этом не изменилась.

Рисунок (`tnf_width.pdf`). Одна и та же арифметика при трёх ширинах интерфейса.

Та же самая ширина решала вопрос корректности

На TNF32 производству значащих частей нужны 52 бита. 32-битный провод усекает, и модуль выдаёт неверные результаты, при этом делая вид, что рунг поддерживается, — ведь его же собственная документация называет TNF32 поддерживаемой конфигурацией. По сравнению с 64-битным эталоном: 1 995 730 расхождений на 2 128 964 комбинациях.

Вывод всех ширин из параметров формата снимает обе проблемы разом. Мы сообщаем об этом потому, что схема не специфична для TNF: в параметрическом арифметическом ядре ширина, которая всего лишь *достаточно велика для той конфигурации, что была протестирована*, — это ловушка молчаливого усечения на следующей конфигурации, и её не поймает тестовый стенд, написанный под протестированную конфигурацию.

Эквивалентность

Каждое изменение выше — это утверждение, что изменились тайминг или ширина, а арифметика — нет. Каждое из них мы проверяли, а не доказывали словами.

Таблица. Свидетельства эквивалентности. Эталон для каждого рунга — тот модуль, которому для этого рунга уже доверяют.

проверка	комбинаций / тактов	расхождений
ширины из параметров против исходного, TNF16 (прогон)	321,156	0
ширины из параметров против исходного, TNF4	374	0
ширины из параметров против исходного, TNF8	3,716	0
ширины из параметров против исходного, TNF16	77,444	0
ширины из параметров против 64-битного эталона, TNF32	300,000	0
конвейерный против комбинационного	199,994	0
исходный 32-битный модуль против 64-битного эталона, TNF32	2,128,964	1,995,730

Прогон TNF16 исчерпывающий в том смысле, который здесь важен: пространство мантисс покрыто целиком при парах смещений, задействующих потерю значимости, середину диапазона и насыщение, а затем смещения покрыты целиком при мантиссах, дающих и не дающих перенос, — то есть все пути через перенос, насыщение и ограничитель потери значимости. Последняя строка — это и есть смысл раздела: модуль с выведенными ширинами согласуется с тем эталоном, который корректен, и расходится с тем, который усекает.

Что закон говорит об улучшении лестницы

Теорема о законе точности делает пространство проектных решений арифметикой, а не делом вкуса. Ошибка масштабируется как $2^{-(M+1)}$, а диапазон — как 3^{E_t} , так что при фиксированной ширине единственное решение — сколько битов позволено съесть полю порядка; а это решается тем, насколько эффективно 3^{E_t} значений упаковываются в $\lceil E_t \log_2 3 \rceil$ битов.

Таблица. Эффективность упаковки поля порядка. Теоретический пол — $\log_2 3 = 1.585$ бита на трит.

E_t	значений 3^{E_t}	b	ёмкость	эффективность
3	27	5	32	84.4%
4	81	7	128	63.3%
5	243	8	256	94.9%
6	729	10	1024	71.2%
7	2187	12	4096	53.4%
8	6561	13	8192	80.1%
10	59049	16	65536	90.1%

Соглашение о ширине считает позиции, и один рунг его нарушает. Ширина рунга — это число его позиций, причём трит занимает одну позицию, так что раскладка есть $1 + E_t + M = N$. Три из четырёх специфицированных рунгов удовлетворяют этому в точности: TNF4 — это $1 + 2 + 1$, TNF8 — $1 + 3 + 4$, TNF32 — $1 + 6 + 25$. А TNF16 — это $1 + 4 + 9 = 14$, на две позиции меньше, чем обещает имя, и ни один из файлов спецификации не объясняет почему. Вероятную историю видно по параметрам: TNF16 наследует у GF16 φ -оптимальное $M=9$ и заменяет шестибитный порядок GF16 четырьмя тритами, что покупает диапазон — $3^4=81$ шаг против $2^6=64$ — и освобождает две позиции, которые так и не были потрачены заново. Это единственный рунг, выведенный из своего родителя, а не из правила ширины.

Заодно это закрывает вопрос, во что рунг обходится на двоичной фабрике, — а это вопрос, отдельный от его ширины: трит, хранимый двухбитным кодом, делает TNF16 восемнадцатибитным, а смещение, хранимое обычным двоичным целым, — семнадцатибитным. Это стоимости фабрики, а не ширины формата, и именно путаница между этими двумя вещами создала впечатление, будто лестница не сходится.

Слабина не ограничена одним рунгом. Применение правила $1 + E_t + M = N$ ко всем специфицированным рунгам разводит два документа, определяющих лестницу. Четыре рунга машиночитаемой спецификации подчиняются ему на TNF4, TNF8 и TNF32; пять верхних рунгов, существующих только в исследовательской заметке, промахиваются на четыре-шесть позиций каждый, а заметка и спецификация ещё и противоречат друг другу насчёт TNF32.

Таблица. Правило ширины по всей лестнице. k — недобор $N - (1 + E_t + M)$. Столбец «согласовано» — это $M = N - 1 - E_t$, что ничего не меняет на тех трёх рунгах, которые правилу уже подчиняются.

рунг	источник	E_t	M	$1 + E_t + M$	k	согласованное M
TNF4	спец	2	1	4	0	1
TNF8	спец	3	4	8	0	4
TNF16	спец	4	9	14	2	11
TNF32	спец	6	25	32	0	25
TNF32	заметка	5	21	27	5	—
TNF64	заметка	7	52	60	4	56
TNF128	заметка	8	115	124	4	119
TNF256	заметка	9	242	252	4	246
TNF512	заметка	10	497	508	4	501
TNF1024	заметка	11	1006	1018	6	1012

Никакое одно альтернативное соглашение верхние рунги тоже не объясняет: чтение порядка как $\lfloor E_t \log_2 3 \rfloor$ битов точно подходит для TNF16, TNF64, TNF128 и TNF1024, но перебирает на единицу для TNF256 и TNF512 и полностью промахивается мимо TNF32. Мы заключаем, что верхняя часть лестницы была затабулирована рунг за рунгом, а не выведена, и что её согласование ничего не стоит на тех рунгах, которые уже правильны.

Чего стоили две свободные позиции и куда они делись. Поскольку порядок в теореме о законе точности сокращается, меню на шестнадцати позициях можно было прочитать ещё до измерений — и измерение подтвердило его до четвёртой значащей цифры:

Таблица. Три способа потратить две нераспределённые позиции TNF16. Ошибка и диапазон меняются друг на друга ровно так, как предсказывает закон; ни одна строка не хуже незаполненного рунга.

вариант	позиции	средн. отн. ошибка	декады	против takum16 (дальняя зона)
незаполненный, $E_t = 4, M = 9$	14	$3.37e-4$	24	$5.46 \times$
$E_t = 4, M = 11$ (принят)	16	$8.63e-5$	24	$22.68 \times$
$E_t = 5, M = 10$	16	$1.69e-4$	73	—
$E_t = 6, M = 9$	16	$3.37e-4$	219	—

Последняя строка — это собственная проверка закона на согласованность: добавление двух тритов порядка при неизменном M оставляет ошибку побитово той же и умножает диапазон на девять.

Позиции потрачены на мантиссу, и причина тому — измерение.

Следствие о том, что нельзя рекомендовать бюджет, не назвав нагрузку, запрещает такую рекомендацию — поэтому мы нагрузку назвали. На 16-битном классе лишний диапазон не расходуется: тензоры квантованного инференса охватывают примерно от 2^{-14} до 2^{14} , и $E_t=4$ уже ничего не отсекает на ± 40 бинадах — ноль отсечений на 2000 значениях. $E_t=6$ купило бы ± 364 бинады при побитово той же ошибке, а такой диапазон на шестнадцати битах никто не тратит. Значит, $M=11$: ошибка падает ровно в 4.00 раза, а против takum16 рунг перемещается с $0.92 \times / 2.88 \times / 5.49 \times$ на $3.70 \times / 11.21 \times / 22.68 \times$, больше не проигрывая ближний бин. В кремнии это стоит 443 LUT против 372, рост на 19%, при 136.44 МГц против 131.73 — никакого проигрыша по частоте.

Все три отношения в этом разделе и на рисунке точности берутся, по замыслу, из одного скрипта, `gen_figures.py`, который вычисляет их из оракулов при семени 20260809, а не переписывает вручную. **Этот скрипт описан, но не реализован.** Файла `gen_figures.py` нет ни в рабочем дереве, ни где-либо в истории репозитория: `git log --all --diff-filter=A -- gen_figures.py` не даёт ни одного коммита, который бы его добавлял. Поэтому приведённые здесь отношения и подпись к рисунку следует считать перенесёнными вручную, а описанную защиту рисунка от расхождения с таблицей — невозпроизводимой до тех пор, пока скрипт не будет написан. До 2026-08-09 они были зашиты в код и успели разъехаться: на рисунке всё ещё стояли $M=9$ и подпись `tekum16` — уже после того, как рунг сдвинулся, а оракул был показан декодирующим идентично takum. Рисунок, противоречащий таблице, которую он иллюстрирует, — это дефект того же класса, что и гейт, который не может загореться красным.

Сверку прогнали заново целиком, а не приняли на веру: круговое преобразование со знаком 2000/2000, кодирование монотонно, ноль инверсий, $\pm$ -симметрия — ноль расхождений на 500, обе границы диапазона декодируются. Тест `width_rule` теперь утверждает $1 + E_t + M = N$ прямо в самой спецификации, так что число позиций проверяется, а не помнится. Все девять рунгов ему удовлетворяют.

Почему троичность — и ровно насколько

Довод в пользу троичного порядка обычно сводят к ссылке на экономию по основанию системы счисления и на этом останавливаются. Мы его формулируем, а затем ограничиваем с двух сторон — сколько отбирает обратно двоичная фабрика и чего стоит основание *масштаба* в отличие от основания *порядка*, — потому что обе границы оказываются решающими для конструкции, и ни одна из них не фольклор.

Теорема (экономия по основанию). [доказано] Представление R различных значений в системе с основанием b стоит $b \log_b R = b \ln R / \ln b$ ячеек-состояний. Множитель $b / \ln b$ минимален при $b = e$; среди целых это $3 / \ln 3 = 2.731$ против $2 / \ln 2 = 2.885$, так что троичное поле на 5.3% экономнее двоичного.

Это классический довод (Хейс 2001; работы по экономии основания), и именно он мотивирует троичный порядок. Он также сформулирован в валюте — ячейках-состояниях, — которой никакая двоичная фабрика не платит. Следующая теорема говорит, что происходит, когда всё же платит.

Теорема (на двоичной фабрике диапазон бесплатным не бывает). [доказано] Поле порядка из E_t тритов охватывает 3^{E_t} значений. Хранимое как двоичное целое, оно занимает $\lceil E_t \log_2 3 \rceil$ битов, то есть поле, охватывающее $2^{\lceil E_t \log_2 3 \rceil} \geq 3^{E_t}$ значений. Следовательно, на двоичной фабрике троичный порядок никогда не охватывает больше значений на бит, чем двоичный, причём равенство достигается только при $E_t = 0$. Обозначая утилизацию $\rho(E_t) = 3^{E_t} / 2^{\lceil E_t \log_2 3 \rceil}$, имеем $\rho \in (1/2, 1]$, и ρ не сходится.

Таблица. Утилизация $\rho(E_t)$ двоичного поля, хранящего порядок из E_t тритов. Нынешние бюджеты лестницы, $E_t \in \{4, 7\}$, — две худшие записи в этом диапазоне.

E_t	2	3	4	5	6	7	8	9	10	11
ρ	56.2	84.4	63.3	94.9	71.2	53.4	80.1	60.1	90.1	67.6

Если запустить то же утверждение как задачу оптимизации, а не как неравенство, оно окажется острее, чем выглядит. Вопрос, какое кодирование порядка предпочитает двоичная фабрика при $(N, R) = (16, 24)$, $(32, 219)$ и $(64, 658)$, возвращает одну и ту же ширину мантиссы для обоих — $M = 10, 23$ и 53 соответственно. Троичное кодирование не проигрывает на двоичной фабрике; на этих ширинах округления вверх поглощают разницу, и получается ровная ничья — сильнейшая форма утверждения «никогда не больше значений на бит».

Теорема о том, что диапазон не бывает бесплатным, — причина, по которой ширины рунгов лестницы надо читать как счёт позиций, а не битов; и именно здесь наше собственное первое прочтение лестницы дало сбой: измерять, во что обходится трит, упакованный в биты, и называть результат шириной формата — значит путать стоимость фабрики со свойством формата. Независимо опубликованный сбалансированно-троичный float Ternary27 считает так же — 2 трита типа, 1 трит знака, 5 тритов порядка и 19 тритов значащей части, ровно 27, — и это внешнее подтверждение, что счёт позиций в этом семействе есть общепринятое соглашение, а не наше личное прочтение.

Теорема (нераспределённые позиции — чистая потеря).

[доказано] Пусть рунг из N позиций несёт знак, E_t тритов порядка и M битов мантиссы, причём $1 + E_t + M = N - k$, $k \geq 0$. Тогда относительно рунга с тем же E_t и $M + k$ его ожидаемая относительная ошибка больше ровно в 2^k раз, а динамический диапазон совпадает.

Доказательство. По теореме о законе точности ожидаемая относительная ошибка равна $\frac{1}{2} \mathbb{E}[1/s] 2^{-(M+1)}$, и порядок в неё не входит; замена M на $M + k$ умножает её на 2^{-k} . Динамический диапазон есть 3^{E_t} бинад — функция одного лишь E_t , значит он не меняется. Больше ничто в раскладке от M не зависит. ■

Эта теорема превращает слабину из раздела об улучшении лестницы в число без всяких измерений, а измерение затем это подтверждает. Против нагрузки, построенной в точной арифметике — необходимой выше TNF32, где двойная точность в качестве пробника меряет собственный порог разрешения, а не разрешение формата, — предсказанные и наблюдаемые множители таковы:

Таблица. Нераспределённые позиции по рунгам, потеря, предсказанная теоремой о нераспределённых позициях, и потеря, измеренная относительно согласованного рунга $M=N-1-E_t$. Нагрузка в точной арифметике, 800 проб на рунг.

рунг	k	предсказано	измерено
TNF16	2	$4\times$	$4.03\times$
TNF64	4	$16\times$	$15.43\times$
TNF128	4	$16\times$	$15.99\times$
TNF256	4	$16\times$	$15.24\times$

Последний вопрос — тот, на который лестница отвечает неявно и который, насколько мы смогли найти, никто не сформулировал: TNF кодирует порядок в основании 3, но масштабирует на 2^e , тогда как Ternary27 масштабирует на 3^e . Это разные решения, и они разделимы.

Теорема (основание масштаба, и почему 2 бьёт 3). [доказано]

Пусть формат масштабирует на r^e , а нормализованная значащая часть равномерно квантуется на $[1, r)$ с помощью M битов. Тогда его ожидаемая относительная ошибка на лог-равномерной нагрузке равна $\kappa(r)2^{-M}$, где

$$\kappa(r) = \frac{(r-1)^2}{r \ln r}, \kappa(2) = 0.72135, \kappa(3) = 1.21365.$$

При неизменной раскладке полей перенос масштаба с $r=2$ на $r=3$ умножает ошибку на $\kappa(3)/\kappa(2) = 1.6825$, умножая динамический диапазон на $\log_2 3 = 1.5850$. В позициях это стоит $\log_2 1.6825 = 0.7506$ точности и покупает $\log_3 1.5850 = 0.4192$ диапазона: чистая потеря 0.3314 позиции на число. Масштаб, кратный степени двойки, строго доминирует масштаб, кратный степени тройки, при равной ширине.

Доказательство. Шаг квантования на $[1, r)$ равен $(r-1)2^{-M}$, а средняя абсолютная ошибка округления — половина от него. На лог-равномерной нагрузке значащая часть имеет плотность $1/(\ln r)$ на $[1, r)$, так что $\mathbb{E}[1/s] = \frac{1}{\ln r} \int_1^r s^{-2} ds = \frac{1-1/r}{\ln r}$. Перемножая, получаем $\frac{1}{2}(r-1)2^{-M} \cdot \frac{1-1/r}{\ln r}$, а $\frac{1}{2}(r-1)(1-1/r) = (r-1)^2/2r$. Поглощая множитель $\frac{1}{2}$ соглашением $2^{-(M+1)}$ из теоремы о законе точности, получаем k в указанном виде. Что до диапазона: E_t тритов охватывают 3^{E_t} шагов по $\log_2 r$ бинад каждый. Один бит мантиссы вдвое уменьшает ошибку, а один трит порядка втрое увеличивает диапазон — это и задаёт обменный курс между двумя валютами и даёт числа в позициях. ■

Прямое измерение на двух рунгах, с кодированием и декодированием через оба масштаба в точной арифметике, даёт отношения ошибок 1.675 при $(E_t, M) = (4, 11)$ и 1.690 при $(6, 25)$ против предсказанного 1.6825, и отношения диапазонов 1.583 и 1.585 против предсказанного 1.5850.

Следствие (что такое TNF, если точно). [доказано] TNF — не троичный float в смысле Ternary27; это float с двоичным основанием, порядок которого закодирован в сбалансированной троичной системе. Теорема об основании масштаба говорит, что из двух доступных вариантов этот лучше на 0.331 позиции на число, а теорема о том, что диапазон не бывает бесплатным, говорит, что троичное кодирование окупается только на троичной фабрике. Вместе они очерчивают формат в точности: выбор основания верен на любой фабрике, а выбор кодирования — на одной.

Поле режима — это универсальный код

Теорема о дихотомии опознаёт кодирование порядка как префиксный код. Это опознание стоит проговорить явно, потому что оно помещает таксономию из раздела о таксономии внутрь иерархии, которая старше любого формата в каталоге на десятилетия (Элиас 1975).

Теорема (лестничная функция — это функция длины кодового слова). [доказано] Пусть формат из N позиций кодирует свой порядок префиксным кодом, приписывающим e кодовое слово длины $\ell(e)$. Тогда $M_{\text{eff}}(e) = N - 1 - \ell(e)$, а форма ступенчатой кривой — это скорость роста ℓ и ничего больше.

Три класса сужения из каталога оказываются тогда тремя точками классической иерархии: унарный режим, $\ell \propto |e|$, даёт арифметическую ступеньку *posit*; двоичный режим с префиксом длины, $\ell \propto \log_2 |e|$, даёт геометрическую ступеньку *takum*; фиксированное поле, $\ell = \text{const}$, не даёт ступеньки вовсе — ценой ограниченного носителя. Коды Элиаса γ , δ и ω занимают логарифмический класс со всё более тонкой асимптотикой.

Теорема (логарифм — это пол). [доказано] Для любого префиксного кода на целых числах $\ell(e) \geq \log_2 |e|$ для бесконечно многих e . Следовательно, ни один формат с неограниченным диапазоном не сужается асимптотически мягче, чем на один бит на каждое удвоение $|e|$.

Доказательство. Неравенство Крафта даёт $\sum_e 2^{-\ell(e)} \leq 1$. Если $\ell(e) < \log_2 |e|$ для всех e , кроме конечного числа, то хвост суммы превосходит $\sum_e 1/|e|$, который расходится. ■

Эту теорему стоит сформулировать прямо, потому что она ограничивает и нашу работу не меньше, чем конкурентов: режим *takum* асимптотически оптимален, и ни один формат — включая TNF — не может его превзойти на неограниченном диапазоне. Вместо этого TNF от неограниченного диапазона отказывается, а теорема о

дихотомии показывает, что другого варианта нет. Поэтому измеренные преимущества из раздела о точности — это преимущества *внутри диапазона TNF*, там, где сужение уже стоило *takum* битов, и их ни в коем случае нельзя читать как утверждение за его пределами.

Теорема (основание режима не имеет значения). [измерено]

Режим, тратящий одну цифру на каждое умножение $|e|$ на r , над алфавитом из r символов, стоит $\log_2 r \cdot \log_r |e| = \log_2 |e|$ битов — независимо от r . Форма ступеньки инвариантна к основанию режима.

Эта теорема отзывает конструкцию, которую мы предложили в более раннем черновике этой работы. Заметив разрыв между арифметическим и геометрическим классами, мы предположили, что режим, тратящий один трит на каждое утроение $|e|$, — естественный способ для троичного формата этот разрыв занять. Он его не занимает: если построить оба кода и нормализовать каждый так, чтобы выполнялось неравенство Крафта, две функции длины кодового слова отличаются аддитивной константой ровно 1 при каждом $|e|$ от 1 до 4096. Один трит на утроение — это ступенька *takum* с другой константой, а не третья форма. Троичная фабрика меняет то, во что режим обходится при декодировании; она не меняет того, во что он обходится при кодировании.

Разрыв реален, занимаем — и пуст по причине, которую мы можем только предполагать. Скорости роста строго между $\log_2 |e|$ и $|e|$ существуют и допустимы по Крафту — $\ell(e) = \lceil \sqrt{|e|} \rceil$ одна из них, — и такой код не доминируется. При $N=16$, с нормировкой, при которой все три являются законными префиксными кодами:

Таблица. M_{eff} по $|e|$ и полезный диапазон для трёх кодов режима при $N=16$, каждый нормирован так, чтобы выполнялось неравенство Крафта. Корневой код — настоящая парето-точка: больше точности, чем у логарифмического кода, во всём ядре, и больше диапазона, чем у унарного.

режим	$\ e\ = 1$	4	16	64	256	бинады
унарный, $\ell \propto \ e\ $ (класс posit)	11	10	7	–5	–53	43
$\ell = \lceil \sqrt{\ e\ } \rceil$	11	10	8	4	–4	121
$\ell \propto \log_2 \ e\ $ (класс takum)	9	7	5	3	1	511

Для сравнения: фиксированное поле, дотягивающееся до тех же 121 бинады, что и корневой код, тратит 8 битов на порядок и держит 7 битов плоско везде — лучше корневого кода при $|e| \geq 30$ и хуже ниже этой отметки, что и есть размен на сужение в миниатюре.

Итак, разрыв пуст не потому, что его обитатели доминируются, — они не доминируются. Нашим первым объяснением была стоимость декодирования, и постройка железа его опровергает.

Во что режим обходится в кремнии

Мы написали три кодека режима к одному интерфейсу — на вход 16-битное слово, на выход порядок и сдвиг мантиссы, плюс зеркальный кодер — и синтезировали каждый под единым стендом на XC7A200T с Yosys и nextpnr-xilinx, с отключёнными аппаратными умножителями. Это структурные модели трёх классов, а не воспроизведение опубликованных кодеков posit или takum; сравнивают они режим со сканированием серии против режима с префиксом длины — то есть ровно ту структурную разницу, которую называет таксономия.

Таблица. Стоимость кодека режима по классам, после трассировки на XC7A200T под одним стендом, без DSP. Цифры очищены от стенда, который стоит 24 LUT и был синтезирован отдельно и вычтен; столбец «круговой путь» — это декодер плюс кодер.

класс	декодирование, LUT	декодирование, $F_{\max}$	кодирование, LUT	кодирование, $F_{\max}$	круговой путь, LUT
унарный, $\ell \propto \ e\ $ (posit)	379	36.8 МГц	59	211.2 МГц	438
$\ell \propto \log_2 \ e\ $ (takum)	6	805.8 МГц	34	285.1 МГц	40
$\ell = \lceil \sqrt{\ e\ } \rceil$	393	33.1 МГц	123	96.4 МГц	516
фиксированное поле (GF, TNF)	0	—	0	—	0

Гипотеза гласила, что корневой код окажется слишком дорогим, чтобы его вообще строить. Нет: он стоит на 18% больше, чем круговой путь унарного режима, а унарный режим поставляется в продакшн. Что измерение действительно показывает — так это что дороговизна лежит не там, где мы предполагали, и у этого есть замкнутая форма.

Теорема (стоимость декодирования задаётся сканированием, а не ступенькой). [измерено] Режим, длина кодового слова которого ограничена серией, вынужден для декодирования просмотреть до $N-1$ позиций, что даёт стоимость $\Theta(N)$. Режим, длина которого записана в фиксированном поле из b битов, не требует сканирования и сдвига более чем на 2^b-1 , что даёт стоимость $\Theta(1)$ по N . Обе стоимости не зависят от скорости роста ℓ , а значит, и от формы ступеньки.

Эта теорема предсказывает таблицу в точности. Унарный и корневой коды делят одно и то же сканирование по 15 позициям и стоят 403 и 417 LUT — разница в 3.5%, и это извлекатель корня и ничего больше; код с префиксом длины не сканирует вовсе и стоит 30. Корневой код геометрически промежуточен и структурно идентичен унарному, и в этом всё дело: *форма ступеньки и стоимость декодирования — независимые оси*, и таксономия одной не предсказывает другую.

Следствие (логарифмический режим — не размен). [измерено] Против унарного режима режим с префиксом длины лучше по каждой измеренной оси: асимптотически оптимален по теореме о логарифмическом поле, в $11\times$ меньше по круговому пути и в $22\times$ быстрее в декодировании. Опубликованные варианты posIt с ограниченным режимом, которые обрезают поле режима именно ради того, чтобы избежать сканирования, — это тот же самый вывод, полученный с другой стороны.

Следствие (измеренная цена неограниченного диапазона). [измерено] У фиксированного поля кодека режима нет вообще. Поэтому у теоремы о дихотомии есть аппаратная формулировка:

неограниченный диапазон стоит 40 LUT в лучшем случае и 438 в худшем на этом кристалле — против 0 у формата, который от него отказался. Раздел о переводе кремния в точность переводит эти LUT в ту же валюту, что и результаты по точности, — только так и можно сказать, важны они или нет.

Мы остаёмся без объяснения пустому разрыву. Это не доминирование, поскольку корневой код парето-недоминируем; и это не стоимость, поскольку он в пределах 16% от режима, который поставляется. Оба наших объяснения теперь измерены, и оба неверны — и мы это фиксируем, а не подменяем третьим.

Перевод кремния в точность

Следствие об измеренной цене неограниченного диапазона даёт стоимость режима в LUT, а раздел о точности даёт отношение точностей, и сравнить эти две вещи нельзя, пока они не в одной валюте. А привести их к одной валюте можно: при фиксированном классе у бита мантиссы есть предельная площадь, а по теореме о законе точности — известная ценность в точности.

Теорема (площадь и точность соизмеримы). [измерено]

Пусть $\lambda = dA/dM$ — предельная площадь бита мантиссы для данного класса. Тогда кодек режима стоимостью C LUT эквивалентен C/λ битам мантиссы по кремнию, а по теореме о законе точности — множителю $2^{C/\lambda}$ в точности при равной площади.

Прямое измерение λ — тот же множитель с выведенными ширинами, синтезированный при $M=7, 9, \dots, 25$ под одним стендом — даёт 285, 372, 443, 567, 696, 870, 1238 и 1683 LUT. Предельная стоимость непостоянна: она растёт с 35.5 LUT на бит между $M=9$ и $M=11$ до 111.2 между $M=21$ и $M=25$. Беря локальное значение для каждого класса, $\lambda=48.8$ на 16-битном классе и 111.2 на 32-битном:

Таблица. Площадь режима, переведённая в биты мантиссы по теореме о соизмеримости. Унарный режим стоит — в одном лишь кремнии — больше, чем вся мантисса TNF16.

режим	LUT	битов на 16-битном классе	битов на 32-битном классе
унарный (класс posit)	438	8.98	3.94
с префиксом длины (класс takum)	40	0.82	0.36

Как растёт площадь вдоль лестницы

Теореме о соизмеримости нужна λ , а λ — производная закона площади, который мы можем сформулировать. Подгонка $A(M) = cM^a$ по восьми рунгам от $M=7$ до $M=25$ даёт $a=1.406$ при $R^2=0.984$, и вместе с теоремой о законе точности это предсказывает соотношение масштабирования:

Предложение (ошибка убывает как растянутая экспонента по площади). При $\varepsilon \propto 2^{-M}$ и $A \propto M^a$ имеем $\ln(1/\varepsilon) \propto A^{1/a}$.

Измеренная на $M \in [7, 25]$ — шестикратный размах по площади и 250,000-кратный по ошибке — константа пропорциональности держится в пределах 27% и монотонно дрейфует вниз, так что соотношение — приближение, и мы называем его предложением, а не теоремой. У дрейфа есть причина, и её поиск опроверг предсказание, которое мы зарегистрировали до синтеза.

Показатель непостоянен, и подгонка не экстраполируется. По подгонке на $[7, 25]$ мы предсказали, что согласованный TNF64 при $M=56$ будет стоить 4,762 LUT. Он стоит 7,479 при 48.20 МГц — предсказание занижено в $1.57\times$. Чтение a локально, а не глобально, объясняет почему: он растёт монотонно, с 1.06 между $M=7$ и $M=9$ до 1.78 между $M=15$ и $M=17$ и 1.85 между $M=25$ и $M=56$.

Теорема (закон площади лестницы). [измерено] Площадь рунга складывается из фиксированных накладных расходов — регистры, сумматор порядков, перенормировка — и умножителя мантиссы, квадратичного по M :

$$A(M) = f + cM^2.$$

Значит, эффективный степенной показатель $a(M) = d \ln A / d \ln M$ — вовсе не константа: он монотонно растёт к 2, а предельная цена одного бита мантиссы $\lambda = dA/dM = 2cM$ линейна по M , а не фиксирована.

Мы синтезировали один и тот же выведенный из ширины умножитель на четырнадцать значениях ширины мантиссы, от $M=7$ до $M=90$ — получилось 285, 372, 443, 567, 696, 870, 1238, 1683, 2601, 3990, 5934, 7479, 12232 и 19980 LUT, — и подгонка обоих параметров даёт $f=141$ LUT и $c=2,4455$ LUT на бит в квадрате при $R^2=0,99963$, с максимальным отклонением 9,9% на семидесятикратном диапазоне площадей. Если читать локально, a поднимается с 1,06 между $M=7$ и $M=9$ до 2,20 между $M=56$ и $M=70$ — и это именно теорема о законе площади лестницы, а не шум. На зарегистрированном заранее предсказании структурная модель точна до +4,4%, тогда как единый степенной закон занижил результат на 36,3%; мы приводим оба числа, потому что степенной закон был нашим собственным и предсказание мы опубликовали до синтеза.

Теорема (ценность поля режима в точности падает как $1/M$). [измерено] По теореме о конвертации и теореме о законе площади кодек режима фиксированной площади C стоит

$$\frac{C}{\lambda} = \frac{C}{2cM}$$

битов мантиссы — обратно пропорционально ширине мантиссы. Он стоит не меньше одного бита мантиссы ровно до тех пор, пока $M \leq C/2c$.

Таблица. Те же кодеки режима, оценённые на пяти значениях ширины мантиссы через теорему о $1/M$, при $c=2,4455$, измеренном на XC7A200T. Кремний унарного режима превосходит целую мантиссу TNF16; режим с префиксом длины остаётся ниже одного бита везде выше 8-битного класса.

M	λ (LUT/бит)	унарный, 438 LUT	с префиксом длины, 40 LUT
9	44,0	9,95	0,91
11	53,8	8,14	0,74
25	122,3	3,58	0,33
56	273,9	1,60	0,15
90	440,2	1,00	0,09

Следствие (ширины безубыточности). [измерено] При $c=2,4455$ унарный режим стоит не меньше одного бита мантиссы кремния вплоть до $M=89,6$ — то есть на всём протяжении 128-битного класса, — тогда как режим с префиксом длины дотягивает до одного бита лишь до $M=8,2$. Значит, режим *takum* обходится дешевле одного бита мантиссы кремния на каждом рунге выше TNF8, а режим *posit* обходится дороже одного бита на каждом рунге ниже TNF256.

Следствие о ширинах безубыточности — это точная форма того, что следствие об узких словах (ниже) могло сказать лишь качественно, и это самое резкое утверждение, какое мы можем сделать о том, где фиксированное поле окупает себя по площади: против *posit* — почти везде, против *takum* — только у самого низа лестницы.

Следствие (за диапазон платят трижды, и эти платежи соизмеримы). [измерено] Сужающийся формат платит за неограниченный диапазон в трёх валютах: позициями слова — в поле режима; кремнием — в кодеке; и точностью на краях — в самом сужении. Теорема о конвертации и теорема закона ширины переводят все три в биты мантиссы, так что их можно складывать.

Применённое честно, это следствие разделяет два сужающихся семейства, а не сваливает их в кучу. Против *posit* один только кремниевый член составляет 8,98 бита в 16-битном классе — больше, чем вся девятибитная мантисса TNF16, так что площадь, которую реализация *posit* тратит на декодирование своего режима, будучи потрачена на значащую часть, примерно удвоила бы бюджет точности

формата с фиксированными полями того же класса. Против `takum` кремниевый член равен 0,82 бита — это мало. Режим с префиксом длины дешёв, и любой довод в пользу фиксированного поля, построенный на площади, — это довод против `posit`, а не против `takum`. Против `takum` остаётся результат по точности (см. раздел о точности), ограниченный, как того требует теорема о пороге, собственным диапазоном TNF.

Следствие (довод о площади — это довод об узких словах).
[измерено] Поскольку λ растёт сверхлинейно по M , а стоимость кодека режима примерно фиксирована, один и тот же кодек конвертируется во всё меньшее число битов мантиссы по мере расширения класса: 8,98 бита на 16 превращаются в 3,94 на 32 для унарного режима. Площадное преимущество фиксированного поля, стало быть, сильнее всего там, где слова уже всего, — а это как раз режим, в котором работает квантованный инференс, а не тот, в котором работает численная линейная алгебра.

Закон значения стоит дороже, чем раскладка полей

Разделы о цене режима и о конвертации оценивали режим как поле, которое надо извлечь, — и синтез опубликованных декодеров показывает, что это меньшая половина вопроса. Мы берём декодеры `takum` и `posit` из существующего открытого набора верификации — Verilog, сверенный с эталоном на `mpmath`, — и прогоняем их через тот же стенд на XC7A200T.

Таблица. Опубликованные 32-битные декодеры, после трассировки на XC7A200T под одним стендом, без DSP, с конвертацией в `binary32`. Это настоящие кодеки, а не структурные модели из таблицы о цене режима.

декодер	LUT	RAMB36	$F_{\max}$	что доминирует
posit32_decode	480	0	49,0 МГц	подсчёт ведущих нулей, сдвиг
takum32_decode , опубликованный	76821	0	—	таблица 2^f , в логике
takum32_decode , тактированный	10967	84	11,4 МГц	та же таблица, в BRAM
TNF32, тракт порядка	0	0	—	поле читается напрямую

Строка *takum* приведена дважды, потому что одно число само по себе вводило бы в заблуждение. Её декодер вычисляет $2^{f_{hi}/2^{16}}$ по таблице 65536×48 бит, а затем применяет тейлоровскую поправку через 108-битное и 80-битное умножение. В опубликованном виде чтение таблицы комбинационное, а такое ни один синтезатор не отобразит на блочную память, — поэтому оно было построено из логики за 76821 LUT. Если это чтение зарегистрировать и пометить `ram_style="block"` — один лишний такт задержки, и это конвейеризованный вариант опубликованного модуля, а не сам модуль, — таблица попадает туда, где ей место: 10967 LUT и 84 блока RAMB36, что составляет 3,1 Мбит и 23% всей блочной памяти этого кристалла. Мы предсказали 85 блоков из геометрии таблицы ещё до синтеза, а измерили 84. Ни то, ни другое число не является свойством формата *takum*; оба они — свойства декодирования логарифмического закона значения в линейный тракт данных, и ни одно из них не про поле режима.

Теорема (линейные и логарифмические законы значения).

[доказано] Пусть закон значения формата есть $v = s \cdot 2^e$, где s и e читаются из полей (*линейный*), либо $v = \pm 2^\ell$ для декодированного вещественного ℓ (*логарифмический*). Линейный закон декодируется сдвигом, и его цена не зависит от ширины мантиссы. Логарифмический закон требует вычисления экспоненты, чья цена — это таблица либо итеративное ядро, и она не зависит от раскладки полей, а значит, не зависит и от ступенчатой формы из раздела о таксономии.

Эта теорема задаёт вторую ось, ортогональную первой. Ступенчатая таксономия классифицирует, как формат тратит *точность*; закон значения классифицирует, во что обходится его *чтение*. *posit* — сужающийся и линейный; *takum* — сужающийся и логарифмический;

LNS — несужающийся и логарифмический; GF, TNF и IEEE — несужающиеся и линейные. Оси независимы, а наше более раннее прочтение — что цена декодирования определяется тем, сканируется ли режим (теорема о сканировании), — описывает лишь член извлечения поля. Внутри этого члена оно остаётся верным, и таблица настоящих кодеков показывает, что для логарифмического формата деньги лежат не в нём.

Теорема (логарифмическое декодирование стоит по целевой точности, а не по ширине источника). [доказано]
Таблица на 2^k входов для 2^f с тейлоровской поправкой степени d даёт

$$P(k, d) = (d+1) \left(k + \log_2 \frac{1}{\ln 2} \right) + \log_2 (d+1)!$$

битов относительной точности, так что таблица занимает $P \cdot 2^k$ битов при $k = \frac{P - \log_2 (d+1)!}{d+1} - \log_2 \frac{1}{\ln 2}$: экспоненциально по целевой точности, и уменьшить это можно лишь подняв степень полинома, то есть обменяв память на умножители. Ширина исходного формата сюда не входит.

Доказательство. Остаток после обращения к таблице равен $x = f_{lo} \ln 2$ при $f_{lo} < 2^{-k}$, а тейлоровский остаточный член степени d для e^x равен $x^{d+1}/(d+1)!$. Взяв $-\log_2$, получаем заявленное P . ■

Проверенная против прямого моделирования схемы — таблица, остаток и полином, в точной арифметике, — формула держится с точностью $\pm 0,1$ бита в каждой точке сетки $k \in \{4, 8, 12, 16, 20\}$ на $d \in \{1, 2, 3, 4\}$, вплоть до 51-битного разрешения самого измерения.

Эта теорема фальсифицируема против набора верификации — и выживает. Декодер takum32 несёт $d=2$ и $k=16$, и $48/3=16$ ровно, где 48 — ширина выхода таблицы. Если бы цену задавала ширина источника, декодеру takum64 понадобилась бы таблица побольше; если её задаёт цель — та же самая. Он читает `takum32_2frac.mem` — *тот же самый*

файл — и выдаёт `binary32`, как и декодеры `takum8`, `takum16` и `takum64`. Одна таблица обслуживает четыре исходные ширины, потому что все четыре нацелены на один тракт данных.

Следствие (кривая обмена и во что на деле обходится каждая цель). [измерено] Теорема о целевой точности задаёт кривую, а не точку, и самая дешёвая точка на ней, влезаящая в данный кристалл, — величина вычислимая. Если оценить d умножений в $2,4455P^2$ LUT — квадратичный член нашего собственного измеренного закона площади — и потребовать, чтобы таблица влезла в 365 блоков RAMB36 этого кристалла:

цель	самая дешёвая допустимая точка	блочная память	LUT
<code>binary32</code>	$d=2, k=15$	43 блока	11269
<code>binary64</code>	$d=5, k=16$	187 блоков	134808

У XC7A200T 134600 LUT, так что декодирование логарифмического формата в тракт данных `binary64` стоит примерно целого кристалла.

Раньше мы писали, что декодирование в `binary64` «не является проектом» примерно при 3,6 Тбит. Эта цифра — угол кривой при $d=2$, и принять её за всю кривую было нашей ошибкой: при $d=5$ таблица занимает 6,9 Мбит, и проект существует. Защитимое утверждение — это следствие о кривой обмена: оно стоит кристалла, — а не то, что его нельзя построить.

Измеренный `takum32_decode` сидит на одну ступень k выше минимума для `binary32`, при $k=16$ против 15, что даёт двукратный запас по памяти и является обычной инженерной практикой, а не потерей. Линейному закону значения при любой целевой точности не нужны ни таблица, ни полином, потому что значащая часть уже находится в представлении тракта данных.

Следствие (во что обходится тракт порядка у TNF). [измерено] Закон значения TNF линеен, а поле порядка фиксировано, поэтому его тракт порядка — это провода: ни сканирования, ни таблицы, ни итераций. Против `posit32` это 480

непотраченных LUT; против takum32 — 3,15 Мбит неразвёрнутой таблицы. Это более крупное и более устойчивое утверждение, чем результат по точности, и, в отличие от него, теорема о пороге не накладывает на него никаких ограничений по диапазону.

Одну асимметрию мы отмечаем прямо, а не проходим мимо. Логарифмический закон значения покупает взамен нечто реальное: умножение превращается в сложение. Сравнение выше — это сравнение *декодеров*, то есть цены заведения значения в двоичный тракт данных, — и проект, остающийся в логарифмической области, её никогда не платит. Именно в таком режиме строят LNS-ускорители, и против них довод следствия о тракте порядка не работает вовсе.

Есть ли оптимальный формат — и в каком смысле

Приведённые выше результаты по отдельности ограничивающие: каждый говорит, чего формат иметь не может, или на что нельзя распространять то или иное утверждение. Прочитанные вместе, они сильнее этого. Они сводят пространство проектирования к четырём осям, три из которых допускают безусловные ответы, — так что, стоит назвать три входа, и формат оказывается определён, а не выбран.

Теорема (двоичная шкала — единственный реализуемый оптимум). [доказано] Пусть формат масштабируется как r^e . Масштабирование есть сдвиг — и, по теореме о законах значения, свободно от всякого полинома или таблицы — тогда и только тогда, когда $r=2^k$ при целом $k \geq 1$. На этом множестве $\kappa(r) = (r-1)^2 / (r \ln r)$ строго возрастает, принимая значения 0,7213, 1,6230, 2,9455 и 5,0720 при $r=2, 4, 8, 16$. Следовательно, $r=2$ минимизирует ожидаемую относительную ошибку среди всех оснований, чьё масштабирование реализуемо без трансцендентной функции, и делает это единственным образом.

Рисунок. $\kappa(r)$ в зависимости от основания шкалы. Безусловный минимизатор сидит около $r=1,9$ и недостижим: сдвигом масштабируются только $r=2^k$, а на этом множестве κ строго возрастает.

Утверждение проверено перебором, а не опирается на одну лишь монотонность κ . Прогоняя r непрерывно по $[1,1;16]$ при целом E и M , забирающем остаток ширины, на восьми комбинациях ширины и требуемого диапазона от $(P,R)=(8,24)$ до $(32,5925)$, лучшим реализуемым основанием во всех случаях оказывается $r=2$, а проигрыш относительно недостижимого оптимума составляет $1,09\times$ во всех восьми — константа, как и должно быть, поскольку они различаются лишь множителем $\kappa(1,9)/\kappa(2)$. Безусловный минимизатор — не 2: он около 1,9 и стоит 7,9% ошибки. Этот выигрыш недоступен, потому что любое $r\neq 2^k$ превращает r^e в трансцендентное вычисление, а следствие о кривой обмена оценивает такие вычисления в 84 блока RAMB36 — 23% этого кристалла — ради цели binary32. Менять 23% чипа на 7,9% бита мантиссы никто не станет. Поэтому теорема о двоичной шкале — результат об оптимальности интересного рода: оптимум здесь не экстремум, а лучшая точка из тех, что можно построить.

Следствие (три входа определяют формат). [измерено] В пространстве проектирования четыре оси, и лишь одна из них — свободный выбор.

1. *Основание шкалы.* Зафиксировано на 2 теоремой о двоичной шкале, безусловно.
2. *Основание кодирования порядка.* Задаётся фабрикой. Теорема об отсутствии дарового выигрыша показывает, что троичный порядок никогда не охватывает больше значений на бит, чем двоичный, если упаковывать его в биты, — а значит, он окупается на троичной фабрике и больше нигде.
3. *Фиксированные поля или сужение.* Это настоящий выбор, и по теореме о дихотомии вариантов ровно два: ограниченный диапазон при постоянной точности либо неограниченный диапазон при точности, убывающей не мягче чем на бит за удвоение (теорема о пороге).

4. *Разделение между E и M .* Определяется, как только назван требуемый диапазон, теоремой закона ширины — и *только* тогда, поскольку ошибка зависит лишь от M , а диапазон лишь от E .

Назовите фабрику, требуемый диапазон и ту сторону дихотомии, на которой сидит нагрузка, — и формат следует сам.

Итак, вопрос «существует ли идеальный формат» имеет ответ — при условии, что его задают со всеми тремя аргументами. Заданный без них, он не недоопределён, а некорректно поставлен: следствие о трёх валютах уже говорит, что бюджет нельзя рекомендовать без нагрузки, а теорема о дихотомии говорит, что ни один формат не может служить обеим сторонам сразу.

Следствие (чем это делает TNF). [измерено] Для аргументов (троичная фабрика; диапазон, который нагрузка действительно посещает; фиксированные поля) следствие о трёх входах возвращает число с плавающей точкой с двоичным основанием, полем порядка в сбалансированной троичной системе и равномерной мантиссой, отделяемой по $M=N-1-E_t$. Это и есть TNF, и он вынужден, а не спроектирован. Отсюда следует, что TNF оптимален ровно на этих аргументах и не несёт никаких утверждений вне их — в частности, никаких на двоичной фабрике, где ось 2 возвращает двоичный порядок, и никаких против сужающегося формата вне собственного диапазона TNF, где теорема о пороге это запрещает.

Там, где формат и есть тракт данных

Все аппаратные цифры до сих пор сравнивали декодер с трактом данных, который содержит умножитель мантисс. Такое сравнение льстит всем форматам одинаково, потому что умножитель в тысячи LUT амортизирует декодер в сотни. Есть нагрузка, где умножителя нет, — и она меняет не поля, а сам порядок мест.

В сети с троичными весами — в стиле BitNet, с весами из $\{-1, 0, +1\}$ (см. работу о BitNet) — произведение $w \cdot a$ есть выбор, отрицание или ноль. Умножитель не разворачивается ни для какого формата. Тракт данных выглядит как *декодировать, применить вес, сложить, накопить*, и декодер теперь сравнивается с сумматором.

Таблица. Один троичный нейрон, ширина против равной ширины, каждый бит аккумулятора наблюдаем, медиана по пяти семенам размещения, XC7A200T, без DSP. Каждый кандидат декодирует *упакованное* слово. За этими числами стоят две поправки. Более ранняя версия сообщала 440 против 895 LUT, отношение $5,1 \times$, — там TNF получал предварительно расширенные поля, а конкуренты распаковывали свои, и это снято. Следующая версия сообщала $3,1 \times$ на 16 битах и $5,6 \times$ на 32 — на модулях TNF, которые не реализовывали заданный формат. При заданных параметрах и при сверке с эталоном преимущество меньше — и оно настоящее.

ширина	формат	LUT	$F_{\max}$	МГц/LUT	отношение
8b	TNF8	545	62,13 МГц	0,1140	1,39 ×
8b	posit8	540	44,33 МГц	0,0821	—
16b	TNF16	565	66,28 МГц	0,1173	2,07 ×
16b	posit16	712	40,38 МГц	0,0567	—
32b	TNF32	569	66,91 МГц	0,1176	3,99 ×
32b	posit32	953	28,16 МГц	0,0295	—

$2,03 \times$ по площади и $2,54 \times$ по частоте, то есть $5,16 \times$ по пропускной способности на LUT — измерено сквозным образом, а не собрано из отдельных цифр по декодеру и сумматору. Разложение объясняет почему: против умножителя в 1477 LUT и сумматора в 384 LUT декодер posit на 456 LUT составляет 20% тракта данных; без умножителя — уже 54%. Для takum тот же сдвиг переводит 85% в 97% и приносит с собой 84 блока RAMB36.

Следствие (убрать умножитель — значит повысить декодер в должности). [измерено] Пусть тракт данных стоит $C = D + M + A$ — декодер, умножитель и сумматор. Нагрузка, устраняющая M , поднимает долю декодера с $D/(D+M+A)$ до $D/(D+A)$, а отношение между двумя форматами, различающимися только D ,

— с $1 + \Delta / (D_0 + M + A)$ до $1 + \Delta / (D_0 + A)$. Значит, преимущество формата без декодера растёт ровно по мере того, как умножитель сжимается, и максимально там, где умножителя нет вовсе.

Три вещи, которых это не говорит, — и каждую иначе нашли бы за считанные минуты.

Это не значит, что TNF избегает умножения. Он платит за умножение значащих частей, как и всякий другой формат, когда оно нужно; квадратичный член его собственного закона площади *и есть* этот умножитель. То, что убирает троичная сеть, убрано для сети, а не для формата.

Это не значит, что TNF лучше для ПЛИС вообще. Мы синтезировали декодирование порядка TNF против обычного двоичного фиксированного поля — и они измеряются одинаково, по 32 LUT каждое, ровно как того требует теорема об отсутствии дарового выигрыша. Преимущество в таблице троичного нейрона — это преимущество над *сужающимся* форматом, а не над форматом с фиксированными полями: поле в стиле IEEE встало бы рядом с TNF, а не рядом с posit.

И это не измерение на троичной фабрике. Такой фабрики не купить, а перед нами двоичная ПЛИС, где наша же теорема об отсутствии дарового выигрыша говорит, что троичное кодирование не даёт ничего. Ему и не нужно: выигрыш здесь идёт от отсутствующего декодера и был бы таким же для любого формата с фиксированными полями.

Что структурно добавила бы троичная фабрика. На фабрике, чьи позиции хранят триты, двоичное поле порядка можно хранить либо плотно, в $\lceil b / \log_2 3 \rceil$ тритах, либо складываемо, по одному биту на трит, — но не одновременно. Плотную форму нельзя складывать без конверсии основания; складываемая форма теряет $1 - 1 / \log_2 3 = 36,9\%$ каждой позиции, что для ширин порядка из этого каталога составляет от 33 до 57% поля. Сбалансированный троичный порядок не платит ни того, ни другого. Это структура, а не измерение, и это единственное

место, где троичное кодирование из теоремы об отсутствии дарового выигрыша перестало бы быть даровым и начало бы быть бесплатным по цене.

Лестница — это семейство, а семейство — это фронтир

Приведённые выше результаты трактуют каждый рунг как формат. Если вместо фиксации бюджета порядка прогнать его по всем значениям, меняется то, что именно утверждается, — и утверждение усиливается.

Теорема (правило ширины порождает фронтир). [измерено]

Зафиксируем ширину N . Множество $\mathcal{F}_N = \{ (E_t, M) : 1 + E_t + M = N, E_t \geq 2, M \geq 1 \}$ содержит $N-3$ члена, каждый из которых точен по построению. По теореме закона ширины член E_t имеет эффективную мантиссу $M = N - 1 - E_t$ и динамический диапазон $3^{E_t} - 1$ бинад, так что члены полностью упорядочены и ни один не доминирует другого: $\mathcal{F}_N$ — дискретный фронт Парето в координатах $(M_{eff}, \text{диапазон})$.

Отсюда следует утверждение не про один рунг, а про весь каталог.

Следствие (ни один каталогизированный конкурент не выживает). [измерено]

В каждом измеренном классе ширины каждый каталогизированный формат этой ширины строго доминируется некоторым членом $\mathcal{F}_N$: найдётся член, у которого эффективная мантисса не меньше и диапазон не меньше, а хотя бы по одному из двух — строго больше.

Измерено против эталонов: двадцать восемь конкурентов в пяти классах, с указанием доминирующего члена в каждом случае:

Таблица. Доминирование по классам ширины. Для самого трудного случая в каждом классе указан член семейства, который его доминирует. Эффективная мантисса измерена; диапазон равен $3^{E_t}-1$, что для фиксированного поля аналитично.

класс	конкурентов	выживших	самый трудный случай доминируется
8	7	0	$E_t=4, M=3$ против <code>fp8_e5m2</code>
16	7	0	$E_t=6, M=9$ против <code>takum16</code>
32	9	0	$E_t=5, M=26$ против <code>posit32</code>
64	4	0	$E_t=10, M=53$ против <code>cray_float</code>
128	1	0	$E_t=10, M=117$ против <code>binary128</code>

Почему это не артефакт плотности. Достаточно плотное семейство доминирует любое конечное множество точек тривиально, поэтому содержание — в том, *почему* конкуренты сидят под фронтиром, а не на нём. Каждый проигрывает как минимум по одному из трёх пунктов, и каждый пункт — одна из теорем выше. Сужающийся формат измеряется ниже своей заявленной мантиссы — по теореме о пороге. Формат, оставляющий позиции нераспределёнными, платит за них 2^k — по теореме о распределении, и именно это делала историческая лестница GF-T на 2, 4 и 6 позициях. А двоичный порядок при равном числе позиций охватывает меньше бинад, поскольку трит несёт $\log_2 3$ бита против одного бита у бита. Семейство не проигрывает ни по одному из трёх, потому что правило ширины запрещает второе, а кодирование улаживает третье.

Чего это не говорит. Пара $(M_{eff}, \text{диапазон})$ — это свойство числа, и доминирование в ней не есть доминирование в кремнии. Раздел о троичной сети измеряет те же форматы через один тракт данных и ставит TNF в середину группы форматов с фиксированными полями, на 0,114–0,118 МГц на LUT, как только каждый рунг измерен при своих заданных параметрах. Против `int8` эти два неразличимы, как только прогнаны семена (см. таблицу отвергнутых утверждений). Одно другому не противоречит: фронтир спрашивает, что несёт слово данной ширины, а тракт данных спрашивает, во что обходится его распаковка, и всякое фиксированное поле распаковывается чтением полей. Разрыв в кремнии проходит между фиксированными и сужающимися форматами, от 2,4 до 6,4×, и принадлежит теореме о сканировании и теореме о законах значения, а не этой лестнице.

Заявленный предел измерения. Для фиксированного поля диапазон аналитичен, но проверить его на границе удаётся не всегда: при $E_t=49$ край диапазона равен $2^{1,4 \times 10^{23}}$, чего никакая точная арифметика построить не может. Выше примерно $E_t=30$ диапазон — факт о кодировании, а не измеренная величина, и так он и сообщается.

Какие именно первые принципы

Правило ширины излагалось как правило. Но это не правило: это решение задачи максимизации с ограничениями, и именно название ограничения делает так, что остальная часть статьи следует, а не накапливается.

Правило есть условие Каруша — Куна — Таккера. Запишем задачу проектирования прямо. Максимизировать точность при бюджете позиций и диапазоне, который надо покрыть:

$$\max E, MM \text{ при } 1+E+M=N, r^E \geq b+1.$$

С множителями λ на бюджет и μ на диапазон лагранжиан $\mathcal{L} = M + \lambda(N-1-E-M) + \mu(E - \log_r(b+1))$ даёт $\partial_M \mathcal{L} = 1 - \lambda = 0$ и $\partial_E \mathcal{L} = -\lambda + \mu = 0$, откуда $\lambda = \mu = 1$. Поскольку $\mu > 0$, условие дополняющей нежёсткости делает ограничение по диапазону *активным*: в оптимуме порядок ровно настолько широк, насколько требует диапазон, и ни на позицию шире. Это и есть теорема об оптимальном члене, теперь выведенная, а не постулированная. Множитель — это ещё и теневая цена, и $\lambda = 1$ говорит, что позиция порядка стоит ровно позицию мантиссы: обменный курс единичен, потому что бюджет линеен.

Почему три, а не два, — результат 1950-х, а не наш. Экономичность системы счисления $\rho(r) = r/\ln r$ минимальна при $r=e$. Троичная сидит на 0,46% выше этого оптимума, двоичная — на 6,15% выше. Мы не претендуем тут ни на какую заслугу; вклад настоящей работы — в том, где это применяется, и это оказывается всем примирением наших двух осей:

$$r \sim \text{цена позиции} \times \log_r V \sim \text{число позиций}.$$

Числовая ось считает только второй множитель, и там $\log_3$ против $\log_2$ даёт запас 0,3691E из следствия о запасе — безусловно. Кремниевая ось возвращает первый множитель, и упаковка трита в двоичную фабрику

берёт $\kappa(3)/\kappa(2)=1,68$ за позицию, — вот почему BNF16 и TNF16 синтезируются с разницей в пределах 1%. Теорема об отсутствии дарового выигрыша и следствие о запасе не противоречат друг другу: это два множителя одного произведения, сообщённые по отдельности.

Что здесь вывод, а что лишь сходство. Три связи буквальные и используются как таковые: приведённый выше аргумент с множителями; экономичность системы счисления, которая фиксирует основание порядка; и неравенство Крафта, ограничивающее сужение в теореме о пороге. Ещё два сходства стоит назвать лишь затем, чтобы их не приняли за аргументы. Вариационная форма напоминает принцип наименьшего действия, но здесь нет ни времени, ни траектории — задача статическая, и сходство не влечёт никаких следствий. А $1+E+M=N$ — это бюджет, а не закон сохранения: позиции распределяет проектировщик, а не удерживает природа. Мы проговариваем это различие, потому что физически звучащий вывод делал бы риторическую работу, которую аргумент с множителями уже делает честно.

Стандарт выпадает сам — при одном выборе процентиля и не при других. Применённый к блочной оси, аргумент с множителями возвращает элементный формат, но какой именно — зависит от того, какая доля блочного распределения защищается, а этот выбор в более ранней версии данного раздела остался непроговорённым.

Таблица. Правило ширины на блочной оси. Измеренный внутриблочный размах задаёт $E^* = \lceil \log_2 b \rceil$, и ответ движется вместе с защищаемым процентилем.

защищаемый процентиль	размах, бинад	E^*	элемент
50-й (медиана)	1,89	1	E1M2
90-й	2,45	2	E2M1
99-й	3,04	2	E2M1
99,9-й	3,75	2	E2M1

Защита медианы возвращает E1M2; защита 90-го процентиля и выше возвращает E2M1 — тот самый элемент, который фиксирует спецификация микромасштабирования ОСР. Поэтому корректное утверждение — это порог, а не вывод: *правило ширины возвращает элемент MX для любого проекта, защищающего 90-й процентиль блоков или больше, и другой элемент ниже этого порога.* Что верно, — вопрос о

том, скольким блокам позволено насыщаться, а не вопрос, решаемый из первых принципов, и мы говорим это прямо, вместо того чтобы цитировать один лишь согласующийся случай.

Абзац. В применении к блочной оси это перестаёт быть абстракцией. Внутриблочный размах весов обученной сети, измеренный до всякого выбора формата, составляет 1,89 бинады на медиане и 3,04 на 99-м процентиле. Оптимум равен $E^* = \lceil \log_2 b \rceil$, так что защита медианы возвращает E1M2, а защита 99-го перцентиля возвращает E2M1 — тот самый элемент, который фиксирует спецификация микромасштабирования ОСР. Стандарт воспроизводится не по совпадению: это единственное решение задачи с множителями при том хвосте, который у этих весов действительно есть. Теорема о сожалении говорит, какой из двух должен победить, поскольку недоразмер платит на хвосте линейно, а переразмер платит логарифмически на каждом значении, — а раздел о смежных работах по блочной оси приводит измерение, которое это решает.

Правило ширины — это оптимизация с единственным решением

Теорема о покрытии сравнивает с заданным конкурентом. Сформулированное как задача проектирования, правило выбирает члена напрямую, — и именно это делает его фальсифицируемым: оно называет победителя до всякого измерения.

Теорема (оптимальный член для названного диапазона).

[доказано] Пусть нагрузка посещает b бинад. Среди всех членов $\mathcal{F}_N$, представляющих её без переполнения, эффективная мантисса максимальна — и единственным образом — при

$$E_t^* = \lceil \log_3(b+1) \rceil, M^* = N-1 - \lceil \log_3(b+1) \rceil.$$

Доказательство. Представимость требует $3^{E_t} - 1 \geq b$, то есть $E_t \geq \log_3(b+1)$, а E_t целое, поэтому $E_t \geq E_t^*$. По правилу ширины $M = N-1-E_t$ строго убывает по E_t , значит ограничение активно, и минимизатор единствен. ■

Итак, у правила нет свободного параметра, как только названа нагрузка, а название нагрузки — это измерение, а не выбор. Раздел о троичной сети использует это как предсказание: сначала измеряется размах бинад весов сети, из него вычисляется E_t^* , и только затем перплексия оценивается по всему семейству. Другой победитель фальсифицировал бы правило.

Теорема (сожаление при неверно названном диапазоне). [измерено] Размер, рассчитанный на b' , когда нагрузка посещает b , стоит

$$R = \{ \lceil \log_3(b+1) \rceil - \lceil \log_3(b'+1) \rceil \text{ битов} \\ \text{мантиссы, } b' > b, \text{ переполнение на множестве} \\ \text{меры } 1 - \frac{\log_3(b'+1)}{\log_3(b+1)}, b' < b. \}$$

Штраф асимметричен: переразмер стоит точности логарифмически, недоразмер стоит диапазона линейно, — поэтому, когда посещаемый диапазон неизвестен точно, правило следует округлять вверх.

Асимметрию стоит назвать, потому что она противоречит интуиции, будто более широкий порядок — консервативный выбор. Он и правда консервативен, но цена платится в каждом значении, которое формат когда-либо представит, тогда как цена излишней узости платится только на хвосте. Что перевесит — эмпирический вопрос о весе хвоста, а не дело вкуса.

Следствие (когда трит окупает свою упаковку). [измерено]

Троичность покупает $\Delta = E(1 - \log_2 3)$ позиций и стоит $\kappa(3)/\kappa(2) = 1,683$ за позицию при упаковке в двоичную фабрику, где $\kappa(r) = (r-1)^2 / (r \ln r)$. Эти два эффекта взаимно гасятся при E , удовлетворяющем $\Delta = 0,331E$, — так что на позициях трит выигрывает вчистую, а на двоичных битах выходит в ноль; вот почему BNF16 и TNF16 синтезируются с разницей в пределах 1%, хотя на числовой оси фронтир их разделяет.

Четыре семейства на двух осях

TNF — это один угол квадрата, и квадрат стоит нарисовать, потому что два правила, задающие его оси, были выведены порознь и по несвязанным причинам.

Одна ось — это *правило, фиксирующее бюджет порядка*. Правило золотого сечения берёт $E = \text{round}((N-1)/\varphi^2)$ и порождает лестницу GF; правило ширины берёт $1+E+M=N$ и порождает лестницу BNF. Другая ось — это *основание кодирования порядка*: двоичное или сбалансированное троичное. Их пересечение даёт четыре семейства — GF и GF-T на правиле золотого сечения, BNF и TNF на правиле ширины, — и каждое является самостоятельным форматом, а не переименованием другого.

Таблица. Четыре семейства на восьми битах, одна нагрузка, одна метрика. SmolLM2-135M на wikitext-2, 40 окон по 2048 токенов, тензорный масштаб, базовая линия 14.4874 проверена в коридоре до всякого сравнения. Предсказания были напечатаны до того, как посчитали перплексии.

семейство	правило, основание	величин	ppl	против fp32
эталон fp32	—	—	14.4874	1.000×
GF8 , $E=3$	золотое сечение, двоичное	129	14.6130	1.009×
GF-T8, $E_t=3$	золотое сечение, троичное	109	15.5147	1.071×
BNF8 , $E=3$	правило ширины, двоичное	129	14.6130	1.009×
TNF8 , $E_t=2$	правило ширины, троичное	73	14.7012	1.015×

Два имени, одно кодирование, два правила. Углы квадрата — не четыре несвязанных формата. GF и BNF делят двоичное кодирование и различаются лишь тем, какое правило фиксирует E ; GF-T и TNF делят троичное кодирование и различаются ровно тем же. А насколько правила потом расходятся — это измерение, и по большей части они не расходятся вовсе.

Таблица. Лестницы GF-T и TNF, рунг за рунгом. Восемь из девяти специфицированных рунгов совпадают — оба правила выбирают один и тот же бюджет — и расходятся они только на 32 позициях.

рунг	GF-T (E_t, M)	TNF (E_t, M)			
4	2	1	2	1	совпадают
8	3	4	3	4	совпадают
16	4	9	4	9	совпадают
32	5	21	6	25	единственное расхождение
64	7	52	7	52	совпадают
128	8	115	8	115	совпадают
256	9	242	9	242	совпадают
512	10	497	10	497	совпадают
1024	11	1006	11	1006	совпадают

Вот почему кремниевые таблицы несут строки TNF и не несут отдельных строк GF-T: на любом рунге, кроме 32, это были бы одни и те же числа дважды. И вот почему восьмибитное измерение ниже даёт GF-T8 и «TNF8 при $E_t=3$ » одной строкой с 109 величинами и 15.5147 — это один и тот же формат, до которого дошли с двух сторон. BNF тоже измерен, 97 LUT и 388.35 МГц, но на стенде с изолированным декодером (см. раздел об аппаратуре), а не на датапути троичного нейрона из таблицы полной пропускной способности, где он теперь и появляется — строкой 2, измеренной на том же стенде, что и всё остальное.

Два правила называют один и тот же формат, и ни одно его не предсказало. GF8 при $E=3$ и BNF8 при $E=3$ дают 14.6130 — то же самое число, не близкое, а то же, потому что на восьми битах оба вывода выбирают идентичную раскладку: $E=3$ и $M=4$. Это утверждение сильнее, чем каждое из правил по отдельности, и мы не собирались его находить.

А предсказание правила ширины было опровергнуто дважды. Оно называло BNF8 при $E=4$ и TNF8 при $E_t=3$; измеренные победители — $E=3$ (14.6130 против 14.6592) и $E_t=2$ (14.7012 против 15.5147). Оба предсказания оказались на шаг шире, чем нужно. *Форма* правила выживает и видна в развёртке: единственный оптимум, в обе стороны от него хуже, и штраф асимметричен ровно так, как требует теорема о штрафе, — $E=1$ даёт 4.5 миллиона, $E=2$ даёт 288, тогда как $E=4$ стоит 0.3%, а $E=5$ — 6%. Неверной оказалась оценка диапазона, который посещает нагрузка, а не правило, эту оценку потребляющее: мы мерили размах от 0.1-го перцентиля до максимума, а это засчитывает

хвост, не несущий почти никакой энергии. Записано, а не тихо подкручено, — потому что правило, называющее победителя до измерения, может ошибиться, и это правило ошиблось.

Троичность проигрывает на двоичной фабрике и здесь, уже в третий независимый раз. GF-T8 несёт 109 величин там, где GF8 несёт 129, и платит 15.51 против 14.61; TNF8 при $E_t=2$ несёт 73. Три измерения теперь сходятся с разных сторон: BNF16 и TNF16 в пределах 1% в размещённом кремнии, GF8 против GF-T8 здесь и MXFP4 против TNF4 на блочной оси (см. раздел о блочной оси). Троичный порядок зарабатывает на позициях и платит на битах — ровно так, как требует теорема об отсутствии бесплатного сыра, — а эта фабрика берёт плату битами.

Два формата закрывают троичный узел

В троичном узле ровно три места, где мог бы жить числовой формат, и только одному из них он нужен.

1. **Вес** — это код. Умножение на него есть выбор знака (+ x , $-x$, 0), а не умножение.
2. **Отсчёт** приходит в родном виде с АЦП и используется таким, каким пришёл.
3. **Аккумулятор** — единственный объект в системе, несущий динамический диапазон, за который надо платить.

Вот и вся задача проектирования, и она меньше, чем кажется по литературе. Опубликованные троичные методы отвечают на (1) и оставляют (3) в `fp16` или `int8`; опубликованные работы по форматам отвечают на (3) и предполагают множитель в (1). Мы отвечаем на оба — двумя форматами, выведенными независимо друг от друга и оказавшимися дополняющими.

Почему алфавит весов — это φ , и почему иначе нельзя

GF Ternary — двухбитный алфавит $\{-\varphi, 0, +\varphi\}$. Это φ не украшение, и дальше сказано, почему это единственный возможный выбор.

Теорема (Золотой алфавит единствен). [доказано] Пусть трёхсимвольный алфавит весов есть $\{-r, 0, +r\}$ с $r > 1$, и потребуем, чтобы произведение двух весов выражалось в той аддитивной решётке, в которой датапуты уже считает, то есть $r^2 = r + 1$. Тогда $r = \varphi$ и это единственная возможность.

Доказательство. У $r^2 - r - 1 = 0$ корни $(1 \pm \sqrt{5})/2$, из которых положителен один. ■

Следствие (Глубина не стоит умножителя). [доказано] Усиление стопки из k троичных φ -слоёв в точности равно

$$\varphi^k = F_k \varphi + F_{k-1},$$

паре целых чисел, где F — последовательность Фибоначчи. Пересчёт масштаба между слоями поэтому есть сдвиг-и-сложение, и свойство «ноль умножителей» доживает до произвольной глубины.

Это практическое содержание теоремы о золотом алфавите, и именно оно отделяет алфавит φ от алфавита $\{-1, 0, +1\}$, которым пользуется вся литература по троичным сетям. При единичных весах усиление слоя равно 1 и не несёт никакой информации, поэтому каждый опубликованный метод подвешивает обучаемый вещественный масштаб a_ℓ на слой — а умножение на a_ℓ возвращает умножитель на место. При алфавите φ межслойный масштаб известен точно и равен паре целых, так что на него никогда не умножают. Хранение в обоих случаях — два бита, число символов в обоих случаях — три; алфавит φ просто несёт масштаб, который единичному алфавиту приходится выучивать, а потом за него платить.

Линейный путь точен

Теорему о золотом алфавите обычно читают как «убирает умножитель». Вместе с ним она убирает и округление: на том алфавите, который она фиксирует, весь линейный путь сети вычисляется точно.

Представим значение целочисленной парой (a, b) , означающей $a + b\varphi$. Тогда

$$\varphi(a + b\varphi) = a\varphi + b\varphi^2 = a\varphi + b(\varphi + 1) = b + (a + b)\varphi,$$

так что применение веса — это отображение $(a, b) \mapsto (b, a + b)$, рекуррента Фибоначчи, одно целочисленное сложение, без сдвига. Отрицание переворачивает обе компоненты, а нулевой вес — это пропуск.

Теорема (Точность пути без умножений). [доказано] $\mathbb{Z}[\varphi] = \{a + b\varphi : a, b \in \mathbb{Z}\}$ — кольцо, и алфавит весов $\{-\varphi, 0, +\varphi\}$ лежит внутри него. Следовательно, для входов из $\mathbb{Z}[\varphi]$ вся линейная часть троичной сети — каждое применение веса и каждое накопление, при произвольном фан-ине и произвольной глубине — остаётся в $\mathbb{Z}[\varphi]$ и вычисляется без какой бы то ни было ошибки округления.

Доказательство. Замкнутость по сложению покомпонентна. Замкнутость по умножению следует из $\varphi^2 = \varphi + 1$, что сводит любое произведение к указанному отображению целочисленных пар. Аккумулятор поэтому есть пара точных целочисленных регистров. ■

Измеренная на слое с фан-ином 512, с весами из этого алфавита, целочисленная пара воспроизводит вещественную сумму с точностью проверяющей арифметики, а не с точностью датапути: в датапути просто нечему быть неточным. Ширины компонент растут логарифмически, доходя до восьми бит на этих 512 слагаемых.

Теорема, сверенная с кремнием. [измерено] Теорема о решётке ничего не говорит о том, реализует ли RTL эту решётку, а шесть модулей в этой работе, как выяснилось, реализовывали не тот формат, которым были подписаны. Поэтому синтезированный узел сравнивается с точной целочисленной арифметикой в $\mathbb{Z}[\varphi]$, и ни одного числа с плавающей точкой в сравнении нет: пара (a, b) датапути против пары эталона, как целые.

Шестнадцать тысяч векторов при фан-ине 8, 16 и 32, восьмибитные отсчёты, веса взяты равномерно из алфавита: **ноль расхождений**. Глубина проверена так же — утверждение ведь про глубину, а не только про один слой: один шаг, заведённый сам на себя, — это k уложенных слоёв, и после k тактов пара обязана быть (F_{k-1}, F_k) . Тридцать глубин, ноль расхождений; при $k=30$ усиление равно ровно $(514229, 832040)$, два целых числа, и на всём пути нигде нет умножения.

Первый прогон разошёлся на 3,601 из 4,000 — только в компоненте b , при том что a совпадала всегда. Это подпись различия в соглашении, а не в арифметике, и ошибочное кодирование было у эталона. Форма расхождения подсказывает, кого подозревать: «всё неверно в одной компоненте» — соглашение, «неверно на краях» — пропущенный граничный случай, «разбросано» — арифметика.

Именно замкнутость убирает всю машинерию

Датапуть без умножений не нов как цель, и опубликованные попытки делают роль теоремы о точности видимой по контрасту.

Ближе всего FQR — процессор фибоначиевой квантизации, превращающий произведения фибоначиево-квантованных весов в сложения. Ради этого он несёт два разных арифметических блока — сумматор дуалистического преобразования для больших операндов и бит-исключающий сумматор для малых — и схему маршрутизации в топологическом порядке, раскладывающую данные по ним. Fibbinary-квантизация для нейросетевых радиоприёмников идёт по той же подложке более мягким путём и сообщает об экономии 45% и 44% по мощности и площади умножителя: умножитель сделан дешевле, но не убран.

Машинерия в первом случае — не артефакт реализации. Она следует из свойства самого числового множества.

Следствие (Замкнутость убирает стадию преобразования).

[доказано] Пусть A — алфавит весов, $a s$ — операция применения веса. Если аддитивная группа, порождённая A , замкнута относительно s , датапути нужно только сложение этой группы. Если не замкнута, каждое произведение приходится заново

выражать в представимом множестве, а для этого нужны стадия преобразования и расписание, маршрутизирующее к ней операнды.

Доказательство. Немедленно из теоремы о точности и её отрицания. Для $A = \{-\varphi, 0, +\varphi\}$ порождённая группа есть $\mathbb{Z}[\varphi]$, а $s = \varphi \cdot$ отображает $(a, b) \mapsto (b, a+b)$, что попадает внутрь неё, — значит, сложения достаточно. Для A — множества фибоначиевых целых $\{F_k\}$ — произведение $F_i F_j$ в общем случае не является фибоначиевым целым ($F_4 F_4 = 9$ не лежит в $\{1, 1, 2, 3, 5, 8, 13, \dots\}$), поэтому произведение выходит из представимого множества и должно быть преобразовано обратно в него. ■

Сколько всего масштабов без умножения

Замкнутость объясняет, почему φ работает. Сама по себе она не говорит, что φ — единственный выбор, и честный ответ: не единственный — он единственный в своём размере.

Масштаб r применяется без умножителя ровно тогда, когда r — алгебраическое целое, у которого сопровождающая матрица имеет элементы из $\{0, \pm 1\}$: если записать $r^d = a_{d-1}r^{d-1} + \dots + a_0$, то умножение на r переводит координатный вектор $(x_0, \dots, x_{d-1})$ в $(a_0 x_{d-1}, x_0 + a_1 x_{d-1}, \dots)$, и любое $|a_i| > 1$ снова вносит умножение на константу. Условие конечно, поэтому масштабы можно перечислить, а не спорить о них.

Теорема (Масштабы без умножения, по степеням).

[доказано] При коэффициентах из $\{0, \pm 1\}$: - **Степень 1** допускает $r \in \{0, \pm 1\}$, что есть отсутствие масштабирования, и $r = \pm 2^k$, бесплатное сдвигом. Один регистр, отношение 2. - **Степень 2** допускает ровно два уравнения с вещественным корнем модуля, отличного от 0 и 1: $r^2 = r + 1$ и $r^2 = -r + 1$, корни которых — φ и φ^{-1} , то есть одна и та же лестница. Одно сложение, два регистра, отношение 1.618. - **Степень 3** допускает три:

$r^3=r+1$ (пластическое число, 1.3247), $r^3=r^2+1$ (суперзолотое, 1.4656), $r^3=r^2+r+1$ (трибоначчи, 1.8393). Не более двух сложений, три регистра.

Доказательство. Полный перебор по девяти тройкам коэффициентов степени 2 и двадцати семи — степени 3; дискриминанты a^2+4b для степени 2 отрицательны в четырёх случаях, а оставшиеся пять дают корни из $\{0, \pm 1\}$, кроме случая $b=1, a=\pm 1$. ■

Следствие. [измерено] Между сдвигом и φ нет ничего. φ — единственное измельчение лестницы степеней двойки без умножения, доступное при двух регистрах.

Итак, масштаб без умножения — это не точка, а иерархия, индексированная алгебраической степенью, и каждая степень покупает более тонкую гранулярность ценой ещё одного регистра. Таблица ниже измеряет этот обмен на первых двух рунгах, где он вообще является обменом. Пластический шаг $(a, b, c) \mapsto (c, a+c, b)$ стоит того же единственного сложения, что и шаг Фибоначчи, плюс один лишний регистр, а лестница его тоньше: соседние уровни отличаются на 13.97% против 23.61% у φ .

Таблица. Одно применение масштаба, $xc7a200t$, медиана по пяти семенам. Сопоставимо: ни в одном блоке нет мультиплексора направления. Ошибка уровня равна $(r-1)/(r+1)$ — худший относительный зазор на лестнице с отношением r .

масштаб	LUT	FF	$F_{\max}$ (МГц)	ошибка уровня
φ , 16 бит	149	128	307.69	23.61%
пластическое, 16 бит	186	160	318.47	13.97%
φ , 32 бита	223	192	247.10	23.61%
пластическое, 32 бита	228	200	231.21	13.97%

На 32 битах измельчение стоит 2.2% площади и 6.4% частоты. Это достаточно близко к бесплатному, чтобы выбор стал вопросом про сеть, а не про арифметику, — вот мы и спросили сеть.

Какой рунг брать и как ответ смещается

В таблице ниже каждый двумерный линейный вес SmolLM2-135M квантуется к $\pm r^k$ с масштабом на выходной канал, кодовая книга имеет ровно 2^b записей, включая ноль и знак, и оценивается перплексия на wikitext-2 по двенадцати окнам в 2048 токенов. Базовая линия fp32 равна 14.36.

Таблица. Перплексия по лестнице и кодовому бюджету, SmolLM2-135M на wikitext-2. Грубые лестницы перекрывают диапазон и тратят точность впустую; тонкие делают наоборот. Лучший рунг спускается вниз по иерархии по мере роста бюджета.

бит	сдвиг (2)	ϕ (1.618)	суперзолотое (1.4656)	пластическое (1.3247)
3	2309.86	41835.53	1367268.11	6720092.29
4	76.81	24.43	25.10	55.72
5	77.52	22.73	18.93	16.45

Та же развёртка на Qwen2.5-0.5B — другая архитектура, в четыре раза больше параметров, другой токенизатор, другие обучающие данные — возвращает того же победителя на каждом бюджете и тот же полный порядок на каждом бюджете: сдвиг 2384.96, затем ϕ **16.24** против суперзолотого 17.98 и пластического 47.41, затем пластическое **13.17** против суперзолотого 14.09 и ϕ 15.33, при базовой линии fp32 12.27. На четырёх битах более крупная модель разводит ϕ и суперзолотое на 10.7% там, где меньшая дала 2.7% и никакого разделения.

Лестница с отношением r и n величинами перекрывает r^{n-1} , так что более тонкие шаги покупаются размахом. На трёх битах ничего не перекрывает достаточно, и побеждает самая грубая; на четырёх баланс приходится на ϕ ; на пяти самая тонкая из проверенных выигрывает вчистую. Это закон «диапазон против точности» (см. раздел о законе) в применении к иерархии масштабов, а не к раскладке полей, и он прямо говорит: ϕ оптимально при определённом бюджете, а не везде. На четырёх битах две модели зажимают сравнение с двух сторон: SmolLM2 разводит ϕ и суперзолотое на 2.7%, чего двенадцать окон не разрешают, а Qwen разводит их на 10.7%, что окна разрешают. ϕ там в худшем случае вничью и впереди на большей модели, а суперзолотое в любом случае стоит регистра. Битом выше пластическое число

выигрывает на обеих, на 28% и 14%, а φ третье из четырёх — и к этому моменту мы уже проехали и ту часть иерархии, которую можно себе позволить.

Для выбора сеть не нужна. Ранжирование лестниц по среднеквадратичной ошибке квантования одних только весов воспроизводит порядок по перплексии в точности на трёх и на пяти битах, а на четырёх переставляет лишь ту пару, которую перплексия и не различает.

Теорема (Закон лестницы). [измерено] Среди лестниц без умножения при фиксированном кодовом бюджете та, что минимизирует среднеквадратичную ошибку весов, минимизирует и перплексия — с точностью до пар, чья ошибка различается меньше, чем разрешает оценка.

Сама ошибка имеет замкнутую форму, так что рунг следует из гистограммы. Нормировка каждого канала на его максимум ставит уровни в r^{-k} при $k=0, \dots, n-1$ с $n = (2^b - 1)/2$, и

$$MSE(r, b) \approx c(r)^2 \mathbb{E}[x^2 \mathbf{1}_{x>t}] + \mathbb{E}[x^2 \mathbf{1}_{x<t}], t = \frac{1}{2} r^{-(n-1)},$$

где $c(r)^2$ — средняя квадратичная *относительная* ошибка округления геометрической лестницы с отношением r : одномерный интеграл по лог-позиции внутри бина, не зависящий от данных. Первое слагаемое падает с падением r , второе растёт вместе с порогом отсечения; минимум — это точка их пересечения. Против измеренной ошибки на 106, 168, 320 нормированных весах форма сходится в пределах 1–9% на четырёх и пяти битах и выбирает ту же лестницу на всех трёх бюджетах. На трёх битах она занижает на треть, и причину стоит назвать: при семи кодах режим — это абсолютное усечение, а не относительное округление, а модель построена на втором.

Что такое четырёхбитная перестановка — и чем она не является

Форма выше ранжирует верно на трёх и пяти битах и переставляет верхнюю пару на четырёх, выбирая суперзолотое там, где перплексия выбирает φ . Перестановка воспроизводится на обеих моделях в одну и

ту же сторону, значит, это эффект. Мы проверили два кандидатных механизма; один провалился, другой устоял, и пара информативнее любого из них по отдельности.

Заметность активаций: провал. Ошибка на выходе слоя равна $\sum_j \delta w_{ij} x_j$, так что вес, умножающий большой входной канал, должен стоить дороже, — а каналы далеко не равны: первая проекция внимания охватывает $28 \times$ между своим наибольшим входным каналом и медианой на SmolLM2-135M и $22.4 \times$ на Qwen2.5-0.5B. Именно на этом наблюдении построен AWQ. Ранжирование по $\sum_j a_j^2 \delta w_{ij}^2$ переворачивает четырёхбитную пару на SmolLM2, совпадая с перплексией, — и не переворачивает на Qwen, где взвешивание масштабирует все четыре кандидата почти ровно на 1.09 и потому не может их переупорядочить. Объяснение, которому нужна корреляция канальной заметности с картиной ошибки, не является общим, потому что в одной модели из двух её нет.

Кривизна лосса: держится. Величина, о которой на самом деле идёт речь в этой перестановке, — не то, насколько велик операнд, а то, насколько сдвигается лосс, когда сдвигается вес. Ранжирование по диагональному фишеровскому взвешиванию $\sum_{ij} g_{ij}^2 \delta w_{ij}^2$ с $g = \partial L / \partial w$ переворачивает четырёхбитную пару к ϕ на *обеих* моделях и оставляет пять бит за пластическим на *обеих* — совпадая с перплексией во всех четырёх случаях.

Теорема (Поправка на кривизну). [измерено] Среди лестниц без умножения при фиксированном кодовом бюджете минимизатор $\sum_{ij} g_{ij}^2 \delta w_{ij}^2$ предсказывает ранжирование по перплексии там, где невзвешенная ошибка этого не делает, и делает это на *обеих* проверенных моделях. Поправка сдвигает выбор в сторону более грубой лестницы.

Два результата вместе говорят то, чего не сказал бы один только успешный. Величина активации — это половина градиента, и не та половина, что решает: $g_{ij} = \delta_i x_j$ несёт и вход, на который умножается вес, и чувствительность лосса к выходу этого нейрона, и именно второй множитель заставляет поправку воспроизводиться. Метод,

откалиброванный на одних активациях — а несколько опубликованных именно таковы, — пользуется прокси, который совпал с целью на одной из двух наших моделей.

Теорема (Оптимальное отношение). [измерено] Для заданного распределения весов и кодового бюджета оптимальное отношение без умножения есть минимизатор $MSE(r, b)$, а требуемая степень — наименьшая, чей минимальный масштаб его не превосходит. Минимизация по непрерывному r даёт $r^* = 2.054, 1.4115, 1.1999, 1.1050, 1.0576$ и 1.0319 при 3–8 битах, причём $\ln r^*$ уполовинивается на каждый бит, поскольку удвоение числа кодов уполовинивает доступный лог-шаг.

Блочная ось и та её половина, что принадлежит нам

Всё сказанное выше касается поканального масштабирования. Другая развёрнутая в продакшене ось делит один масштаб на короткий блок, и там наши форматы проигрывают. Стоит сказать, насколько основательно, потому что ответ здесь — не ранжирование, а потолок.

Ллойд–Макс на внутриблочном распределении — 18,850,950 значений, блок в 32 вдоль оси свёртки, общий масштаб E8M0, как предписывает спецификация MX, — даёт восьмиуровневую кодовую книгу, минимизирующую квадратичную ошибку. Она не свободна от умножения и не является лестницей; это лучшее, что вообще могут сделать восемь величин.

Таблица. *Блочная ось, SmolLM2-135M, 40 окон, базовая линия fp32 14.4874. Оптимальная кодовая книга улучшает квадратичную ошибку почти в пятнадцать раз, а перплексию — на 0.9%, и всякая лестница, обыгрывающая MXFP4 по первому, проигрывает по второму.*

элемент, 8 величин	квадратичная ошибка	× границы	rpl
Ллойд–Макс (оптимум, нереализуемый)	1.341×10^{-3}	1.00	22.2976
MXFP4 (E2M1)	2.007×10^{-2}	14.97	22.4998
суперзолотое/ пластическое, 4 тонких шага	2.068×10^{-3}	1.54	27.3079
суперзолотое/ пластическое, 3 тонких шага	2.116×10^{-3}	1.58	30.8306
φ/суперзолотое, 2 тонких шага	2.749×10^{-3}	2.05	38.6471
φ/пластическое, 2 тонких шага	2.823×10^{-3}	2.11	55.4495

Теорема (Блочная ось — не задача про формат элемента). [измерено] На коротком блоке с общим масштабом кодовая книга элемента, минимизирующая квадратичную ошибку квантования, — не та, что минимизирует перплексию, а зазор между наилучшей достижимой кодовой книгой и уже развёрнутой составляет меньше одного процента. Усилия, потраченные на проектирование форматов элемента для этой оси, не смогут отыграть больше.

Ранжирование не просто не переносится — оно инвертируется. Каждая двухсегментная лестница здесь обыгрывает MXFP4 по квадратичной ошибке в три–десять раз, и каждая проигрывает по перплексии на 21%–146%.

Причина — в том, что такое блок. Тридцать два веса делят один масштаб, а квадратичная ошибка засчитывает отсечённый малый вес по его собственной величине, — поэтому кодовая книга, сосредоточившая уровни у верхнего края диапазона, набирает хороший балл, разместив разрешение там, где масса. Но сеть не читает веса блока по отдельности: они суммируются против активаций, и кодовая книга, тонко разрешающая верх каждого блока и схлопывающая его нижние две трети, уничтожает способность блока представлять *направление*, а не только его величины. E2M1 тратит 0,0.5,1,1.5,2,3,4,6: размах $12 \times$ с уровнями на обоих концах. Она

плохо набирает по квадратичной ошибке именно потому, что отказывается концентрироваться, — и по той же самой причине выигрывает по перплексии.

Мы записываем это как результат конкурента, а не наш. Три наших собственных атаки здесь провалились — троичный порядок в плавающем формате, лестницы с постоянным отношением и двухсегментные лестницы, построенные в подражание расстановке E2M1, — и граница объясняет, почему четвёртая не оправдана.

Вторая половина блочной оси

Граница закрывает формат элемента. Про масштаб она не говорит ничего, а масштаб — это одно значение на тридцать два веса, и его никогда не варьировали. Его цена амортизируется тридцатидвухкратно, а это как раз тот режим, где более тонкая сетка по карману, — и бухгалтерия решает сравнение ещё до того, как измерена хоть одна перплексия.

E8M0 даёт масштабу восемь бит порядка. На 3,317,760 блочных масштабах реально занятый диапазон — 8.32 бинады на SmolLM2-135M и 9.12 на Qwen2.5-0.5B, так что четырёх бит хватает. На четырёх битах 2^k нужно 8.3 шага из шестнадцати доступных, а φ^k — 12.0, при вдвое меньшем шаге. Выбор при равной ширине — ровно между этими двумя.

Таблица. Сетка блочного масштаба, $K=32$, элементы E2M1, 40 окон. При равных битах φ^k обыгрывает 2^k на обеих моделях; более тонкие сетки стоят одного-двух лишних бит, и их выигрыш не воспроизводится.

сетка масштаба	b	SmolLM2	Qwen
2^k (как в спецификации MX)	4	22.4998	14.9447
φ^k	4	21.3545	14.8512
$\sqrt{2}^k$	5	21.8960	14.7456
пластическое k	5	22.7625	15.0333
$2^{k/4}$	6	24.0791	14.4328

Более сильное прочтение первого столбца не выдерживает второго. На SmolLM2 отклик немонотонен, с минимумом на φ , а $2^{k/4}$ падает ниже обычного 2^k , что говорило бы, что φ — оптимум сетки; на Qwen отклик монотонен по гранулярности и выигрывает самая тонкая сетка. Знак

воспроизводится, а величина — нет (5.09% против 0.63%), и утверждение об оптимальности не воспроизводится вовсе. Мы заявляем только то, что воспроизводится.

Фронтир, а не пара. Масштаб — это одно значение на K весов, то есть он стоит *биты/К* на вес, а сравнение форматов попарно это скрывает. Таблица ниже кладёт перплексию против полного числа бит на вес. Множество Парето на обеих моделях одно и то же.

Таблица. Фронтир «цена масштаба», элементы E2M1, 40 окон. φ^k на четырёх битах на блок из 32 строго доминирует MXFP4 — дешевле и лучше на обеих моделях. Выше 4.25 бита на вес фронтир принадлежит масштабам, несущим мантиссу.

схема	биты/вес на масштаб	всего бит/вес	SmolLM2	Qwen
φ^k 4b/64	0.0625	4.0625	22.0978	15.0090
φ^k 4b/32	0.1250	4.1250	21.3545	14.8512
2^k 4b/32	0.1250	4.1250	22.4998	14.9447
MXFP4 , E8M0 8b/32	0.2500	4.2500	22.4998	14.9447
E4M3 8b/32	0.2500	4.2500	19.8628	13.7636
NVFP4-подобное E4M3 8b/16	0.5000	4.5000	18.5445	13.5340

Отсюда следуют три вещи, и только третья наша.

Поле масштаба у MXFP4 шире, чем нужно. То, что диапазон E8M0 превышает занимаемый весами, замечали и до нас, и мы не претендуем на приоритет наблюдения; здесь измерено, насколько именно и чего стоят освободившиеся биты. 2^k на четырёх битах воспроизводит перплексию E8M0 в точности — 22.4998 и 14.9447, до последней цифры, — потому что занятый диапазон составляет 8.32 и 9.12 бинады, и восемь бит порядка не покупают ничего сверх четырёх.

А на этих четырёх битах φ^k обыгрывает 2^k , так что φ^k 4b/32 строго доминирует MXFP4: дешевле на 0.125 бита на вес и лучше на обеих моделях. Это не обмен.

И верхний фронтир достигается тратой большего. E4M3 — масштаб, несущий мантиссу, именно такой использует NVFP4 — даёт 19.8628 и 13.7636 при тех же 4.25 бита на вес, где сидит MXFP4, против наших

21.3545 и 14.8512 при 4.1250. Сравнение идёт не при равной ширине, а следующий подраздел показывает, что происходит, когда её выравнивают.

Геометрическая сетка масштабов доминирует над плавающей

Масштаб читается как множитель, поэтому значима относительная ошибка, а сетка, минимизирующая относительную ошибку на лог-распределённом диапазоне, равномерна в логарифме. Плавающая сетка — нет.

Теорема (Геометрическая сетка масштабов). [доказано]

Плавающий масштаб $2^e(1+m/2^k)$ размещает 2^k точек в каждой бинаде, с наибольшим отношением соседей $1+2^{-k}$ у нижнего края бинады. Геометрическая сетка с тем же числом точек размещает их в $2^{j/2^k}$, и всякое отношение соседей равно $2^{2^{-k}}$. Поскольку $2^x = e^{x \ln 2} < 1+x$ при $x > 0$, худший шаг геометрической сетки строго меньше при любой ширине мантиссы, а отношение двух превышений монотонно растёт до $1/\ln 2 = 1.442695$.

Доказательство. Немедленно из $\ln 2 < 1$. Предел следует из $((1+x) - 1)/(2^x - 1) \rightarrow 1/\ln 2$ при $x \rightarrow 0^+$. ■

Дефект структурный, а не вопрос подкрутки: плавающая сетка равномерна по *значению* внутри бинады и потому неравномерна в логарифме — она тратит разрешение у верха каждой бинады, где относительный шаг и без того мал. Среднеквадратичная относительная ошибка по бинаде согласуется: 0.1138 против 0.1068 при двух битах мантиссы, 0.0550 против 0.0516 при трёх, 0.0270 против 0.0254 при четырёх.

Измеренное при равной ширине, на обеих моделях, предсказание держится (см. таблицу ниже).

Таблица. Сетка масштаба при равной ширине, $K=32$, элементы E2M1, 40 окон. Геометрическая сетка обыгрывает E4M3 на обеих ширинах и на обеих моделях, как того требует теорема о геометрической сетке. Величина отрыва между моделями не переносится; знак переносится.

сетка масштаба	b	SmolLM2	Qwen
геометрическая	7	18.8024	13.6401
E4M3	7	19.8628	13.7636
геометрическая	8	18.1238	13.6910
E4M3	8	19.8628	13.7636

Ещё одно прочтение зависит от модели, и мы его не делаем. На SmolLM2 геометрическая сетка при 4.25 бита на вес обыгрывает NVFP4-подобную конфигурацию при 4.50 — дешевле и лучше; на Qwen она дешевле и хуже, 13.6910 против 13.5340. *«Геометрическая обыгрывает плавающую при равных битах»* воспроизводится; *«геометрическая при меньшей цене доминирует конфигурацию NVFP4»* — нет.

Напряжение, которое здесь вскрывается. Оптимальная сетка геометрическая с отношением $R^{1/N}$ для занятого диапазона R и числа точек N , а это отношение — произвольное вещественное: чтобы его применить, нужен умножитель. φ^k геометрична и свободна от умножения, при отношении, зафиксированном на 1.618, которое оказывается наилучшим там, где диапазон и число точек его хотят, — на четырёх битах, где оно и выигрывает выше. Так что размещение — это пара утверждений, а не одно. Среди всех сеток масштаба геометрическая обыгрывает плавающую, и лучшая геометрическая сетка требует умножителя; среди сеток без умножения φ^k — самая тонкая из доступных на четырёх битах. Первое — вклад в область, второе — то, что этот датапуть получает даром.

Это не победа нашего формата над MXFP4: элемент остаётся E2M1, меняется только основание общего порядка, так что это вклад в семейство microscaling, а не его поражение. И помогающая сетка — именно наша: на двоичном датапути 2^k есть сдвиг, а φ^k нет, тогда как на датапути из раздела о замкнутости φ^k — это k шагов Фибоначчи и никакого умножителя. Сетка масштабов, стоящая всем остальным умножителя, здесь бесплатна.

Область действия. Это сказано для поканального масштабирования и не распространяется на короткие блоки. В применении к внутриблочному распределению при блоке в 32 и общем масштабе E8M0 та же форма возвращает $r^* = 1.3308$, что измеряется как 37.06 против 23.36 у лестницы, которую она ставит ниже, — а перплексия там

даже не монотонна по r : 39.09, 23.36, 39.61 по мере падения отношения через φ , суперзолотое и пластическое. Форма засчитывает отсечение по собственной квадратичной величине веса, и это верно, когда масштаб делят тысячи весов и отсечённые — это хвост, и неверно, когда его делят тридцать два и отсечённые составляют пятую часть блока.

Тонкость стоит регистров, а не сумматоров

Чтение иерархии по степеням, где ценой считается минимальное отношение, доступное на каждой степени, прячет результат. Читайте вдоль семейства.

Что вынуждено, а что нет. Для основания φ три цифры — доказанный минимум для сложения без переносов за постоянное время (Frougny 2011, 2013); двухцифровой алфавит невозможен. $\{-1, 0, 1\}$ — один из минимальных алфавитов, выбранный за симметричность, чтобы вычитание переиспользовало сумматор, — но не единственный: $\{0, 1, 2\}$ тоже минимален, и работающий параллельный сумматор на $\{0, 1, 2\}$ для φ здесь был реализован и испытан. Легерский (2018) распространяет границу на любой конечный алфавит внутри $\mathbb{Z}[\varphi]$, содержащий ноль, так что никакое нецелое множество от неё не уходит.

Граница ограничивает алфавит *цифр* позиционного представления и одну операцию — переписывание поцифровой суммы обратно в алфавит скользящим окном. У множества весов нет ни основания, ни переноса, поэтому она не переносится на алфавит весов скалярного произведения. Она перенеслась бы, только будь сам аккумулятор избыточным позиционным форматом по основанию φ , — а наш есть пара целых, никогда не приводимая к цифровой строке. Мы записываем теорему, потому что она говорит нечто настоящее и узкое — две буквы не заставить работать ни при каком размере окна, — и записываем область её действия, потому что более широкое прочтение, которое мы сами сперва записали, ложно.

И окно тоже не маленькое: опубликованный алгоритм на $\{-1, 0, 1\}$ является 21-локальным, с памятью 10 и упреждением 10. Прочсть «за постоянное время» как «дёшево» было бы ошибкой на порядок.

Таблица. Масштабы без умножения — все в одном месте. Ошибка уровня равна $(r-1)/(r+1)$, худший относительный зазор на лестнице с отношением r ; кремний — одно применение масштаба при 16-битных компонентах, xc7a200t, медиана по пяти семенам. Каждый рунг ниже сдвига стоит ровно одного сложения; растёт только число регистров.

масштаб	многочлен	r	рег.	слож.	ошибка ур.	LUT / $F_{\max}$
сдвиг	$r=2$	2.0000	1	0	33.33%	—
φ	$r^2=r+1$	1.6180	2	1	23.61%	149 / 307.69
суперзолотое	$r^3=r^2+1$	1.4656	3	1	18.88%	—
пластическое	$r^3=r+1$	1.3247	3	1	13.97%	228 / 231.21
	$r^5=r^3+1$	1.2365	5	1	10.57%	164 / 306.56
	$r^4=-r^3+r^2+r+1$	1.1787	4	3	8.20%	469 / 184.98
	$r^6=r+1$	1.1347	6	1	6.31%	—
	$r^8=r+1$	1.0970	8	1	4.63%	225 / 815.66

Оговорка, которая нужна этой таблице. Не всякий рунг перроновский — то есть не у всякого вещественный корень строго доминирует среди сопряжённых. У минимума степени 4 есть сопряжённое модуля 1.5129 при корне 1.1787, а у минимума степени 6 — 1.3069 при 1.0863; минимум степени 5, 1.1237, перроновский, его наибольшее сопряжённое равно 1.0959. Там, где корень не доминирует, координатный вектор r^k растёт как большее сопряжённое, а не как само значение, поэтому цепочке из k применений нужно $k \log_2(\rho/r)$ защитных бит — 0.36 бита на шаг при степени 4. Это цена композиции, а не одиночного применения, и потому в кремниевом столбце её не видно; здесь она названа, а не оставлена читателю на самостоятельное открытие.

Исчерпывающий поиск Ву (2010) устанавливает, что 1.123732821 — наименьшее перроновское число степени 5, так что рунг степени 5 доказуемо оптимален среди перроновских масштабов. Он же — исключительный случай Линда–Бойда: семейство $r^d=r+1$ — это гипотетически наименьшее по Линду перроновское число степени d , поправленное Бойдом (1985) ровно при $d>3$ с $d \equiv 3, 5 \pmod{6}$. Исключительные минимизаторы тоньше — 1.1237 против 1.1673 у семейства при $d=5$, — но они не одно-сумматорные: они стоят 3, 5, 7 и

более сумматоров с ростом d , а за пределами $d=5$ выигрыш меньше 0.8%. Более тонкий рунг за пять сумматоров — это совсем другой обмен, чем более тонкий рунг за один, и новостью была бы только вторая версия.

Таблица выше их собирает. Стоит явно проговорить два прочтения. Вдоль одно-сумматорного семейства число сумматоров постоянно и растут только регистры, так что строка r^4 — три сумматора ради отношения тоньше того, что r^5 даёт одним, — это исключение, а не закономерность, и именно она заставила более раннее прочтение этой иерархии сообщить о цене, растущей со степенью. И столбец частоты не падает вместе с отношением: рунг $d=8$ даёт 815.66 МГц против 307.69 у φ , потому что его единственный сумматор сидит на одном из восьми параллельных путей, а не на одном из двух.

Теорема (Пол Зигеля ограничивает семейство). [доказано]

Число Пизо–Виджаярагхавана — это вещественное алгебраическое целое больше единицы, все сопряжённые которого лежат строго внутри единичной окружности. Зигель (1944) доказал, что наименьшее из них равно 1.3247179572..., корню $r^3=r+1$, — при любой степени и любых целых коэффициентах. Этот корень есть член степени 3 нижеследующего семейства, значит, семейство является пизо при $d=2$ и $d=3$ и ни при какой более высокой степени: каждый последующий член тоньше пола Зигеля и потому не может быть пизо.

Вычисленные модули сопряжённых сходятся в точности: 0.6180 при $d=2$, 0.8688 при $d=3$, далее 1.0633, 1.0990, 1.0992, 1.0920, 1.0837 при $d=4\dots 8$. Граница — не случайность нашего перебора; она там, где ей велит быть теорема 1944 года, и *самый тонкий масштаб, применимый одним сложением и притом пизо*, — это пластическое число.

Различие важно для того, что на рунге можно построить, а не для самой арифметики. Нормализация по основанию θ реализуема конечным автоматом ровно тогда, когда θ — число пизо (Беренд 1994), а при $n \geq 4$ корень $x^n=x+1$ не является даже числом Парри, поскольку его сдвиг несофический (Акияма 2016). Так что перенормализатор с

ограниченной задержкой или компаратор канонической формы существуют при $d=2$ и $d=3$ и доказуемо не существуют выше. В датапути этой статьи ничто не нормализуется — координаты несутся как целые и никогда не сводятся к канонической цифровой строке, — вот почему шаг $d=8$ измеряется как 815.66 МГц, а не проваливает замыкание по времени. Мы это говорим потому, что читатель, предполагающий нормализатор, предполагал бы нечто запрещённое математикой.

Теорема (Одно-сумматорное семейство). [доказано] При каждом d многочлен $r^d = r + 1$ имеет единственный вещественный корень больше единицы. У него ровно два ненулевых коэффициента, поэтому его сопровождающее отображение есть

$$(x_0, \dots, x_{d-1}) \mapsto (x_{d-1}, x_0 + x_{d-1}, x_1, \dots, x_{d-2}),$$

то есть *одно сложение* независимо от d , а корни сходятся к $2^{1/d}$ сверху. Значит, любая гранулярность, какую предлагает логарифмическое дробление, достижима без умножителя — ценой одних лишь регистров.

Отображение проверено на точность на 3000 случайных координатных векторах при $d=5$ и $d=8$. Сходимость быстрая: $r/2^{1/d}$ равно 1.1441 при $d=2$, 1.0159 при $d=5$, 1.0059 при $d=8$ и 1.0014 при $d=16$.

Цена лестницы зависит от её роли. Применение веса лестницы r^{-j} стоит j шагов сопровождающего отображения. Как *блочный масштаб* это одно значение на тридцать два веса, и шаги амортизируются; как *элемент* оно платится на каждый вес, а шестибитной лестнице нужны j вплоть до 31. Поэтому элементная форма — это таблица координат: код напрямую выбирает вектор в $\mathbb{Z}[r]$, — и соответствующая таблица оценивает её против того, что она покупает.

Собранный узел. Каждая кремниевая цифра выше оценивает какую-то часть. Извне же спрашивают про LUT на нейрон, и следующая таблица — это собранная вещь: фан-ин N , веса из алфавита, аккумулятор в $\mathbb{Z}[\varphi]$, 8-битные отсчёты, стенд вычтен.

Таблица. Полный троичный нейрон, xc7a200t, медиана по пяти семенам, стенд вычтен, `-nodsp`. Первый столбец — реализация, которую эта статья построила первой; второй сворачивает отрицание в дерево накопления, где ему и место.

фан-ин	LUT/вес	свёрнуто	$F_{\max}$	свёрнуто	DSP
8	82.5	28.0	109.35	87.29	0
16	87.1	33.1	91.43	73.42	0
32	119.0	33.2	66.53	58.22	0

Дополнительный код — это $(-x)+1$, и эта $+1$ у каждой полосы есть один-единственный бит, так что свёртка — это исключающее ИЛИ со знаком веса плюс узкое второе дерево, несущее эти единицы и добавляемое один раз в корне. Эквивалентность проверили до синтеза: 0 расхождений на 400 случайных векторах (x, w) . Площадь падает в $2.9\text{--}3.6\times$ и садится ровно на ручной подсчёт — трёхпозиционный выбор 16-битного значения примерно за 16 LUT6 плюс 16-битное сложение примерно за 8, то есть 24–30 на полосу, — и это подтверждает, что отрицание было всем зазором, а не его частью. Частота падает примерно на 20%: корневое сложение дерева переносов сидит после основного; по пропускной способности на площадь обмен стоит $2.35\text{--}2.87\times$.

Итак, троичный нейрон стоит **28 LUT на вес при фан-ине 8 и ноль DSP при любом фан-ине**.

ПРИОРИТЕТ [проверено W845]. Нулевой расход DSP **не является нашим вкладом начиная с июля 2026 года**. ELiTeFormer (arXiv:2607.03652, 2026-07) сообщает о вычислительном элементе, который «устраняет все умножения в троичных линейных проекциях за счёт битовых масок, существенно снижая расход ресурсов полным отказом от блоков DSP». Это то же самое, что измерено здесь, получено независимо и **опубликовано раньше**. Приоритет в этой области решается месяцем, и месяц не наш. Что остаётся нашим — независимое измерение на **открытом** тракте синтеза, где число проверяемо читателем без лицензии на вендорский инструмент, и

измеренная цена 28 LUT на вес, которой в той работе нет. Отдельно: arXiv 2604.25183 зарегистрирован как открытая претензия к новизне золотого сита.

И один конвейерный регистр — там, где он окупается. Критический путь — это всё дерево сумматоров, поэтому единственный разрез посередине меняет такт и $N/2$ регистров на частоту. Это стоит измерить, а не предполагать, и ответ неоднороден. Сперва проверили эквивалентность: 0 расхождений на 398 случайных векторах (x, w) , конвейеризованный выход отстаёт ровно на один такт.

Таблица. Один конвейерный разрез в дереве накопления. Цена регистров обгоняет выигрыш в частоте где-то между шестнадцатью и тридцатью двумя, так что при фан-ине 32 лучше неконвейеризованная форма — обратное обычному совету, потому что и померили.

фан-ин	LUT/вес	$F_{\max}$ (МГц)	МГц на LUT
8	28.0 → 32.4	87.29 → 126.73	0.390 → 0.489 (+25.6%)
16	33.1 → 34.9	73.42 → 106.38	0.139 → 0.191 (+37.6%)
32	33.2 → 47.3	58.22 → 75.68	0.055 → 0.050 (−8.7%)

При фан-ине 8 и 16 разрез покупает 45% частоты за 16% и 5% площади; при 32 он покупает 30% за 42%. Перелом там, где $N/2$ регистров перестают быть дешёвыми относительно дерева, которое они режут. Значит, конфигурация, которую стоит называть, — это фан-ин 8 или 16 с одной ступенью: 32.4 LUT на вес при 126.73 МГц либо 34.9 при 106.38, и в обоих случаях ноль DSP. Несвёрнутый столбец мы приводим рядом потому, что это число было опубликовано первым — с оговоркой, называвшей отрицание вероятной причиной, — и причина оказалась всей причиной целиком.

Таблица. Декодеры элементов как таблицы координат, изолированно, медиана по пяти семенам, против перплексии той же лестницы на SmolLM2-135M (базовая линия fp32 14.3607). Тридцать три лишних LUT сдвигают ошибку с 70% до 3.7% без потери частоты.

декодер	таблица	LUT	$F_{\max}$	ppl	сверх fp32
φ , 4 бита	$7 \times 2 \times 5b$	89	621.89	24.4280	+70.1%
$r^5 = r^3 + 1$, 5 бит	$15 \times 5 \times 4b$	95	482.39	15.9242	+10.9%
$r^6 = r + 1$, 6 бит	$31 \times 6 \times 5b$	122	612.00	14.8882	+3.7%

Средний рунг — тот, что выбрал бы бюджет: пять бит и 95 LUT за +10.9%, надбавка в шесть LUT над φ за шестикратное снижение ошибки. Применение масштаба всюду остаётся одним сложением, а датапуть остаётся свободным от DSP.

Таблица. Одно применение масштаба, 16-битные компоненты, xc7a200t, медиана по пяти семенам. Пятикратное измельчение стоит 51% лишних LUT и нисколько частоты: единственный сумматор восьмireгистрового шага сидит на одном из восьми параллельных путей, а не на одном из двух.

шаг	LUT	FF	$F_{\max}$	ошибка уровня	слож.
$\varphi, r^2 = r + 1$	149	128	307.69	23.607%	1
$r^5 = r^3 + 1$	164	160	306.56	10.575%	1
$r^8 = r + 1$	225	212	815.66	4.625%	1

Измеренное как блочный масштаб на двух сетях, одно-сумматорное отношение не уступает стоящему рядом геометрическому на обеих ширинах: на пяти битах 21.5040 против 21.5872 на SmolLM2 и 14.7342 против 14.7814 на Qwen, на шести — 19.3209 против 20.0199 и 13.8528 против 13.8915. Значит, преимущество геометрической сетки из соответствующей теоремы доступно вообще без умножителя.

Это же снимает и напряжение, оставленное теоремой о геометрической сетке. Сетка, минимизирующая относительную ошибку, геометрична с отношением $R^{1/N}$ — произвольным вещественным, требующим умножителя. В общем случае. Но там, где блочный масштаб живёт на самом деле — занятый диапазон 8.32–9.12 бинады, ширина от четырёх до шести бит, — ближайшее отношение без умножения совпадает с оптимумом в пределах 0.06%: 1.551532 против 1.551145 на четырёх битах, 1.236506 против 1.236662 на пяти, 1.110187 против 1.110180 на шести. На восьми битах наш первый перебор сообщил о зазоре, и это была граница нашего же поиска, а не свойство спектра: остановившись на степени 8, ближайшее отношение давало 1.049852 против цели 1.026159, тогда как степень 9 достигает 1.036380,

а степень 10 — 1.027738. Если разрешить коэффициенты из $\{0, \pm 1, \pm 2\}$ — это по-прежнему сложение и провода, ведь ± 2 есть сдвиг, — степень 9 достигает 1.026129, в пределах 0.003%, из 734,533 перечисленных корней.

Настоящий предел — это само одно-сумматорное семейство. На той же восьмибитной цели его лучший член равен 1.0850702 при $\{0, \pm 1\}$ и 1.0510545 при $\{0, \pm 1, \pm 2\}$, то есть ошибки 5.6% и 2.4%, и с ростом степени они не улучшаются. Любое отношение достижимо без умножения, если тратить сумматоры; более грубый набор достигим одним.

Иерархия дёшева только до третьей степени. Следующая таблица добавляет наименьший масштаб степени 4, корень $r^4 = -r^3 + r^2 + r + 1$, чьё отображение $(x_0, x_1, x_2, x_3) \mapsto (x_3, x_0 + x_3, x_1 + x_3, x_2 - x_3)$ проверено на точность на 3000 случайных координатных векторах.

Таблица. Одно применение масштаба, xc7a200t, 32 бита, медиана по пяти семенам. Степени 2 и 3 обе стоят одного сложения; на степени 4 вектор коэффициентов перестаёт быть разреженным, и цена подсказывает.

масштаб	LUT	FF	$F_{\max}$	ошибка уровня	слож./выч.
φ (степ. 2)	223	192	247.10	23.61%	1
пластическое (степ. 3)	228	200	231.21	13.97%	1
1.1787 (степ. 4)	469	320	184.98	8.20%	3

Это ставит алфавит на точное место. φ — верный рунг на четырёх битах, где $r^* = 1.4115$ падает между ним и суперзолотым и оба идут вничью. Это неверный рунг на пяти и выше — а к пяти мы уже оставили позади и ту часть иерархии, которую можно оплатить, поскольку степень 4 стоит в $2.1 \times$ больше площади. Режим, в котором датапуть без умножений вообще стоит строить, — это тот, где степени 2 и 3 суть единственные рунги, которые по карману, и этот режим — от трёх до пяти бит.

Первый предиктор провалился, и провал поучителен: выбор самой тонкой лестницы, чей размах покрывает канальный динамический диапазон весов — медиана $268.95 \times$ по 155,520 каналам, — выбирает самый грубый рунг при каждом бюджете: верно на трёх битах и неверно на четырёх и пяти. Покрытие — неверный критерий, потому

что отсечь самые малые веса почти бесплатно, а округлить самые большие — нет. Решает суммарная ошибка, а суммарная ошибка вычислима до всякого запуска модели.

Различие проходит между квантованием к числам Фибоначчи и работой в том кольце, координатами которого эти числа являются. Оно уже, чем может показаться, и мы формулируем его узко: по формуле Бине $F_n = \text{round}(\varphi^n / \sqrt{5})$ точно при $n \geq 1$ — проверено здесь до $n=25$, — так что фибоначчиева лестница и есть лестница степеней φ с точностью до постоянного смещения. Чего отсюда не следует — так это замкнутости. $F_i F_j$ не является числом Фибоначчи, поэтому произведение покидает представимое множество; $\varphi(a+b\varphi)$ остаётся в $\mathbb{Z}[\varphi]$. Две схемы согласны насчёт лестницы и расходятся насчёт кольца, а стадия преобразования — цена второй. $F_i F_j$ уходит из множества; $\varphi \cdot (a+b\varphi)$ — нет. Цена ухода — это ровно те два сумматора и схема маршрутизации, а замкнутость покупает то, что не нужно ни одного из них: одно целочисленное сложение, и путь без умножений — не приближение к умножению, а в точности равен ему (см. теорему о точности пути без умножений).

Замкнутостная половина этого аргумента машинно проверена. В сопровождающей разработке доказано, что отображение $(a, b) \mapsto (b, a+b)$ вычисляет умножение на φ на вещественных числах, и из этого следует точность целого скалярного произведения — в Coq, без единой допущенной леммы.

Ступень стоит порядок величины, и она работает на каждое накопление. Следствие о замыкании (см. соответствующий раздел) говорит, что ступень необходима; кристалл говорит, сколько она стоит. Таблица о замыкании измеряет один шаг накопления на стенде из раздела об аппаратуре — изолированный блок, каждый выходной бит свёрнут в наблюдаемую редукцию, медиана по пяти семенам размещения.

Таблица. Один шаг накопления, xc7a200t, медиана по пяти семенам. Незамкнутый блок — это жадный нормализатор Цекендорфа, сверенный с оракулом исчерпывающе по всем 65,536 входам: по сумме, по свойству «нет двух соседних цифр» и по согласию с независимо написанной жадной моделью.

одно накопление	LUT	$F_{\max}$ (МГц)
замкнуто, $\mathbb{Z}[\varphi]$, покомпонентно (16 бит)	182	299.94
незамкнуто, нормализация обратно (16 бит)	1756	11.46
замкнуто, $\mathbb{Z}[\varphi]$, покомпонентно (32 бита)	283	239.35
незамкнуто, нормализация обратно (32 бита)	7017	4.25

Ступень работает не по разу на умножение. Произведение можно обработать и так, и эдак: тождество $F_m F_n = (L_{m+n} - (-1)^n L_{m-n})/5$ превращает одно умножение в два табличных обращения и вычитание. А вот у *накопления* такого выхода нет, потому что сумма двух представимых чисел не представима — $F_3 + F_3 = 4 = F_4 + F_1$, и представление меняется нетривиально. При фан-ине 512 ступень отрабатывает 512 раз на нейрон — против пары целочисленных регистров на замкнутой стороне.

Одно ограничение стоит проговорить: жадная нормализация не доказана как самая дешёвая — существуют сумматоры Цекендорфа с постоянным временем, — так что отношение является верхней границей для данной реализации, тогда как качественное утверждение несёт следствие о замыкании.

Другое кажущееся ограничение измерения не переживает. Часть разрыва по частоте — это комбинационная глубина, 46 зависимых ступеней против одного сложения, и конвейеризация должна была бы превратить это в задержку и регистры. Она и превращает — и этого не хватает: при развёртке до 32 ступеней на 16 битах и 48 на 32 частота растёт и затем выходит на плато 249.44 и 111.36 МГц против 299.94 и 239.35 у замкнутого пути, то есть во втором случае меньше половины. Причина есть.

Теорема (Порог нормализации). [измерено] Каскад нормализации нельзя конвейеризовать мельче, чем одно сравнение-и-вычитание на ступень, так что его период тактирования снизу ограничен задержкой переноса сравнения-и-вычитания на ширине тракта данных. Сравнение-и-вычитание строго больше сложения — это вычитание плюс сравнение,

которое его выбирает, — поэтому порог незамкнутого пути лежит выше порога замкнутого пути, шагом которого является одно сложение.

На плато сделка выглядит так: в $7.8 \times$ и $19.7 \times$ больше площади, в $3.6 \times$ и $8.3 \times$ больше регистров, 32 и 48 тактов задержки — и всё равно более низкая частота.

Это утверждение другого рода, нежели остальная статья. Везде мы сравниваем величины ошибок между форматами; здесь сравнивать нечего — ошибки нет. Округление входит в φ -троичную сеть только на нелинейности, где оно неизбежно в любом формате, а цена точности на линейном пути — два целочисленных аккумулятора вместо одного плавающего.

Почему основание алфавита не может быть никаким другим. Замыкание требует $r^2 = pr + q$ с целыми p, q , чтобы $\mathbb{Z}[r]$ было кольцом. Среди таких $r > 1$ самая дешёвая рекуррента — та, что с $p = q = 1$, поскольку любое большее p вводит сдвиг рядом со сложением: $r = 1 + \sqrt{2}$ удовлетворяет $r^2 = 2r + 1$ и стоит сдвига, а $r = \sqrt{2}$ имеет $r^2 = 2$ и вообще теряет масштаб из решётки. Случай $p = q = 1$ — это φ , и он именно минимум, а не один из членов множества приемлемых вариантов.

Область применимости, оговорённая так, чтобы утверждение не перебирало. Речь идёт об арифметике в решётке. Это касается линейной алгебры, которая доминирует и в работе сети, и в её кремниевой цене, и это ничего не говорит про поток управления, адресацию или ветвления — они не решёточные задачи и от φ ничего не выигрывают.

Чего пара стоит в кремнии

Измерено на XC7A200T через полностью открытый маршрут, без единого DSP-блока нигде:

Таблица. Стек с нулём DSP. Один примитив знакового выбора строит обе половины узла, так что колонка DSP остаётся свободной и радио с сетью уживаются на одном кристалле.

блок	DSP	результат		
коррелятор (плавающие отводы)	GF16	8 DSP48E1	—	до
троичный коррелятор (GFTernary)		0	78 МГц, реальный эфир	после
каждый трансформерный блок		0	систолический GEMM 33 GOPS	тот же примитив
обученный троичный трансформер		0	97% сквозной точности	работает
обратное распространение на кремнии		0	XOR 4/4, 24–25 эпох	обучается

ОГОВОРКА W845 к таблице стека без DSP [провенанс отсутствует]. Пять чисел той таблицы — 78 МГц, 33 GOPS, 97%, XOR 4/4, 24–25 эпох — приведены без тега статуса, без сида, без стенда, без даты и без перекрёстной ссылки. Проверка по всему репозиторию: «33 GOPS» не встречается больше нигде, и ни один файл не фиксирует троичный коррелятор на 78 МГц. XOR 4/4 и 24–25 эпох задокументированы в `docs/SILICON_TRAINING_METHODODOLOGY.md` и `docs/NOW.md` как **зависящие от перебора сидов** — часть сидов расходится. Единственные ~97 % в репозитории относятся к двухслойному перцептрону на двумерной задаче, а не к обученному троичному трансформеру. До восстановления провенанса эти пять чисел читать как незаявленные.

Таблица. Пропускная способность на единицу площади среди форматов, несущих собственный диапазон, со сверкой. Один тракт данных троичного нейрона, XC7A200T, без DSP, каждый бит аккумулятора наблюдаем, медиана по пяти семенам размещения. `int8` — в таблице отвергнутых форматов. **Все три рунга TNF здесь стоят на своих специфицированных параметрах**, и все три сдвинулись: ранее измеренные модули реализовывали другие форматы — TNF16 не хватало одного бита мантиссы, TNF32 был с двенадцатью тритами и одиннадцатью битами мантиссы там, где спецификация даёт шесть и двадцать пять, TNF64 так же, — и ни у одного не было эталона, из-за чего расхождение дожило до цитирования. Написанные по спецификации и сверенные с ней, они совпадают на каждом просвеченном коде и стоят каждому по восемь-

одиннадцать позиций. Что остаётся наверху — это алфавит, совпадающий на всех своих четырёх кодах. *Шесть наших модулей сдвинулись, когда для них написали эталон* — четыре рунга TNF и BNF16 были либо на бит мантиссы короче, либо вовсе другим форматом, а GF+8 измерялся с заглушенным селектором кармана, что оценивало один путь из четырёх. Цифра 28% для IBM hex32 здесь снимается вместе с 253, сообщёнными для LNS16, по причине, развёрнутой в разделе о корректной постановке: ни то, ни другое не измеряло декодер. Починенные и перемеренные, IBM hex32 и VAX F точны на каждом сопоставимом коде, и оба упали в таблице, потому что логика, которой им недоставало, — это логика, за которую они теперь платят. `fp8` вообще опускал субнормали. Строки `fp8` теперь — это достроенные декодеры, которые стоят на 10% больше площади и переехали с третьего и пятого мест на десятое и одиннадцатое: пропуск стоил больше, чем собственный запас формата, а починка чужой реализации сдвинула сравнение в нашу пользу — то есть в ту сторону, которую предсказывает правило, и теперь это измерено, а не заявлено. Каждая найденная ошибка, наша и чужая, делала свой модуль меньше: нереализованный случай — это несинтезированная логика. **Разрешение.** Объединённый коэффициент вариации по семенам равен 3.71%, так что две строки расходятся на уровне $\alpha = .05$ только тогда, когда их MHz/LUT отличаются более чем на 5.50% относительно; меньшие зазоры в этой таблице — не результаты. Порог обнаружения разницы в стоимости декодера составляет 41.3 LUT, а это 79% целого декодера на медианной строке — стенд в 472 LUT раздувает этот порог десятикратно. Каждая строка измерена на одном и том же 128-битном стенде; строки, которые измерялись не на нём, перемерены или сняты.

#	формат	b	LUT	MHz/ LUT	±/размах	sd	статус	сверка
1	GFternary	2b	527	0.1584	±0.0018	54	наш	все 4 кода
2	binary32	32b	536	0.1429	±0.0013	54	—	тождество (точно)
3	fp8 e4m3	8b	549	0.1225	±0.0012	54	—	все 256,0 ошибок
4	TNF16a	16b	566	0.1214	±0.0012	54	устар.	все 65{,}534 ✓
5	fp8 e5m2	8b	544	0.1188	±0.0011	54	—	все 256,0 ошибок
6	GF10	10b	597	0.1172	±0.0012	54	наш	все 1{,}024
7	TNF17e	17b	596	0.1155	±0.0013	54	рек.	все 124{,}416 ✓
8	TNF16c	16b	591	0.1152	±0.0010	54	рек.	все 62{,}208 ✓
9	binary16	16b	586	0.1078	±0.0012	54	—	все 65{,}536
10	minifloat	8b	622	0.1075	0.1009–0.1132	5	—	все 254 ✓
11	GF14	14b	651	0.1074	0.0969–0.1140	5	наш	все 16{,}384
12	TNF17e-bare	17b	605	0.1072	0.1041–0.1104	5	контроль	развёртка в ожидании
13	TNF8	10b	609	0.1069	0.1028–0.1099	5	устар.	все 1{,}024,0 ошибок
14	TNF16b	16b	604	0.1048	0.0998–0.1086	5	устар.	все 65{,}534 ✓
15	BNF16	17b	635	0.1034	0.0961–0.1064	5	устар.	все 131{,}072,0 ошибок
16	TNF16	17b	629	0.1005	0.0971–0.1071	5	устар.	все 131{,}072
17	VAX F	32b	624	0.0942	0.0926–0.1000	5	—	все 40{,}000 ✓
18	GF+8	8b (+2 sel)	692	0.0882	0.0831–0.0931	5	наш	1{,}024/1{,}024 ✓
19	posit8	8b	614	0.0728	0.0712–0.0774	5	—	все 255 ✓
20	takum16	16b	853	0.0695	0.0677–0.0745	5	—	632/61{,}505
21	LNS16	16b	722	0.0598	0.0581–0.0600	5	—	107/65{,}536
22	posit16	16b	731	0.0485	0.0467–0.0496	5	—	все 65{,}535 ✓
23	posit32	32b	1031	0.0302	0.0291–0.0314	5	—	все 40{,}000 ✓
24	IBM hex32	32b	1072	0.0292	0.0273–0.0297	5	—	все 40{,}000 ✓

Рисунок (tnf_full_table.pdf). Те же двадцать четыре формата. Наши — это четыре названных выше семейства; подписи столбиков несут число LUT, так что площадь и пропускная способность читаются вместе.

Применитель масштаба, и утверждение, которое мы снимаем

Раздел о паре выносит умножитель из веса и из накопления. Масштаб слоя — третья точка, и две наши первые попытки с ней были ошибочны, каждая по-своему. Обе записаны здесь, потому что результатом является поправка.

Троичная сеть хранит веса в $\{-1, 0, +1\}$ и один вещественный послойный масштаб $a = \mathbb{E}|W|$. Применение a — это настоящее умножение, так что умножитель, который убрали троичные веса, возвращается на границе слоя. Привязка a к сетке убирает его снова, и вопрос в том, какая это сетка.

Первое утверждение, снято. Мы сообщали, что сетка φ^k бьёт степени двойки: избыточная ошибка восстановления 2.44% против 4.86% на 210 слоях — отношение 0.501 против предсказанного 0.500 из плотности $\log 2 / \log \varphi = 1.440$. Это измерение воспроизводится и верно. Но оно сравнивалось не с той базой. Развёрнутое состояние дел для безумножительных масштабов — это не голые степени двойки, а **ARoT**, чьи уровни суть суммы степеней двойки. Измеренный так же, ARoT-2 стоит 0.1651%, а ARoT-3 — 0.0054%, оба за один такт против наших k . По точности сетка φ проигрывает в $15 \times$, и это утверждение снимается.

Второе утверждение, ограничено. Теорема о точности утверждает, что линейный путь точен, потому что $\mathbb{Z}[\varphi]$ — кольцо. Она проверена машинно и остаётся верной, но мы преподносили её как преимущество. Масштаб ARoT — двоично-рациональное число, а $\mathbb{Z}[1/2]$ тоже кольцо, замкнутое относительно операций тракта данных: слой с фан-ином 512, масштабом ARoT-2 и входами Q8 накапливает до знаменателя 2^{15} , что точно в обычной фиксированной точке. У двоичной арифметики это есть с тех пор, как существует фиксированная точка. Теорема стоит; её значимость — нет.

Что показало измерение площади. Обе попытки сравнивали точность. Применители различаются сильнее по площади, причём так, как никто из нас не предсказывал.

Таблица. Применители масштаба, примитивы XC7A200T, один стенд, вывод DSP отключён. Глубина композиции d считает масштабы, перемноженные без переквантования между ними.

применитель	LUT	такты	избыточная ошибка	
умножитель, вещественный a	1215	1	0.0000%	или 2 DSP48
APoT-2, $d=0$	384	1	0.1651%	два барабанных сдвига
APoT-4, $d=1$	659	1	—	четыре сдвига
APoT-8, $d=2$	1217	1	—	восемь сдвигов
φ^k , любое d	173	k	2.4420%	без сдвигателя

Теорема (Рост числа членов). [измерено] Значение, представленное суммой n членов из мультипликативно замкнутого порождающего множества и составленное через d умножений на масштаб без переквантования, имеет n^{d+1} членов. Значение в $\mathbb{Z}[\varphi]$ имеет две компоненты при любом d , поскольку $\varphi^a \varphi^b = \varphi^{a+b}$ складывает порядки, а не перемножает члены.

Теорема (Стоимость сдвигателя). [измерено] Сдвиг, переменный во время выполнения, на W битах стоит $\Theta(W \log W)$ LUT; фиксированная перестановка стоит нуль. Поэтому n -членный применитель APoT — это $\Theta(nW \log W)$, а шаг Фибоначчи — $\Theta(W)$.

Вот эффект, который мы упустили, и он больше того, что мы предсказывали. Мы называли APoT «shift-add» и считали это бесплатным. Сдвиг на константу бесплатен; сдвиг на переменную — это барабан, измеренный здесь в 192 LUT на каждый при 32 битах, а масштаб, который должен обслуживать любой слой, переменен по построению. В шаге Фибоначчи сдвигателя нет вообще.

Следствие (Точка пересечения с умножителем). [измерено] Поскольку площадь APoT растёт как n^{d+1} , а площадь умножителя не растёт, существует глубина d^* , на которой APoT стоит того, ради избежания чего его вводили. Измерено при $W=32$, $n=2$: $d^*=2$, где APoT-8 стоит 1217 LUT против 1215.

Теорема (Единого победителя нет). [измерено] В координатах (площадь, задержка, ошибка) ни один применитель не доминирует другой. Умножитель точен и самый большой, φ^k — самый маленький и наименее точный, АРоТ-2 между ними и не побит ни одним; все три — вершины фронта.

Поэтому рекомендация — это правило, а не победитель. Используйте φ^k там, где путь масштаба ограничен площадью или где масштабы составляются без переквантования: низкоранговая факторизация, свёрнутая свёртка с батч-нормой, скаляр остаточной ветви или накопление вдоль маршрута в меш-сети. Используйте АРоТ там, где ограничение по задержке или по точности. Мы формулируем это так, а не защищаем один ответ, потому что две наши предыдущие попытки защищали по одному ответу, и каждая была неправа по-своему.

Преимущество по площади тоже снимается — и нашей же собственной атакой. Таблица применителей была измерена в одном режиме и при одной ширине поля, и оба выбора нам льстили.

Теорема (Развёрнутая рекуррента линейна). [измерено] Рекуррентный применитель, развёрнутый до глубины K на W -битных операндах, стоит $\Theta(KW)$; аддитивно-членный применитель, чьи сдвиги являются константами времени компиляции, стоит $\Theta(W)$ независимо от масштаба, поскольку константный сдвиг — это проводка. Значит, когда масштаб заморожен после обучения, рекуррента проигрывает при любом $K \geq 1$.

Измерено: применитель АРоТ-2 с константными сдвигами стоит 26 LUT против 64, 128 и 256 у φ^k , развёрнутого при $K=2,4,8$. При реалистичном $K=8$ это $10\times$ против нас, и линии не пересекаются даже при $K=1$.

Теорема (Инверсия режима). [измерено] Порядок по площади между двумя семействами не является свойством ни одного из них. Он переворачивается в зависимости от того, является ли масштаб константой времени компиляции или переменной времени выполнения, потому что константный сдвиг — это проводка, а переменный сдвиг — барабан ценой $\Theta(W \log W)$.

Теорема (Барабан оценивается по диапазону, а не по ширине). [измерено] Поле сдвига времени выполнения должно покрывать лишь диапазон масштабов рабочей нагрузки. На 210 измеренных здесь слоях этот диапазон равен 3.15 октавы, так что двух бит достаточно. Сравнение по площади, задающее поле шире, измеряет конкурента, которого никто бы не построил.

При двухбитном поле АРоТ-2 стоит 130 LUT против наших 199. Заявленное выше преимущество в $2.22 \times$ было артефактом пятибитного поля и снимается.

Ограничение инструмента, названное, а не заглаженное. Развёртка по АРоТ немонотонна — 130, 380, 230, 384 LUT при ширинах сдвига от 2 до 5 — при демонстративно применённом параметре и детерминированном инструменте. Это эвристики технологического отображения, и это значит, что счётчики LUT от одного лишь логического синтеза не являются надёжной метрикой площади на такой гранулярности. Утверждение, опирающееся на 30% разницы между двумя такими точками, ненадёжно; утверждение, опирающееся на $10 \times$ из теоремы о развёрнутой рекурренте, — надёжно.

Что уцелело. Не преимущество по площади. Остаётся только теорема о росте числа членов: φ^k — единственный применитель, чья площадь не зависит от глубины композиции, так что при $d=2$ АРоТ-8 стоит 1217 LUT там, где пара по-прежнему стоит 173. Этот режим — цепочка без переквантования между звеньями — реален, но нечаст. Единственность алфавита и замыкание $\mathbb{Z}[\varphi]$ проверены машинно и не тронуты, поскольку никогда и не были утверждениями о площади.

Картина по трём попыткам этого раздела — одна и та же картина. Каждое благоприятное число приходило из сравнения, условия которого выбирали мы: степени двойки вместо АРoТ, пятибитное поле вместо двух, которые нужны рабочей нагрузке, режим времени выполнения вместо замороженного. Правило, которое поймало бы все три: выпиши, что построил бы конкурент, если бы он пытался выиграть, и измеряй это.

Кремниевая таблица, измеренная на проверенном стенде

Каждая кремниевая цифра в этой статье изначально измерялась через стенд, который объединял декодер с общим аккумулятором и наблюдал часть его выхода. Три отдельных дефекта этой конструкции описаны в разделе о самопроверке; каждый менял заголовочное число на 10–50 процентов, при том что ни одно отдельное измерение не было неверным. Таблица ниже измерена на единственном стенде, проходящем ворота наблюдаемости, которые мы написали после: только декодер, каждый выходной бит свёрнут в наблюдаемую редукцию, медиана по пяти семенам размещения.

Таблица. Стоимость декодирования, изолированно. `nextpnr-xilinx` 1743d0f, XC7A200T, без DSP, медиана по пяти семенам. Голый провод на том же стенде стоит 112 LUT при 827.81 МГц.

#	формат	вид	LUT	$F_{\max}$
1	GFTernary	фикс.	66	974.66
2	<code>int8</code>	фикс.	76	925.93
3	binary32	фикс.	112	886.52
4	TNF16	фикс.	101	407.66
6	BNF16	фикс.	97	388.35
9	VAX F	фикс.	100	311.14
13	binary16	фикс.	164	235.18
14	IBM hex32	фикс.	243	111.10
15	LNS16	логарифмический	270	93.17
16	posit8	сужающийся	214	77.75
17	posit16	сужающийся	302	62.39
18	posit32	сужающийся	517	49.05

Теорема (За сужение платят задержкой, а не площадью).
[измерено] Декодирование сужающегося формата содержит скан — последовательную зависимость, длина которой растёт со

словом. Последовательная зависимость стоит критического пути напрямую и площади лишь попутно, тогда как фиксированный формат с большим основанием стоит широкого сдвига, который тяжёл по площади и мелок по глубине. Поэтому два семейства расходятся по частоте, а не по площади.

Измерено: все четырнадцать фиксированных полей стоят по частоте выше всех трёх сужающихся, причём худший фиксированный опережает лучший сужающийся в $1.43\times$; по площади они перекрываются — posit8 на 214 LUT против 243 у IBM hexadecimal. Перекрытие предсказано теоремой, а не оправдано задним числом.

GFternary против posit32 — это $7.8\times$ по площади и $19.9\times$ по частоте. GFternary стоит 66 LUT там, где голый провод стоит 112, потому что декодирование двухбитного алфавита позволяет синтезатору упростить регистр ниже по потоку сильнее, чем это делает передача 32 сырых бит.

Корреляция площадь–частота на изолированном декодере равна -0.60 против -0.90 на объединённом стенде. Оси здесь несут независимую информацию, поэтому они и приводятся отдельно, и никакого их отношения нигде не появляется.

Рабочая точка

Предыдущие подразделы снимают утверждение о точности и утверждение о площади. Остаётся высказывание о профилях операций, и оно первое в этом разделе, которое само называет свою границу, а не оказывается притиснуто к ней.

Числовая система используется через две операции, и три кандидата покупают каждый по одной дёшево и платят за другую.

Таблица. Стоимость каждой операции, один стенд, XC7A200T, вывод DSP отключён. Логарифмическая строка — это наше собственное измерение `takum32_decode` из раздела об аппаратуре.

система	умножение	сложение
LNS, takum	порядки складываются: бесплатно	таблица $\log(1+2^x)$: 10, 967 LUT
фиксированная точка, APoT	умножитель, или проводка, если заморожено	сумматор: дешево
$\mathbb{Z}[\varphi]$	шаг Фибоначчи, один сумматор	покомпонентно: 64 LUT

$\mathbb{Z}[\varphi]$ — единственная, что дешёва по обеим. Её цена в том, что бесплатное умножение ограничено степенями φ , а не общее, и дальше сказано, почему в этом тракте данных эта цена не платится.

Теорема (Требуемое множество умножений). [доказано] В тракте данных, где каждый вес — это код, применяемый знаковым выбором, масштаб слоя — степень основания алфавита, а накопление — единственная оставшаяся операция, умножения, которые тракт данных обязан выполнять, суть в точности $\{\text{основание}^k\}$. Общее умножение никогда не запрашивается.

Следствие (Достаточность). [доказано] Кольцо, замкнутое относительно сложения и относительно умножения на степени своего образующего, достаточно для такого тракта данных. $\mathbb{Z}[\varphi]$ — такое кольцо, причём обе операции в нём стоят $\Theta(W)$.

Следствие (Две другие снабжены неверно). [доказано] LNS переснабжена: она покупает общее умножение, которое этот тракт данных никогда не просит, и платит за это сложением, которое выполняет на каждом члене каждого скалярного произведения. Фиксированная точка недоснабжена в противоположную сторону: сложение бесплатно, а умножение на масштаб — нет, если только масштаб не заморожен.

Ограничение $\mathbb{Z}[\varphi]$ совпадает с ограничением, которое троичный тракт данных и так несёт. Это совпадение и есть результат; это не утверждение, что формат лучше, — только что он согласован.

Теорема (Композиция на этапе компиляции бесплатна).
[измерено] Если все d масштабов цепочки известны до исполнения, то известно и их произведение, и любое представление применяет его ценой одного масштаба. Рост числа членов из теоремы о росте членов происходит тогда и только тогда, когда d или сами масштабы становятся известны лишь во время выполнения.

Это убирает три из четырёх случаев, где мы заявляли преимущество по глубине. Низкоранговая факторизация, свёрнутая свёртка с батч-нормой и скаляр остаточной ветви — все фиксированы после обучения, так что их произведение вычисляется заранее и никакой композиции в железе не происходит. Один случай выживает: накопление вдоль маршрута в меш-сети без ренормализации, где число хопов — величина времени выполнения. Преимущество по глубине реально, и живёт оно там, а не в сети.

Названные границы. Это не победа по площади над APT — она была измерена и снята. Это не победа по точности, где APT-2 впереди в $15\times$. И там, где масштаб заморожен, выигрывать нечего, потому что замороженный масштаб — это проводка. Это утверждение о том, какая система соответствует профилю операций, и оно держится там, где масштаб не заморожен: общий движок, обслуживающий много слоёв, или композиция, глубина которой не известна до исполнения.

Форматы, в которых обучающий прогон жить не может

Пропускная способность на единицу площади осмысленна только среди форматов, способных выполнить задачу, поэтому кандидаты делятся надвое одним проверяемым, а не риторическим тестом: *несёт ли слово диапазон, или диапазон приходится приносить снаружи?*

Таблица. Отвергнуты для этой задачи. У каждого $e=0$: нет поля порядка, а значит, нет динамического диапазона внутри слова. Каждый работает только рядом с внешним потенциальным или поканальным масштабом, и каждый опубликованный результат обучения ниже 8 бит

с их использованием несёт блочные масштабы и мастер-веса повышенной точности. Градиенты охватывают много порядков величины; слово без диапазона их не удержит.

формат	порядок	что даёт диапазон		обучается сам
<code>int8</code>		$e=0$	обучаемый тензорный масштаб	нет
<code>int4</code>		$e=0$	блочный масштаб (32 или 16 элементов)	нет
<code>int12 – int24</code> аккумулятор		$e=0$	фиксированная точка, задана вручную	нет
элемент E2M1 сам по себе	$e=2$ внутри блока		общий UE8M0, снаружи	нет

Измерено на том же стенде (`int8`): 454 LUT, 78.83 МГц, 0.1736 MHz/LUT, разброс 11.00.

Почему GFternary не в таблице отвергнутых, несмотря на $e=0$.

Различие не в наличии поля порядка, а в том, является ли масштаб внешним и обучаемым или внутренним и точным. Алфавиту $\{-1, 0, +1\}$ нужен вещественный a_ℓ на слой, обучаемый вместе с весами и умножаемый на этапе вывода — умножитель возвращается. Алфавиту φ не нужен никакой: по следствию о глубине межслойное усиление равно $F_k\varphi + F_{k-1}$ — пара целых, известных заранее, применяемых сдвигом и сложением. Его масштаб находится внутри системы по построению, а не подгоняется извне. В этом и разница между «нести диапазон» и «занимать его», и поэтому стек обучается прямо на кристалле.

Почему `int8` приводится отдельно. Тракт данных `int8` набирает 0.180 MHz/LUT при медиане по пяти семенам размещения против 0.181 у GFternary — разница в 0.4% внутри 16% разброса по семенам, так что на этом маршруте они неразличимы. Более ранняя версия этой статьи сообщала 0.189 против 0.177 и называла это отрывом в 7%; та цифра пришла из одного-единственного прогона размещения и снимается. `int8` всё равно исключён из той таблицы, и причина структурная, а не выгодная нам. У `int8` $e=0$: нет поля порядка, а значит, нет динамического диапазона внутри слова. Он работает только с внешним тензорным или поканальным масштабом; без него это просто целые числа $-128...127$. По следствию о побеге это и есть побег через неоднородность — масштаб вынесен за пределы слова, — так что `int8` есть блочный формат, носящий имя скалярного, и оценивать его

против форматов, несущих собственный диапазон, значит измерять не то. Он возглавляет таблицу ровно потому, что отказывается от задачи, которую эта таблица ставит.

Последствия важнее для обучения, чем для вывода. Градиенты охватывают много порядков величины, и формат с нулевым динамическим диапазоном их не удержит: каждый опубликованный результат обучения ниже 8 бит — MXFP4, NVFP4 — несёт блочные масштабы и мастер-веса повышенной точности. Наш собственный стек обучается на кристалле, при нуле DSP, с обратным распространением в самом формате.

Снятие: фронт при равном объёме хранения

Следствие об отсутствии выживших и следствие о запасе считают позиции. А формат хранится в битах, и вся разница именно в этом запасе.

Теорема (Упакованное покрытие). [измерено] Член семейства, реализуемый в N битах, подчиняется $3^{E_t} 2^M \leq 2^{N-1}$. Подстановка минимального $E_t = \log_3(b+1)$ даёт

$$m + \log_2 3 \cdot \log_3(b+1) = m + \log_2(b+1) \leq N-1,$$

что тождественно однородному двоичному бюджету из теоремы о бюджете, поскольку $\log_2 3 \cdot \log_3 x = \log_2 x$.

Следствие (Запас — артефакт позиций). [измерено] Троичное кодирование порядка не даёт никакого бюджетного запаса при равном объёме хранения. Величина $0.3691E$ из следствия о запасе существует только при равном числе *позиций*, то есть на кристалле, где позиция физически троична.

Измеренные при равном объёме хранения против каталога, 6 из 17 выбранных форматов оказываются доминированы — а не все они. Каждый однородный двоичный формат, тратящий все свои биты, сходится ровно вничью: binary16 на 15.06 против бюджета 15, binary32

на 31.10 против 31, binary64 на 63.27 против 63. Доминированы те, кто бюджет теряет: posit32 на 5.12 позиции, posit64 на 14.24, takum32 на 0.82, tekum32 на 0.90, `cray_float` на 0.65 и наш собственный gf8 на 0.70. Несколько сужающихся форматов уходят вовсе: posit8 на 1.16, posit16 на 2.37, takum16 на 1.53, tekum16 на 1.49. Это согласуется со следствием о побеге — побег требует неоднородности, сужение неоднородно, а однородное семейство не может догнать его при равном объёме хранения.

Почему этого не заметили. Тот же дефект появляется в статье дважды в других местах: как теорема об отсутствии дармовщины и как наблюдение, что TNF4 с $E_t=2$ не существует в четырёх битах, потому что $3^2=9>8$. Оба были записаны как локальные факты об упаковке, и ни один не был перенесён обратно на результат, опирающийся на ту же величину. Мы принимаем правило: поправка, обесценивающая сравнение, должна применяться к каждому утверждению, опирающемуся на эту величину, а не только к тому, где она всплыла.

Трит — это и есть запас

Следствие об отсутствии выживших — факт про список. Оно становится фактом про пространство проектирования, как только спросить, что конкуренту пришлось бы сделать, чтобы сбежать.

Теорема (Условие покрытия). [доказано] $\mathcal{F}_N$ содержит член, доминирующий формат с эффективной мантиссой m и диапазоном b бинад, тогда и только тогда, когда

$$m + \lceil \log_3(b+1) \rceil \leq N-1.$$

Доказательство. Член E_t имеет мантиссу $N-1-E_t$ и диапазон $3^{E_t}-1$. Доминирование требует $N-1-E_t \geq m$ и $3^{E_t}-1 \geq b$, то есть $\lceil \log_3(b+1) \rceil \leq E_t \leq N-1-m$. Такое целое существует ровно тогда, когда выполнено указанное неравенство. ■

Теорема (Бюджет позиций однородного формата).

[доказано] Пусть *однородный* формат ширины N несёт знак, поле масштаба из E двоичных позиций и сигнификанд из m бит, удерживаемый при каждой представимой величине. Тогда $b+1 \leq 2^E$ и $1+E+m \leq N$, откуда

$$m + \log_2(b+1) \leq N-1.$$

Два бюджета — это одна и та же бухгалтерия в разных валютах, и обменный курс между ними и есть весь результат.

Следствие (Троичный порядок — это и есть запас).

[доказано] Каждый однородный двоичный формат ширины N доминируется членом $\mathcal{F}_N$ с запасом

$$\Delta = E(1 - \log_3 2) = 0.3691 E \text{ позиций,}$$

строго положительным при $E \geq 1$ и растущим линейно по собственной ширине порядка конкурента. *Доказательство.* По теореме о бюджете позиций $b+1 \leq 2^E$ и $m \leq N-1-E$. Тогда $m + \log_3(b+1) \leq (N-1-E) + E \log_3 2 = (N-1) - E(1 - \log_3 2)$, что удовлетворяет условию покрытия с указанным запасом. ■

Измерено, в позициях: 1.48 у `fp8_e4m3`, 1.85 у `binary16`, 2.95 у `binary32`, 4.06 у `binary64`, 5.54 у `binary128`. Этот запас — не совпадение каталога; это 0.3691 на каждый бит порядка, который тратит конкурент.

Это стоит сказать прямо, потому что оно идёт вразрез с теоремой об отсутствии дармовщины, которая говорит, что упакованный троичный порядок никогда не несёт больше значений *на бит*, чем двоичный, — и которую мы подтвердили в кремнии: BNF16 и TNF16 различаются на 1% по LUT. Оба верны, но на разных осях. На *позицию* трит покрывает $\log_2 3$ бинады против одной у бита — отсюда и берётся Δ ; на *бит* двоичного кристалла упаковка отдаёт всё это обратно. Троичное

кодирование выигрывает на числовой оси и выходит в ноль на кремниевой, а утверждение, не называющее свою ось, — не утверждение.

Следствие (Побег требует неоднородности). [доказано] Формат ширины N , убегающий от $\mathcal{F}_N$, обязан нарушить гипотезу теоремы о бюджете позиций: либо его сигнификанд удерживается не при каждой величине (сужение), либо его поле масштаба находится вне слова (блок).

Граница точна и не пуста. Равенство в теореме о покрытии достигается по построению: правило ширины тратит каждую позицию. Среди каталогизированных форматов `posit16` подходит к ней на 1%: $10.55 + \log_3 113 = 14.85$ против бюджета 15. А путь побега из следствия о побеге — не гипотетический, а настоящий: тот же `posit16`, если оценивать его там, где наклон и задуман жить, а не усреднять по всему диапазону, несёт 12 бит дроби, сохраняя все 112 бинад:

$$12 + \log_3 113 = 16.30 > 15.$$

Он сбегает — и правильно делает: концентрировать точность там, где сидит нагрузка, — это и есть *назначение* наклона. Граница, которую не может нарушить ни один формат, ничего бы не мерила.

Что отсюда следует ожидать. Следствие о побеге называет ровно два выхода, и оба — открытые вопросы, а не результаты этой работы. Путь наклона ограничен теоремой о пороге разрешения и стоит того, что меряет раздел про кремний. Путь блока — МХ и его потомки, где один масштаб обслуживает 32 элемента, — лежит вне всех сформулированных здесь бюджетов, потому что масштаб не внутри слова. Мы сообщаем о нём как о рубеже этой работы, а не как об удержанной территории.

Теорема (Преобразования и форматы не композируются). [доказано] Пусть $D^*(b, p)$ — наилучшее достижимое искажение при b битах на элемент при фиксированном предварительном преобразовании p . Преобразование меняет D^* тем, что меняет

источник, который оно предъявляет; формат выбирает разбиение, дающее некоторое $D \geq D^*$. Поэтому два рычага, оба действующие на разбиение, делят один потолок, и их выигрыши суб-аддитивны; а преобразование и формат действуют на разные аргументы, и их композиция зависит от порядка.

Измеренная зависимость от порядка доходит до $2.78\times$: применение одних и тех же двух рычагов в двух порядках даёт разные результаты, а это запрещает объявлять любой из них самостоятельным множителем. Мы отмечаем это, потому что сами совершили эту ошибку: ранняя версия этой работы считала отношение одного рычага против проигрывающей базы и подавала рычаги как почти мультипликативные. Они не такие.

Куда блочная литература не смотрит

Блочная ось — это тот путь побега, который следствие о побеге оставляет открытым, и именно там сейчас сосредоточено внимание отрасли. Стоит быть точным насчёт того, что эта работа варьирует, а что держит фиксированным.

Спецификация *microscaling* от Open Compute фиксирует блок из 32 элементов, делящих один масштаб UE8M0 — голая степень двойки, без мантиссы, — при элементе в E2M1. NVFP4 от NVIDIA меняет две вещи: блок сжимается до 16, а общий масштаб становится float E4M3, а не степенью двойки. Оба живут в железе современных ускорителей — вот почему условие остановки для этой работы сформулировано против них, а не против *posit* или *takum*.

Литература 2026 года варьирует почти исключительно *преобразование*, применяемое перед квантованием, а не формат под ним: обучаемая поблочная оптимизация ради устойчивости к выбросам, округление, подогнанное под множество уровней E2M1, дополненные остаточные каналы, связка с разреженностью активаций. Каждая работа сообщает о выигрышах, и каждая держит формат элемента на E2M1.

Именно эту переменную и адресует теорема об оптимальности. E2M1 тратит две из трёх своих беззнаковых позиций на порядок — *внутри блока, который и так несёт общий масштаб*. Правило ширины говорит:

порядок размерен под диапазон, который посещает нагрузка, а внутри блока, нормированного собственным максимумом, этот диапазон измерим, а не постулируем. Если он уже четырёх бинад, то E2M1 покупает диапазон, за который блок уже заплатил, и позиции принадлежат мантиссе. Мы меряем внутриблочный размах напрямую и позволяем правилу назвать победителя *до* вычисления перплексии; раздел о блочной границе сообщает, что вернулось, включая случай, когда правило ошиблось.

Это более узкое утверждение, чем может показаться. Оно ничего не говорит о преобразованиях, которые композируются с любым форматом элемента и чьи выигрыши мы не оспариваем. Наша собственная теорема о некомпозируемости говорит, что преобразования и форматы действуют на разные объекты — преобразование двигает потолок, формат его делит, — так что улучшение элемента ортогонально улучшению поворота, и ни одно не поглощает другое.

Что на самом деле меряет таблица пропускной способности

Строка, которую эта работа оставила помеченной *не прогонялась*, оказалась самой информативной во всей таблице. Декодер `binary32` — это

```
assign fp32_out = binary32_in;
```

просто провод, потому что `binary32` и есть `fp32`. Его сверка поэтому точна по построению, а его 472 LUT — это один только стенд: LFSR, аккумулятор и свёртка на выходе, и никакого декодера внутри.

Это калибровка, которую данная работа никогда не снимала. Каждая строка таблицы лежит между 463 и 1008 LUT, значит *стенд составляет от 47% до 102% каждого измерения*, а для верхних десяти строк он никогда не ниже 80%. Отсюда два следствия, и оба структурные, а не лечимые добавлением семян.

Предложение (Доминирование стенда). Пусть измерение соединяет фиксированный стенд стоимости H с декодером стоимости D_f и сообщает $R_f = M_f / (H + D_f)$. Тогда

$$\frac{\partial R_f}{\partial D_f} = -\frac{M_f}{(H+D_f)^2},$$

что стремится к $-M_f/H^2$ при $D_f/H \rightarrow 0$. Когда $H \gg D_f$, отчёт меняется почти целиком вместе с M_f , а стоимость декодера из него выжимается. Здесь $H=472$ и $D_f \in [-9, 536]$, причём $D_f < 115$ для верхних десяти строк.

Предложение (Отсечение по выходному пространству).

Декодер с n входными битами имеет не более 2^n различных выходов. Любая логика, наблюдающая только эти выходы, может быть специализирована под этот образ, поэтому её синтезированная стоимость не возрастает по n независимо от собственной сложности декодера. Сравнение при разных n поэтому смешивает конструкцию формата с шириной формата, и только сравнение внутри одного n изолирует конструкцию.

Предложение об отсечении — это не опасение, оно видно в числах.

GFternary берёт 2 входных бита и меряется в 463 LUT — *на девять меньше, чем голый провод*, — потому что четыре различных выхода позволяют синтезу специализировать стоящий за ним аккумулятор. Измеренный отдельно, против минимального стока «регистр и свёртка», голый провод стоит 112 LUT, а **GFternary** — 66. Формат не может быть дешевле провода; меряется здесь ширина его образа.

Таблица. Те же измерения, сгруппированные по ширине хранения — единственная группировка, которую предложение об отсечении оставляет допустимой. Наши выделены жирным.

бит	формат	LUT	МГц	МГц/LUT
2	GFternary	527	82.93	0.1574
8	fp8 e4m3	549	68.30	0.1244
8	fp8 e5m2	544	66.05	0.1214
8	minifloat	622	66.87	0.1075
8	GF+8	692	61.06	0.0882
8	posit8	614	44.71	0.0728
10	GF10	597	70.96	0.1189
10	TNF8	609	65.08	0.1069
14	GF14	651	69.91	0.1074
16	TNF16a	566	69.77	0.1233
16	TNF16c	591	68.35	0.1157
16	TNF16b	604	63.31	0.1048
16	binary16	586	60.90	0.1039
16	takum16	853	59.31	0.0695
16	LNS16	722	43.15	0.0598
16	posit16	731	35.43	0.0485
17	TNF17e	596	71.94	0.1207
17	TNF17e-bare	605	64.87	0.1072
17	BNF16	635	65.68	0.1034
17	TNF16	629	63.22	0.1005
32	binary32	536	76.23	0.1422
32	VAX F	624	58.75	0.0942
32	posit32	1031	31.18	0.0302
32	IBM hex32	1072	31.30	0.0292

Прочитанная так, таблица говорит нечто иное, чем её ранжирование, и разницу стоит выговорить прямо.

Где мы встречаем конкурента на той же ширине, а где нет. Из девяти классов ширины шесть содержат только наши форматы, а два — только чужие. *Один класс содержит и тех и других:* на 8 битах **GF+8** даёт 0.1004 против 0.1254 у **fp8 e4m3**, и мы проигрываем на 19.9%. В колонке сверки у этой строки написано *селектор живой; референса нет*, так что это же единственная наша строка, которую никогда не проверяли против оракула — проигрыш и отсутствующая проверка суть один и тот же долг.

Ближайшее честное сравнение — 16 против 17 бит, разница в один бит:

BNF16 (17 бит) против	
binary16 (16 бит)	−3.1%
takum16 (16 бит)	+57.4%
LNS16 (16 бит)	+79.8%
posit16 (16 бит)	+113.4%

Против каждого *исследовательского* формата сопоставимой ширины — takum, LNS, posit, то есть тех форматов, с которыми эта статья на самом деле спорит, — наша 17-битная конструкция выигрывает от половины до чуть более чем вдвое. Против IEEE half она проигрывает три процента, а против IEEE single не может выиграть вообще, потому что декодер IEEE single — это провод, и никакой декодер не побьёт отсутствие декодера.

Вот защитимое утверждение, и оно уже, чем подсказывает ранжирование. *Общее первое место принадлежит двухбитному формату, у которого в каталоге нет конкурента той же ширины, а второе — тождественной функции.* Ни один из этих фактов не был виден, пока таблица сортировалась по одному числу.

Сверка не сертифицирует спецификацию

GF+8 был последней нашей строкой без оракула, и это тот единственный класс, где наш формат встречает конкурента на той же ширине и проигрывает. Написание оракула закрыло оба вопроса и подняло третий, который этой работе прежде не приходилось задавать.

GF+A — адаптивный контейнер: двухбитный заголовок выбирает один из четырёх карманов над общей восьмибитной полезной нагрузкой — $\phi\text{-e3m4}$ (1,3,4) со смещением 3, e2m5 (1,2,5) со смещением 1, симметричная фиксированная точка с масштабом $1/64$ и слот, объявленный логарифмическим. Реализовано по этим четырём наборам параметров и сравнено с модулем: **1024 из 1024 кодов точны, ноль расхождений.** Декодер верен.

Четвёртый карман — нет.

Предложение (Сверка не сертифицирует спецификацию).

Пусть D реализует f точно на всём C . Сверка устанавливает $D=\rho \circ f$ и ничего не говорит про f . В частности, $|f(C)|$ может быть сколь

угодно меньше $|C|$, и декодер, добросовестно воплощающий вырожденное f , одновременно и соответствует спецификации, и бесполезен.

Логарифмический карман выдаёт 2^{-s} с тождественно нулевой мантиссой, где $s = \lfloor i/16 \rfloor + \lfloor i/32 \rfloor$ и $i = 127 - |w|$. Это лестница степеней двойки, и она даёт **семнадцать различных значений из 256 кодов** там, где каждый из трёх остальных карманов даёт 255. Девяносто три процента кодового пространства этого кармана не отображаются ни во что новое. Собственный комментарий объявляет масштаб $i \cdot 8/127$, тогда как код реализует $i \cdot 3/32$, в полтора раза круче — второй раз в этом каталоге, когда строка RTL противоречит комментарию рядом с ней.

Вырожденность не безобидна для таблицы, и направление её эффекта — то самое, которое эта работа видела уже одиннадцать раз. Перестроить этот карман как `lns8`, который данный репозиторий и так объявляет — знаковое семибитное лог-поле при `frac_bits = 3`, значение $2^{L/8}$, что даёт 255 различных значений, — стоит того же, чего всегда стоит доведение декодера до конца:

карман 11	LUT	МГц	МГц/LUT
как опубликовано (17 значений)	170	313.38	1.8434
как <code>lns8</code> (255 значений)	182	271.22	1.4902
	+7.1%	-13.5%	-19.2%

Итак, 0.1004 у `GF+8`, который и без того проигрывает 0.1254 у `fr8 e4m3` на 19.9% в единственном очном классе каталога, сам по себе раздут карманом, выбрасывающим большую часть собственного кодового пространства. После починки он проигрывает больше. *Мы сообщаем об этом, потому что это правда и потому что альтернатива — число, стоящее на вырожденном компоненте, — ровно тот сорт утверждения, который эта статья двадцать семь разделов подряд разбирает у других.*

На пятидесяти четырёх семенах утверждения расходятся

Раздел о полосах разрешения оставил ранжирование распадающимся на семь уровней на пяти семенах, с порогом разделения 5.50% и порогом обнаружения разницы в стоимости декодера в 41.3 LUT — 79%

целого декодера. И то и другое — следствия объёма выборки и того, что сообщается медиана, чья эффективность при $n=5$ равна 0.698. Поэтому девять форматов, на которых стоят утверждения статьи, были прогнаны заново на **пятидесяти четырёх семенах**, и сообщаемая статистика — **среднее**.

Порог разделения падает с 5.50% до **1.43%**.

Таблица. Пятьдесят четыре семени, среднее с 95% доверительным интервалом, всё на общем 128-битном стенде. Тест Уэлча против **binary16** с поправкой Холма по восьми сравнениям; выживает каждое.

	МГц/LUT	±95%	против binary16	p_{Holm}
GTernary	0.1584	0.0018	+46.99%	$<10^{-300}$
binary32	0.1429	0.0013	+32.53%	$<10^{-300}$
fp8 e4m3	0.1225	0.0012	+13.68%	1.9×10^{-63}
TNF16a	0.1214	0.0012	+12.59%	8.1×10^{-56}
fp8 e5m2	0.1188	0.0011	+10.26%	5.6×10^{-41}
GF10	0.1172	0.0012	+8.70%	5.6×10^{-28}
TNF17e	0.1155	0.0013	+7.17%	2.1×10^{-17}
TNF16c	0.1152	0.0010	+6.92%	6.9×10^{-21}
binary16	0.1078	0.0012	—	—

Запасы меньше, чем подсказывали оценки на пяти семенах: **TNF16c** читался там как +12.5%, а здесь читается как +6.92%, — и это ожидаемое направление: точечная оценка, взятая около порога разрешения, с равной вероятностью льстит и занижает, а бóльшая выборка убирает лесть вместе с шумом.

Утверждение, которое эта статья делает при равном хранении, — +6.92% над IEEE half, с 95% интервалом ± 0.0010 на 0.1152 и $p_{\text{Holm}} = 6.9 \times 10^{-21}$. [источник в тексте] У **TNF16c** шестнадцать хранимых бит против шестнадцати у **binary16**, его внеспецификационные смещения зарезервированы, так что каждый код либо определён, либо помечен, а его порядок охватывает ± 121 — внутри окна, которого требует предложение об окне порядка.

TNF17e с +7.17% и **TNF16c** с +6.92% не разделены *между собой*: их интервалы перекрываются. Статья заявляет пару, а не порядок внутри неё.

Почему лестница останавливается на пяти тритах, когда мишень — fp32

Каждый декодер в этом каталоге формирует порядок fp32 одинаково:

```
wire [7:0] e32 = e[7:0] + 8'd127;
```

что верно, только пока $e+127$ лежит в $[0, 255]$. Вне этого окна сложение заворачивается, а не насыщается, и завёрнутый порядок — это неверное конечное число, а не переполнение, поэтому ничто ниже по потоку его не обнаружит. У TNF32 смещения 364, 620 и 876, чьи порядки равны 0, 256 и 512, все декодируются ровно в 1.0.

Предложение (Окно порядка ограничивает рунг). Рунг с E_t тритами имеет $E = (3^{E_t} - 1)/2$, а его порядок охватывает $[1 - E, 3^{E_t} - 2 - E]$. Этот охват лежит внутри $[-127, +128]$ ровно тогда, когда $3^{E_t}/2 \leq 128$, то есть

$$E_t \leq 5.$$

$3^5/2 = 121$ помещается, а $3^6/2 = 364$ нет, так что на декодере, выдающем fp32, лестница допускает не более пяти тритов порядка. Более широкий рунг должен либо насыщаться за окном, либо выдавать больше 32 бит, и то и другое меняет его стоимость.

Граница резкая, и падает она ровно туда, куда следствие о рунгах поместило рунг наименьшей потери. $E_t = 5$ — одновременно то k , где $w(k)$ минимально ниже $k = 10$, и самый широкий порядок, который декодер fp32 может нести без заворачивания. Совпадение двух никак не связанных ограничений — одно арифметическое, другое про целевой тип, — и именно поэтому TNF16с и TNF17е суть те два рунга, которые эта статья рекомендует.

Три рунга сняты за нарушение этого правила — вручную. Описанный далее гейт **описан, но не реализован**: файла tools/check_exponent_window.py нет ни в рабочем дереве, ни в истории репозитория (`git log --all --diff-filter=A -- tools/`

`check_exponent_window.py` не даёт ни одного добавляющего коммита), и ни один рабочий процесс непрерывной интеграции на него не ссылается. Предполагалось, что он падает на любом рунге, чей охват выходит за окно, и перечисляет снятые явно, а не удаляет их, — гейт, молча игнорирующий заведомо плохую запись, это способ вернуть дефект. Описанная проверка «на красное» удалением этого списка не проводилась и провестись не могла: проводить её нечем. Пока правило держится только на дисциплине авторов.

Инструмент разрешает семь уровней, а не двадцать восемь

До этого места таблица пропускной способности хранила по строке медиану и разброс. Медиана с разбросом не может ответить, различаются ли две строки, а значит *ни одна упорядоченность в этой статье никогда не проверялась*. Каждая строка была прогнана заново с сохранением всех пяти частот по семенам, и из этого вышло две вещи.

Таблицу мерили на двух стендах. Семнадцать из двадцати одной строки воспроизвели счёт LUT точно; четыре не сошлись. Причина точна: две строки гонялись от 128-битного LFSR, а остальные — от 64-битного. Перемер `GFTernary`, который воспроизводится на 463 LUT на узком стенде, даёт 527 на широком — **разница ровно 64 LUT, и ровно 64 для всех двадцати одной строки**. Значит, две строки ранжировались против девятнадцати других, неся 64 LUT стенда, которых у остальных не было. Всё в этом разделе перемерено на широком стенде, так что сравниваются декодеры, а не обёртки.

Ранжирование распадается на семь уровней. Проверка каждой соседней пары на перекрытие их пятисеменных диапазонов даёт семь полос, и одна из них держит двадцать форматов. *Полоса — это цепочка перекрытий, а не класс эквивалентности*; два члена на противоположных концах цепочки всё же могут быть разделены, поэтому полезная проверка — попарная и прямая.

полоса	члены
1	GFternary
2	binary32
3	двадцать форматов, от fp8 e4m3 до GF+8
4	posit8 , takum16
5	LNS16
6	posit16
7	posit32 , IBM hex32

GFternary один в первой полосе: 0.1524–0.1616 против 0.1376–0.1474 у **binary32**, без перекрытия. Этот результат стоит. Внутри полосы три тонкая упорядоченность — нет: ни один читатель не должен заключать из ранга, что **TNF16** бьёт **BNF16**, потому что инструмент этой разницы не видит.

Что отделено от IEEE half при равном или почти равном хранении. У шести наших форматов пятисеменные диапазоны целиком выше диапазона **binary16** 0.0985–0.1071:

	против binary16	пятисеменной диапазон
GFternary	+51.4%	0.1524–0.1616
TNF16a	+18.6%	0.1192–0.1278
TNF17e	+16.1%	0.1135–0.1238
GF10	+14.4%	0.1134–0.1204
TNF32	+13.2%	0.1137–0.1243
TNF16c	+11.3%	0.1118–0.1210

TNF16c — это шестнадцатибитный рунг при $E_t = 5$ с зарезервированными внеспецификационными смещениями: хранение равно **binary16**, каждый код либо определён, либо помечен, и он отделён выше. Раздел о рунге ровно в шестнадцать бит сообщал это сравнение как +5.1% с узкого стенда; на общем стенде оно составляет +11.3%, и это первое утверждение в статье, сформулированное диапазоном, а не точкой.

Порядок рангов, который инструмент не может разрешить, — не результат. Таблица выше напечатана с диапазонами именно поэтому, и любое утверждение о порядке в других местах статьи, попадающее внутрь полосы три, следует читать как неразрешённое.

Троичный порядок никогда не замощает двоичное поле

Раздел о рунге ровно в шестнадцать бит закончился дилеммой: поле смещения рунга А держит 128 значений, а его четыре трита именуют 81, так что либо 37% слова лежит вне формата и должно быть защищено, либо порядок не троичный. У дилеммы есть структурная причина, и её название закрывает дело, а не выбирает сторону.

Теорема (Точного замощения нет). [доказано] Для целых $k, m \geq 1$ выполняется $3^k \neq 2^m$. Значит, для каждого рунга поле смещения ширины $\lceil k \log_2 3 \rceil$ имеет ёмкость строго большую, чем те 3^k смещений, которые оно должно держать, а потерянная доля

$$w(k) = 1 - \frac{3^k}{2^{\lceil k \log_2 3 \rceil}} = 1 - 2^{-(1 - \{k \log_2 3\})}$$

строго положительна при каждом k , где $\{\cdot\}$ — дробная часть.

Доказательство. В разложении 3^k есть тройка, а в 2^m её нет, так что единственность факторизации запрещает равенство при $k, m \geq 1$. Ёмкость равна $2^{\lceil x \rceil}$ для $x = k \log_2 3$, а содержимое равно 2^x ; их отношение есть $2^{-(\lceil x \rceil - x)}$, и $\lceil x \rceil - x = 1 - \{x\}$ всякий раз, когда $x \notin \mathbb{Z}$, что и гарантирует первое предложение теоремы. ■

Следствие (Потеря неравномерна, а хорошие рунги редки). [доказано] $w(k)$ мало ровно там, где $\{k \log_2 3\}$ близко к 1, то есть где 3^k падает чуть ниже степени двойки. При $k \leq 15$ минимум — $k=5$ при $w=5.1\%$; максимум — $k=7$ при $w=46.6\%$.

Таблица. Потеря на каждом рунге и то, какие рунги эта работа построила. Лестница брала $E_t \in \{3, 4, 6, 7\}$, три из них плохие, и никогда не брала $E_t = 5$, который в этом диапазоне лучший вдвое.

E_t	3^{E_t}	b	ёмкость	потеря	построено здесь
1	3	2	4	25.0%	
2	9	4	16	43.8%	
3	27	5	32	15.6%	TNF8
4	81	7	128	36.7%	TNF16, TNF16a
5	243	8	256	5.1%	TNF16c
6	729	10	1 024	28.8%	TNF32
7	2 187	12	4 096	46.6%	TNF64
8	6 561	13	8 192	19.9%	
9	19 683	15	32 768	39.9%	
10	59 049	16	65 536	9.9%	
11	177 147	18	262 144	32.4%	
12	531 441	20	1 048 576	49.3%	
13	1 594 323	21	2 097 152	24.0%	
14	4 782 969	23	8 388 608	43.0%	
15	14 348 907	24	16 777 216	14.5%	

Рунг, который лестница пропустила. $E_t=5$ требует 8 бит смещения, значит $1+8+7=16$: третий шестнадцатибитный кандидат, и тот, о котором эта работа не подумала. Построен и проверен исчерпывающе против эталона лестницы: **62208 из 62208 внутриспецификационных кодов точны, все 3328 внеспецификационных кодов помечены, ноль ошибок.**

	LUT	МГц	МГц/LUT	против binary16
рунг А, без защиты	502	69.87	0.1392	+14.7%
рунг А, смещения зарезервированы	549	64.86	0.1181	-2.6%
рунг С, без защиты	509	66.11	0.1299	+7.1%
рунг С, смещения зарезервированы	527	67.20	0.1275	+5.1%

Защита рунга А стоит 15.1% и уводит формат ниже IEEE half. Защита рунга С стоит 1.8% и оставляет его выше. **Утверждение, которое эта статья может сделать при равном хранении, когда каждый код либо определён, либо помечен, — это +5.1% против IEEE half [измерено] — меньше, чем сообщённые сперва 14.7%, и, в отличие от них, не стоящее на непроверенном поле.**

Лестница, перестроенная. Следствие о рунгах — это правило построения, а не только диагноз, стоит добавить ограничение по диапазону: выбирай то k с наименьшим $w(k)$ среди тех, чей охват $3^k/2$ покрывает класс. На 17 битах это $E_t=5$, $M=8$ — 36.7% потери падают до

5.1% ценой ровно одного бита мантиссы. Сгенерировано из параметров, а не отредактировано руками, защищено и проверено исчерпывающе: **124416 из 124416 внутриспецификационных кодов точны, все 6656 внеспецификационных кодов помечены, ноль ошибок.**

17-битный рунг	E_t	потеря	МГц/LUT	защищён
TNF16, как построен	4	36.7%	0.1005	нет
TNF17e	5	5.1%	0.1207	да

На 20.1% быстрее на LUT — *и при этом* с проверкой поля смещения, чего заменяемый им рунг не делает; обе цифры перемерены на общем стенде из раздела о полосах разрешения. То же правило отправляет 65-битный рунг к $E_t=8$, но этот рунг снят: независимый разбор нашёл, что его тракт порядка держит восемь бит порядка, охватывающего ± 3279 , так что `off` и `off + 256` декодируются одинаково. `TNF32` и `TNF64` несут тот же дефект и сняты вместе с ним; `TNF16c` и `TNF17e` — нет, их порядки охватывают ± 121 внутри окна `fp32`.

Дилемму раздела о шестнадцатибитном рунге не нужно разрешать, потому что она была артефактом стояния на $E_t=4$. Теорема о невозможности замощения говорит, что ни один рунг не уходит от потери; следствие о рунгах говорит, что лестницу надо строить на том k , где потеря наименьшая, — а эта работа построила её на трёх худших и пропустила лучший.

Рунг ровно в шестнадцать бит и чего стоит там победа

Раздел о том, что меряет таблица пропускной способности, оставил эту работу с одним честным сравнением и извинением при нём: наш ближайший к шестнадцатибитным конкурентам рунг был шириной в семнадцать бит, так что всякое утверждение «при той же ширине» несло однобитовую оговорку. Хранимая ширина рунга есть $1 + \lceil E_t \log_2 3 \rceil + M$, и ровно два назначения дают шестнадцать:

	E_t	бит смещения	M	динамический диапазон
A	4	7 (81 смещение)	8	$2^{-39} \dots 2^{39}$
B	3	5 (27 смещений)	10	$2^{-12} \dots 2^{12}$

Который из них — *тот самый* шестнадцатибитный член, решает измерение, а не предпочтение, поэтому построены оба и оба исчерпывающе проверены против эталона лестницы: **65534 из 65534 точны, каждый**. Рунг А выигрывает по пропускной способности, 0.1392 против 0.1144, и выигрывает свой класс ширины начисто:

TNF16a (16 бит) против	
binary16	+14.7%
takum16	+86.3%
LNS16	+112.8%
posit16	+152.6%

Это первая в каталоге победа против IEEE при равной ширине, и это тот результат, которого, как говорил предыдущий раздел, у статьи ещё не было. У него есть и цена, которую нужно печатать рядом.

Цена, как оценивалась и как измерилась. Сначала этот раздел сообщал, что победа рунга А куплена кодовым пространством: четыре трита дают 81 смещение там, где двоичное поле держит 128, так что 37.5% кодов смещения лежат вне формата, и рунг доставит 15.32 эффективных бита против шестнадцати у binary16. *Эта оценка была вычислена, а не измерена, и измерение её опровергает.*

Прогон каждого кода через RTL с подсчётом различных выходов даёт рунгу А 64518 различных значений на 65536 кодов: **утилизация 98.4%** против 100.0% у binary16. Внеспецификационные смещения не выбрасываются — декодер присваивает каждому из них отдельное значение. Победа над IEEE half при равном хранении стоит 1.6% кодового пространства, а не 37.5%, и результат сильнее, чем этот раздел заявлял изначально.

Но остаётся на виду другой дефект. *Декодер не резервирует те 47 смещений, что лежат вне формата:* повреждённое поле смещения даёт правдоподобное конечное значение вместо сигнала. Этим страдает каждый рунг лестницы, и ни один тест в этой работе такого бы не нашёл, потому что декодер, отвечающий на любой вход, отвечает на любой тест.

Таблица. Утилизация кодов, измеренная: различные выходы декодера на прогнанные коды. Исчерпывающе при 2^{18} кодов и ниже, равномерная выборка 200000 выше, с указанием рода прогона, чтобы частичный прогон никогда не читался как полный.

формат	различных	кодов	утилизация
GF+8	454	1 024	44.3%
IBM hex32	105 526	200 000	52.8% (выборка)
GTernary	3	4	75.0%
minifloat	227	256	88.7%
takum16	61 351	65 536	93.6%
TNF16b	61 446	65 536	93.8%
TNF17e	123 398	131 072	94.1%
TNF16c	61 702	65 536	94.1%
TNF8	966	1 024	94.3%
BNF16	129 030	131 072	98.4%
TNF16	129 030	131 072	98.4%
TNF16a	64 518	65 536	98.4%
TNF17e-bare	130 048	131 072	99.2%
VAX F	199 211	200 000	99.6% (выборка)
posit32	199 964	200 000	100.0% (выборка)
binary32	199 997	200 000	100.0% (выборка)
binary16	65 536	65 536	100.0%
GF10	1 024	1 024	100.0%
GF14	16 384	16 384	100.0%
fp8 e4m3	256	256	100.0%
fp8 e5m2	256	256	100.0%
posit8	256	256	100.0%
posit16	65 536	65 536	100.0%
LNS16	65 536	65 536	100.0%

На чём стоит победа. Незащищённый декодер даёт каждому из 47 внеспецификационных смещений отдельное конечное значение, и именно оттуда берётся победа при равной ширине. Защита их — компаратор на поле смещения, недопустимые коды сигнализируют NaN — была построена и прогнана: 41472 из 41472 внутриспецификационных кодов точны, все 24064 внеспецификационных кода помечены, ноль ошибок.

рунг A	LUT	МГц	МГц/LUT
без защиты	502	69.87	0.1392
смещения зарезервированы	549	64.86	0.1181
	+9.4%	-7.2%	-15.1%

`binary16` меряется в 0.1213. Значит, рунг бьёт IEEE half на 14.7%, пока его недопустимые коды необнаружимы, и проигрывает ему 2.6%, как только они обнаруживаются. **Победа при равном хранении держится на том, что поле смещения не проверяется.** IEEE half в такой проверке не нуждается — все его 65536 кодов определены, — так что сравнение идёт между форматом, который обязан платить за защиту, и форматом, которому нечего защищать.

Предложение (Стоимость резервирования). Пусть спецификация формата покрывает $S \subseteq C$ его кодового пространства. Любая реализация либо (i) защищает $C \setminus S$, платя логикой за его распознавание, либо (ii) присваивает значения $C \setminus S$, что расширяет спецификацию до C . Реализации, сохраняющей одновременно и спецификацию, и стоимость варианта (ii), не существует.

Это ставит вопрос под лестницу, а не рядом с ней. Четыре трита именуют 81 смещение; поле, которое их держит, шириной в семь бит и именует 128. Либо порядок троичный и 37% кодов слова лежат вне формата — и тогда предложение о стоимости резервирования выставляет нам счёт в 15.1%, и сравнение с IEEE проиграно, — либо порядок есть семибитное двоичное поле, все коды определены и победа стоит, но тогда *троичная структура, именем которой названа эта статья, не дотягивается до 37% собственного кодового пространства*. Оба прочтения защитимы, и одновременно их использовать нельзя, — вот почему число выше появляется дважды, а не один раз.

У нас нет измерения, которое это решает, и мы говорим об этом, а не выбираем прочтение, льстящее результату.

И механизма, который этот раздел предполагал, там нет. Утверждалось, что метрика пропускной способности награждает формат за выбрасывание кодового пространства, — на основании предложения об отсечении и трёх точек. По всем 23 измеренным форматам корреляция между утилизацией и МГц/LUT равна **+0.173** —

$t=0.80$ при 21 степени свободы, не значимо и *противоположного* знака по отношению к предсказанному. Два наименее эффективных формата каталога, GF+8 с 44.3% и IBM hex32 с 52.8%, сидят на местах 17 и 22.

Гипотеза снята. Предложение об отсечении остаётся верным в том виде, как оно сформулировано, — меньший образ действительно допускает специализацию, — но вывод, будто выброшенные коды поэтому покупают измеримую пропускную способность, не переживает столкновения с двадцатью двумя измерениями. Три точки с ним соглашались, а двадцать три — нет, и это ровно та причина, по которой оговорку лучше измерять, чем вычислять.

Одна поправка касается нашего собственного заголовочного утверждения. Эта статья заявляла, что GFTernary «использует все четыре своих кода и ничего не выбрасывает». Это не так: $\{-\varphi, 0, +\varphi\}$ — три значения в двух битах, а измеренная утилизация равна 75.0%, третья снизу в каталоге. Само измерение пропускной способности этой поправкой не затронуто, но говорить о первом месте больше нельзя: эрратум от 13 августа отзывает его по независимой причине — int8 на том же стенде даёт 0.1736 против 0.1584 MHz/LUT, — а фраза, защищавшая его здесь, была ложной и отзывается вместе с ним.

Когда сверка меряет саму себя

Три из расхождений между декодером и его эталоном, о которых сообщала эта работа, расхождениями насчёт декодера не были. Их стоит сформулировать предложением, потому что отказ здесь безмолвный: каждое участвующее число верно, а у теста нет сигнала, который мог бы выявить ошибку.

Теорема (Корректность постановки сверки). [доказано] Пусть $f:C \rightarrow \mathbb{R}$ — точное отображение значений формата на его кодовом пространстве, пусть $D:C \rightarrow F$ — декодер в конечный плавающий тип $F \subset \mathbb{R}$, и пусть $\rho:\mathbb{R} \rightarrow F \cup \{\perp\}$ — корректное округление, причём $\rho(x) = \perp$, когда у x нет конечного образа в F . Соответствие D — это предикат

$$D(c) = \rho(f(c)) \text{ на } \{c \in C : \rho(f(c)) \neq \perp\}.$$

Предикат $D(c) = f(c)$ ему не эквивалентен. Он ложен при каждом c с $f(c) \notin F$, а это множество определяется одними только динамическими диапазонами f и F — оно одно и то же и для верного D , и для произвольного.

Доказательство. F конечно и $f(C) \subset \mathbb{R}$, так что $f(c) \in F$ выполняется только на тех кодах, чьё точное значение принадлежит F . Для c вне этого множества из $D(c) \in F$ следует $D(c) \neq f(c)$, каким бы ни было D , включая $D = \rho \circ f$. Значит, второй предикат ложен на множестве, не зависящем от D , тогда как первый истинен на всей ограниченной области определения при $D = \rho \circ f$. ■

Следствие (Порог ложной ошибки). [измерено] Сверка, опускающая ρ , сообщает как ошибку по крайней мере $|\{c : f(c) \notin F\}|/|C|$ кодового пространства — для любого декодера. Для IBM hex32, чей диапазон есть $16^{\pm 64}$, против fp32 этот порог равен 45.0%, измеренный на равномерной выборке в 40000 кодов; 28%, ранее сообщённые для этого декодера, лежат целиком под ним и потому были неинформативны. Применение ρ переводит сопоставимое множество с 22001 на 30245 кодов, а расхождение — в ноль.

Предложение (Расстояние между вариантами — не ошибка декодера). Пусть D реализует вариант f_1 , тогда как эталон вычисляет $f_2 \neq f_1$. Измеренное расхождение равно $|\{c : \rho(f_1(c)) \neq \rho(f_2(c))\}|$ — свойство одной лишь пары (f_1, f_2) . Никакая сверка не может отделить это от ошибки декодера, потому что согласие — её единственный сигнал, а оба варианта внутренне непротиворечивы. Вариант поэтому должен фиксироваться декларацией, внешней по отношению к тесту.

Предложение о расстоянии между вариантами здесь не гипотетическое. Два формата этого каталога публикуют больше одного варианта, и оба сравнивались не с тем. `takum16` реализует логарифмический `takum` со значением $(-1)^{S_e \ell/2}$, а сравнивался с линейным, значение $(1+M/2^p)2^c$: отчитанное расхождение 98,7%, фактическое — 632 из 61,505, и всё это на границе денормалей `fp32`. `lns16` реализует масштаб 128, а сравнивался с масштабом 256: отчитано 253, фактически 107, из которых три — это соглашение о нуле и NaN, а 104 — точность ниже *ulp* в субнормальном диапазоне. Каждый RTL называет свой вариант в заголовочном комментарии, и прочитать его не стоило ничего.

Лекарство механическое, но пока только описано, а не применяется. Предполагалось, что `conformance/variant_map.json` хранит для каждого формата эталонный модуль, с которым идёт сравнение, и токен, который обязан появиться в собственном заголовке декодера, а `tools/check_variant_declared.py` падает, если объявления нет или если карта и заголовок расходятся. **Ни карты, ни инструмента в репозитории нет** — ни в рабочем дереве, ни в истории: `git log --all --diff-filter=A` не даёт добавляющих коммитов ни для `conformance/variant_map.json`, ни для `tools/check_variant_declared.py`, и ни один рабочий процесс непрерывной интеграции на них не ссылается. Описанная проверка гейта «на красное» — привязать `lns16` к неверному масштабу и убедиться, что он загорается красным — не выполнялась. Исправленные ниже числа получены разовым ручным прогоном, и от повторения этого дефекта ничто автоматически не защищает.

Таблица. Отчитанное расхождение против расхождения, измеренного корректно поставленным способом. Первый столбец — то, что эта работа опубликовала; второй — то, что тот же декодер получает, когда эталон округлён к типу декодера и используется объявленный вариант. Ни один декодер не менялся ради второго столбца, кроме случаев, о которых говорит третий столбец.

формат	отчитано	корректно поставлено	в чём была разница
takum16	98,7%	632/61,505	линейный эталон, логарифмический декодер
IBM hex32	28%	0/30,245	не было ρ ; затем добавлен субнормальный путь
LNS16	253	107/65,536	эталон на масштабе 256, декодер на масштабе 128
VAX F	151	0/40,000	не было ρ ; затем добавлен субнормальный путь
posit16	4	0/65,535	два дефекта режима на насыщении, починены
posit32	36	0/40,000	одна ветвь <code>casez</code> недостижима, починена

Все шесть теперь либо нули, либо объяснены, а два настоящих были починены, а не отчитаны. Именно это соотношение и есть полезная часть раздела: тест, который не может ошибиться насчёт самого себя, нашёл бы два дефекта вместо шести, а четыре ложных стоили больше внимания, чем два настоящих.

Два настоящих дефекта — и канал, который об одном из них уже докладывал. `posit16` ошибался ровно на четырёх кодах: на двух значениях насыщения и их отрицаниях, каждый раз на $2^{\pm 2}$ — один шаг поля порядка при $es=2$. Причиной были две независимые ошибки на единицу: длина «режим плюс терминатор» занижалась на единицу, когда серия заполняет слово, и бит-терминатор читался как порядок; а нулевая ветвь подсчёта ведущих бит присваивала $k=13$ там, где серия из пятнадцати одинаковых бит даёт $k=14$ — причём собственный комментарий рядом говорил $k=+14$, а код выдавал 13.

У 48 неверных значений `posit32` причина проще. Один шаблон `casez` был написан шириной в тридцать два символа в тридцатиоднобитном `case`. Verilog обрезает слева, из-за чего ветвь $lzc=11$ стала посимвольно равна ветви выше — недостижимой, — и каждый код ровно с одиннадцатью ведущими нулями проваливался в `default`. Две итерации это принимали за арифметику режима.

Компилятор говорил об этом на каждом без исключения запуске:

```
warning: Extra digits given for sized binary constant.
```

Стенд перенаправлял вывод компилятора в лог-файл, а лог читали только тогда, когда что-то падало. *Предупреждение, которое никто не читает, — не предупреждение*; это инструмент с отключённым выходом, и это тот же самый дефект, что и инструмент внутри области отказа, только заметить его труднее — потому что инструмент работает

безупречно. `tools/check_literal_widths.py` [НЕ РЕАЛИЗОВАН, W845] теперь превращает этот класс в код возврата: он проверяет 188,021 литерал с заданной шириной, закрывает гейтом те пути, которые измеряет эта статья, и печатает — а не прячет — те 41 вхождение, которые находит в тестовых стендах за пределами этой области.

Инструмент находился внутри той самой области отказа, которую он измерял — та же ошибка, только в другой среде, что и диагностика сетевой неисправности инструментом, работающим по этой самой сети.

Что это решает

Вывод мы формулируем ровно с той силой, которую несут измерения, — а это утверждение о тракте данных, а не о числе.

Для троичного тракта данных — такого, где веса суть коды и в котором, следовательно, нет умножителя, — пара {GF Ternary, TNF} является эталонным форматом.

Слово *эталонный* держат три вещи, и каждая из них — измерение, а не предпочтение.

Она полна. У троичного узла три места, где нужен формат (см. раздел о паре); алфавит весов закрывает одно, лестница аккумулятора — другое, а отсчёту ничего не нужно. Ни одна другая пара в литературе не закрывает оба: методы троичных сетей отвечают на вопрос веса и оставляют аккумулятор в двоичном float, спроектированном для тракта, который умножает, а литература по форматам отвечает на вопрос аккумулятора, предполагая, что умножитель существует.

Она вынуждена, а не выбрана. Основание алфавита — единственное $r > 1$ с $r^2 = r + 1$ (см. теорему о золотом алфавите). Разбиение «порядок — мантисса» есть решение задачи максимизации с ограничениями, в которой ограничение по диапазону активно (см. теорему об оптимальном разбиении), так что свободного параметра не остаётся,

как только диапазон нагрузки измерен. Ни то ни другое не выбиралось по вкусу, и то и другое читатель, который с нами не согласен, может вывести заново.

Она предсказательна. Ошибка подчиняется замкнутой форме, зависящей от одной только ширины мантиссы, так что проектировщик может определить размер аккумулятора до того, как построит его, — и этот подбор размера проверен на реальном захвате из эфира и на реальных троично квантованных весах, а не на синтетических данных.

И чем она не является. Это не лучший формат для тракта, который умножает: там держит позицию блочная ось (MX, NVFP4), и мы ничего не утверждаем. Это не выигрыш 7% у `int8`, потому что `int8` не несёт диапазона и делает не ту же работу. И результат о фронтире был бы пустым без причины, по которой каждый конкурент сидит ниже него, — поэтому каждая такая причина здесь теорема, а не строка таблицы.

Где это стоит в шестидесятивосьмилетнем споре

Утверждение, что тройка — лучшее основание, не наше, и назвать, чьё оно, — значит сделать вклад этой статьи меньше и защитимее.

Те, кто выстроил аргумент. Экономичность основания $\rho(r) = r/\ln r$ минимальна при $r=e$, а тройка — ближайшее целое: троичность сидит на 0,46% выше оптимума там, где двоичность — на 6,15% выше. Аргумент был выдвинут механически прежде, чем электронно: Томас Фаулер построил уравновешенно-троичную счётную машину в Девоне около 1840 года, и записанная мотивация была той же — экономия разрядов. Работающим компьютером это стало в 1958-м, когда **Николай Брусенцов** построил *Сетунь* в Московском государственном университете при поддержке **Сергея Соболева**, используя пары ферритовых сердечников, чтобы держать три устойчивых состояния. Между 1959 и 1965 годами выпустили примерно пятьдесят машин. За ней Брусенцов сделал *Сетунь-70* в 1970-м, чьи идеи системы команд предвосхитили доводы, позже независимо высказанные в пользу RISC; она так и не дошла до серийного производства, и троичная линия была закрыта административно, а не технически. **Дональд Кнут** держал идею в обороте, назвав уравновешенную троичную систему «пожалуй, самой красивой системой счисления» в *Искусстве программирования*.

Линия продолжается скорее в работах по приборам, чем в архитектуре: троичная логика на CNTFET и мемристорах, а в последнее время — кремний-фотонные уравновешенно-троичные предложения, и все они строят ткань, чья *позиция* физически троична.

Новое здесь — граница, а не само утверждение. Мы не добавляем ничего в поддержку тезиса «троичное бьёт двоичное» как общего высказывания. Мы мерили его трижды, и каждый раз результат был против нас — см. теорему об отсутствии бесплатного выигрыша и разделы об аппаратуре и о родственных блочных работах. Наш вклад — условие, при котором шестидесятивосьмилетний аргумент применим, сформулированное точно:

Теорема (Аргумент об основании держится на позициях, а не на битах). [доказано] Запишем экономичность как $\text{cost} = r \times \log_r V$. Если физической единицей является позиция, первый множитель общий для всех оснований и сравнивается только второй, что даёт троичности безусловное преимущество в $E(1 - \log_3 2) = 0,3691E$ позиций. Если же саму позицию приходится кодировать битами, формат ограничен неравенством $3^{E_t} 2^M \leq 2^{N-1}$, и, поскольку 3^{E_t} никогда не делит степень двойки, остаток теряется.

Остаток — это арифметика, а не инженерия, и он наибольший ровно там, где живут современные форматы:

Таблица. Коды, потерянные на упаковке, когда троичный порядок хранится в двоичном слове. Потеря — свойство целых чисел, а не реализации.

ширина	лучший упакованный член	кодов использовано	потеряно
4b	$E_t = 1, M = 1$	6 / 8	25,0%
6b	$E_t = 3, M = 0$	27 / 32	15,6%
8b	$E_t = 3, M = 2$	108 / 128	15,6%
16b	$E_t = 5, M = 7$	31 104 / 32 768	5,1%

Предшествующие работы предвидели механизм: анализ Пархами двоично-кодированной уравновешенной троичности разбирает ровно эту плату за кодирование. Наш вклад — измерить, во что она обходится

в живой сети и в размещённом и разведённом кремнии, а не в количестве операций, и сформулировать её как условие, которое проектировщик может проверить.

Читать как предписание, а не как приговор. Результат говорит, какая ткань должна существовать, чтобы аргумент об основании окупился: такая, где троичная позиция держится физически — как её держали парные сердечники Сетуни и как предлагают держать снова CNTFET-, мемристорные и фотонные троичные приборы. На такой ткани преимущество $0,3691E$ собирается целиком и никакой потери на упаковке не возникает. На двоичной ткани, которую сегодня действительно можно купить, — нет. Это более узкое утверждение, чем обычно делает область, и, в отличие от более широкого, оно переживает наши собственные измерения.

Что достраивает золотое сечение

Сквозь эту статью идут две линии рассуждения, и развивались они независимо и по несвязанным причинам. Их стоит поставить рядом в конце, потому что встречаются они в одном-единственном числе.

Первая линия арифметическая и старая. Экономичность основания делает тройку лучшим целым основанием; Фаулер построил на нём машину около 1840 года, Брусенцов и Соболев построили на нём работающий компьютер в 1958-м, а линия продолжилась в CNTFET-, мемристорных и фотонных приборах. Раздел об истории показывает, почему она заглохла, а не распространилась: в тот момент, когда троичная позиция кладётся в двоичное слово, 3^E перестаёт делить степень двойки и остаток теряется — четверть алфавита при четырёх битах. Препятствие арифметическое, поэтому никакая реализация его не снимает, и три наших собственных измерения подтверждают его против нас.

Вторая линия геометрическая и в этом контексте новая. Если веса сети должны быть символами, а не числами, тракт обязан умножать без множителя — а это возможно ровно тогда, когда произведение двух символов падает обратно в сумму, которую тракт и так образует. Записанное, это требование есть $r^2=r+1$. У него одно положительное решение, так что алфавит не спроектирован, а определён: $\{-\varphi, 0, +\varphi\}$ и

ничего больше (см. теорему о золотом алфавите). Следствие в том, что композиция замкнута — усиление k слоёв равно $F_k\varphi + F_{k-1}$, пара целых чисел (см. следствие о глубине), — так что и глубина свободна от умножителей, чего алфавит $\{-1, 0, +1\}$ никогда не достигал, потому что его обучаемый послойный масштаб — вещественное число, и на него надо умножать.

Эти две линии отвечают друг другу. Троичной программе нужна была не лучшая ткань, а основание, при котором её символы komponуются; а у золотого сечения, и уже с тех пор, как Евклид разделил отрезок в крайнем и среднем отношении, было ровно свойство $r^2 = r + 1$. Помещение его в алфавит весов превращает шестидесятивосьмилетний аргумент из утверждения о плотности хранения в утверждение об арифметической замкнутости — и измерения из раздела о паре следуют: ноль DSP через радио, через каждый блок трансформера и через обратное распространение на самом кристалле.

Мы ставим это в конец, а не в начало, потому что это прочтение результатов, а не один из них. Каждая составляющая сформулирована и измерена выше: единственность — две строки алгебры, замкнутость по Фибоначчи — арифметика, потеря на упаковке — таблица целых чисел, а стек с нулём DSP размещён и разведён. Во что они складываются вместе — оставлено читателю.

Шестнадцать способов, которыми измерение может быть верным, а сравнение вокруг него — нет

Отзывы утверждений в этой статье для неё не побочны. За время работы было отозвано шестнадцать утверждений, два из них затем отозваны в свою очередь, и почти в каждом случае измерение было верным, а сравнение вокруг него — нет. Породившие их закономерности изложены здесь, потому что они обобщаются за пределы этого предмета и потому что читатель вправе знать форму ошибок, которые статья уже сделала.

О том, что именно сравнивается. Отношение осмысленно только тогда, когда обе стороны построены так, как их построил бы их собственный защитник; семь раз здесь отношение меняло знак или

порядок величины, когда другая сторона перестраивалась (см. раздел о применителе масштаба). У систем, чей диапазон масштабируется с шириной по-разному — одна точно, другая приближённо, — согласованного по объёму хранения отношения не существует вовсе: величина, зависящая от неозвученного выбора, не имеет предела и сходиться не может, и именно поэтому шесть последовательных поправок к инструменту не сумели устоять на одном числе. Там, где показатель качества одной системы постоянен по параметру, а другой по нему меняется, корректно поставленное сравнение — это точка пересечения, а не приговор (см. теорему о задержке сужающихся форматов и раздел о чистой таблице), и такое пересечение есть утверждение о нагрузке, а не о форматах.

Об инструменте. При варьировании семени размещения количество LUT инвариантно, а достигнутая частота гуляет на 11,4%, так что любая метрика, их объединяющая, наследует разброс по таймингу целиком, и ранжирование разрешает только те строки, что разнесены больше, чем на эту величину. Отношение по коррелированным множителям пересказывает доминирующий: на объединённом стенде ранжирование по пропускной способности на площадь воспроизвело ранжирование по $1/\text{площадь}$ с корреляцией Спирмена 0,961. А результат, который при каждой очередной поправке двигается в одну и ту же сторону, всё ещё двигается.

О стенде. Там всплыли три дефекта, ни один из которых не был виден ни в одном числе, и каждый менял заголовочную цифру на 10–50 процентов. Компонент, общий для всех кандидатов, — это не константа, а конфаундер, потому что он оставляет разницу по площади нетронутый, а разницу по задержке — нет. Стенд, наблюдающий часть выхода схемы, подрезает каждого кандидата пропорционально *его собственной* ширине выхода: наблюдение четырёх бит 32-битного тракта оставило 27 LUT из 179, оставив соседнюю 6-битную схему почти целой. А вычитание стенда требует, чтобы стенд присутствовал целиком в каждой сборке, поскольку схема, потребляющая меньше его источников, обрезает остальные.

Об источниках. Дефект, найденный в аннотации, — это гипотеза: аннотации сжимают, а сжатие неотличимо от пропуска. Агрегированный результат поиска может слить утверждения из

нескольких статей в формулировку, которой нет ни в одной. Обе внешние претензии, которые мы выдвинули по ходу этой работы, растворились при соприкосновении с самими статьями.

Чего это стоит. Приложенная к нашим собственным утверждениям, процедура дала шестнадцать отзывов. Приложенная к двум внешним статьям, прочитанным целиком, она не дала ни одного, и оба раза вместо этого поймала наше собственное их прочтение. Поэтому мы описываем её как инструмент проверки собственной работы, а не как метод аудита чужой. Три проверки описаны как гейты в непрерывной интеграции, но ни одна из них не реализована: соответствующих файлов нет ни в рабочем дереве, ни в истории репозитория, и ни один рабочий процесс непрерывной интеграции на них не ссылается. Полная формулировка — в сопроводительной заметке; реализация ещё предстоит.

Цифры, чьих порождающих данных нет в репозитории

За время работы было отозвано шестнадцать утверждений, а число отозванного утверждения само себя из статьи не убирает. **Описанная в этом разделе проверка прослеживаемости не реализована:** файла `tools/check_paper_numbers.py` нет ни в рабочем дереве, ни в истории репозитория — `git log --all --diff-filter=A -- tools/check_paper_numbers.py` не даёт ни одного коммита, — и ни один рабочий процесс непрерывной интеграции его не вызывает. Всё, что описано ниже, — это её замысел, а не её вывод. Предполагается, что она извлекает отсюда каждый отличительный числовой литерал и спрашивает, встречается ли он хоть в одном файле под `research/`, `fpga/` или `conformance/`, и что два класса она исключает, а не терпит: константы, заявленные рядом с формулой, которая их порождает (таких 60, и в файлах данных их правильно нет), и округление, где литерал в данных, продолжающий напечатанные цифры, засчитывается как источник.

ОГОВОРКА W845 [проверено]. Инструмент, которым получена эта цифра, `tools/check_paper_numbers.py`, **отсутствует и в рабочем дереве, и во всей истории репозитория** — `git log --all --diff-filter=A` возвращает ноль коммитов,

когда-либо его добавлявших. То же верно для четырёх других гейтов, названных ниже по тексту как действующие: `check_literal_widths.py`, `check_variant_declared.py`, `check_exponent_window.py`, `conformance/variant_map.json`, а также для `gen_figures.py`. Ни один из них не написан. **Поэтому соотношение 444/451 невоспроизводимо**: оно описывает прогон, который нечем повторить. Оно оставлено на месте как заявленное, а не удалено, потому что удалить число легче, чем признать, чем оно не является.

Из 451 отличительного литерала 444 прослеживаются до файла. **Семь — нет**, и они перечислены, а не удалены, потому что цифра, которую нельзя проследить, тем самым ещё не показана неверной:

Таблица. Цифры, для которых в репозитории нет порождающего файла. Каждая сохранена, помечена и должна быть перемерена, прежде чем на неё опираться.

цифра	где встречается
106,68,215,90,63,04	расширенные форматы: double-double, quad-double, x87
0,113	наклон сужения takum16 в таблице ступенчатых форм
0,247	наклон posit32 в тексте (измеренный здесь же в другом месте как 0,260)
8,14	одна строка таблицы ширин полей
8,99	одна ячейка таблицы эффективной мантиссы по полосам

Проверка не может показать, что число верно; она находит числа без источника, что строго слабее и всё же достаточно. Цифра, которую никто не может проследить до файла, — это цифра, которую никто не может перепроверить, а после шестнадцати отзывов именно там и выживает устаревшее значение. Две из семи имеют прямо измеренную замену в другом месте этой статьи и с ней расходятся — а это ровно тот случай, ради которого проверка и заведена.

Теорема (Документ — тоже артефакт, и его пары нуждаются в защите). [измерено] Проверки согласованности пишутся между артефактами, которые какой-нибудь инструмент и так читает вместе, — компилятор, симулятор, скрипт сборки.

Документы же читают только люди, поэтому любая пара, в которую входит документ, по построению остаётся непроверенной, и именно там сохраняются отозванные числа.

Перечисление пар — это то, что делает поиск таких дефектов систематическим, а не случайным. Три более ранних гейта в этой работе были найдены каждый через спотыкание о тот дефект, который они теперь ловят, и все три защищают код от кода. Перечисление же всех пар, которые обязаны согласовываться, вскрыло пять незащищённых за один проход, и во всех пяти с одной стороны стоял документ.

Ступенчатая форма предписывает сравнение

Раздел о каталоге классифицирует сужающиеся форматы по ступенчатой форме, чтобы их *описать*. Измерение точек пересечения из раздела о чистой таблице по двум окнам подгонки показывает, что классификация делает больше этого.

Теорема (Ступенчатая форма решает, нужно ли окно).
[доказано] Пересечение между фиксированным полем и тейпером (форматом с сужающейся точностью) зависит от окна подгонки тогда и только тогда, когда ступенчатая форма тейпера не арифметическая. У арифметического тейпера M_{eff} линейна по $|e|$, поэтому его наклон определён без всякого окна. У геометрического — линейна по $\log |e|$, поэтому никакая прямая её не описывает, а видимый наклон есть свойство одного только окна.

Измеренное по окнам 1–9 и 1–20 разделение точное: posit16, posit32 и posit64 двигаются в $1,08\times$, $1,00\times$ и $1,09\times$, тогда как каждый takum и tekum двигается в $1,39\times$ – $1,72\times$.

Следствие (Замкнутая форма для геометрического тейпера).

[доказано] Для геометрического тейпера $M(e) = p - \log_2 e$, так что пересечение с фиксированным полем постоянной m решается точно:

$$x = 2^{p-m},$$

и никакого окна не требуется.

Таблица. Пересечения, вычисленные той моделью, которой требует форма каждого тейпера. Подгонка прямой завышала дальнобойность каждого геометрического тейпера, потому что прямая недооценивает, как быстро логарифм падает у начала координат.

тейпер	форма	пересечение, бинад
posit8	арифметическая	1,5–2,5
posit16	арифметическая	8,5
posit32	арифметическая	9,2
posit64	арифметическая	54,0
takum16	геометрическая	1,8
takum32	геометрическая	2,6
takum64	геометрическая	62,7
tekum16	геометрическая	2,1
tekum32	геометрическая	2,4
takum8, tekum8	геометрическая	TNF везде

Шесть цифр, ранее отчитанных как диапазоны, были посчитаны неверной моделью и здесь исправлены: takum32 переходит с 5,9–10,2 на точные 2,6, tekum16 — с 4,2–6,5 на 2,1. Оставшаяся зависимость от окна заперта в posit8, чей пик стоит в пределах 0,45 бита от пика TNF8, так что любая оценка наклона делит очень маленькое число.

Стоит запомнить не арифметику. **Таксономия, введённая, чтобы маркировать форматы, оказывается предписывающей, как их обязаны сравнивать** — арифметические тейперы по линейному пересечению, геометрические — по 2^{p-m} . Классификация, которая только маркирует, так бы не сделала, а мы почти всю работу пользовались ею только для маркировки.

Теорема (Вобл не допускает пересечения). [источник в тексте] Если M_{eff} формата периодична по $|e|$ с амплитудой A и периодом P , а m конкурента с постоянной формой лежит строго внутри полосы, то не существует такого x , при котором один лидировал бы для всех $|e| < x$, а другой — для всех $|e| > x$. Корректно поставленная статистика — это скважность (доля каждого периода, на которой лидирует каждый) вместе с P .

Измерено на IBM hexadecimal 32, чья M_{eff} по бинадам читается как 21,03, 21,75, 22,94, 19,95, а затем повторяется: амплитуда 3,07 бита, период 4 бинады, что есть $\log_2 16$, как и требует основание. Против упакованного TNF32 при $M_{eff}=21,32$, внутри полосы [19,94,23,11], победитель чередуется с этим периодом со скважностью 50/50, и ни одна бинада их не разделяет.

Следствие (Таксономия предписывает). [источник в тексте] Каждая из четырёх ступенчатых форм определяет форму корректно поставленного сравнения с форматом постоянной точности: разность — для постоянной, линейное пересечение — для арифметической, экспоненциальное пересечение 2^{p-m} — для геометрической и скважность — для вобла.

Отображение «форма $\rightarrow$ вид ответа» сформулировано здесь, и мы ничего не утверждаем о том, как отчитывается литература. В более ранней версии этого абзаца говорилось, что опубликованные сравнения дают приговоры или отношения; при сверке со статьёй Хунхольда о *takum* это оказалось неверно — он отчитывается по режимам с числовыми порогами, отмечая, что эффективность *posit* «заметно снижается при отклонении от единицы» и что точность *takum* «превосходит точность *posit* вблизи единицы, почти совпадая с ней при чуть больших порядках». Фраза отозвана.

Чего мы в той литературе не нашли — так это замкнутой формы 2^{p-m} для геометрического тейпера против конкурента постоянной формы или какого-либо разбора случая вобла. Вклад здесь — само

отображение: вид ответа фиксируется ступенчатой формой конкурента, и классификация, введённая только ради маркировки, оказывается решающей.

Ограничения

Ткани, на которой этот формат оптимален, нельзя купить. Теорема об отсутствии бесплатного выигрыша запирает выгоду троичного кодирования на троичной ткани, а никакого такого техпроцесса коммерчески не существует. Поэтому каждая аппаратная цифра в этой статье измерена на двоичной FPGA, где троичный порядок ни выигрывает, ни проигрывает: перебор в разделе о том, зачем троичность, показывает, что на проверенных ширинах он в точности сравнивается с двоичным кодированием. Утверждения, выживающие на кремнии, который можно купить, — это утверждения о фиксированных полях: нет кодека режима, нет экспоненты, которую надо вычислять. Троичное утверждение архитектурное и повсюду так и помечено.

- 1. Сравнение идёт с `takum16`, а не с `tekum16`.** Оракул, помеченный как `tekum`, декодирует все 65,536 16-битных кодов идентично оракулу `takum`. Мы отчитываемся о сравнении как о сравнении с `takum` и отмечаем, что настоящее сравнение с `tekum` ещё предстоит сделать.
- 2. Эталонный оракул нёс дефект знака, теперь исправленный.** Круговой прогон 1,5 через параметризованный оракул возвращал -1,5 при ширинах мантиссы 21 и 25 бит: бит знака сидел на фиксированной позиции, тогда как маска порядка брала больше бит, чем вмещает поле, и они перекрывались. Теперь оба выводятся из параметров формата, и ошибка кругового прогона TNF32 падает с $5,4e-1$ — того, во что в среднем выливается систематически инвертированный знак, — до $5,3e-9$.

Мы записываем, как он выжил, потому что урок переживёт сам баг. Проверкой лестницы была коммутативность сложения и умножения, а инвертированный знак выживает по обе стороны $a+b=b+a$. Проверка была настоящей и слепой к этому классу. Утверждение о круговом прогоне его видит, стоит одну строку на

пробу и теперь выполняется на всех девяти рунгах. `tekum` (Хунхольд, arXiv:2512.10964) — потомок `takum` (Хунхольд, arXiv:2404.18603), приспособленный к уравновешенной троичности. Наш оракул реконструирует его из схемы полей `takum`; полная потритовая спецификация требует статьи о `tekum`. Отношения хороши ровно настолько, насколько хороша эта модель.

3. **Нет лобового сравнения в аппаратуре.** RTL кода `takum` открыт и написан на VHDL (см. arXiv:2408.10594). Мы не синтезировали его рядом с TNF, поэтому цифры стоимости — это собственные цифры TNF. Что видно в том исходнике и о чём мы сообщаем как о структуре, а не как об измерении: 16-битный кодек инстанцирует сгенерированный счётчик ведущих нулей на 725 строк и бочкообразный сдвигатель — тот самый декод режима, которого у формата с фиксированными полями нет.
4. **Диапазон ограничен** значением ± 40 в степенях двойки, примерно ± 12 десятичных порядков, тогда как поле режима не ограничено. Фиксированные поля покупают дешёвый тракт и равномерную мантиссу; диапазон — это цена, и нагрузкам, которым нужно больше, следует поднять бюджет тритов или взять сужающийся формат.
5. **Точность выше TNF32 ограничена измеряющим инструментом.** После починки дефекта оракула TNF32 меряется как $5,3e-9$, ровно по всей нагрузке, — это то, что должны давать 25 бит мантиссы. TNF64 несёт 52 бита мантиссы, а это ровно мантисса того IEEE double, в котором вычисляется наша метрика, так что возвращаемая цифра описывает метрику, а не формат. Измерение верхних рунгов требует точной рациональной арифметики повсюду — такой, какую сам оракул уже использует.
6. **Пять рунгов из девяти измерены в аппаратуре.** TNF4 — TNF64 помещаются в кристалл, TNF64 — при 7,479 LUT и 48,20 МГц. TNF128 при $M=119$ не сходится в разводке на этом кристалле и не заявляется; TNF256 и выше целиком превосходят одну ткань Artix-7. Таблица спецификации — это спецификация и покрытие оракула; таблица лестницы — это то, что было размещено и разведено. Мы держим их порознь намеренно.

7. **Одно семейство кристаллов.** Artix-7 на открытом флоу. Это не многоугольная характеристика; отображение на ASIC будет другим.
8. **Аргумент о троичной ткани архитектурный.** Троичного техпроцесса, на котором можно было бы синтезировать, не существует. Раздел об аппаратуре — это результат на двоичной ткани; троичное утверждение есть рассуждение о декоде режима и о нативном сложении порядков, и читать его следует именно так.

Воспроизводимость

Эталонный оракул, RTL, стенды на эквивалентность и команды синтеза открыты. Инструментарий — Yosys 0.65, nextpnr-xilinx 1743d0f, Icarus Verilog 13.0 и Python 3.14, всё с открытым исходным кодом, ничего не требует лицензии, и в этом весь смысл: результат, который нельзя воспроизвести, ничего не купив, не является публичным результатом.

Раскрытие

При подготовке программного обеспечения, инструментария измерений и этой рукописи использовался ИИ-ассистент программирования. Каждая приведённая здесь цифра получена запуском названных инструментов на аппаратуре, находящейся во владении автора; ни одно измерение в этой статье не является оценочным, а там, где цифра выведена, а не наблюдена, это помечено.

Список литературы

1. L. Hunhold. Takum arithmetic. arXiv:2404.18603.
2. L. Hunhold. Tekum: balanced-ternary tapered-precision arithmetic. arXiv:2512.10964.
3. L. Hunhold. Design and implementation of a takum arithmetic hardware codec in VHDL. arXiv:2408.10594. Исходники: <https://github.com/takum-arithmetic/Takum-Codec-RTL>
4. D. Vasilev. GoldenFloat: a ϕ -derived static-split floating-point family from GF4 to GF1024 with a Lucas-exact integer identity. arXiv:2606.05017v3, 2026.

5. D. Vasilev. An 83-format numeric catalog with bit-exact conformance vectors: a vendor-neutral reference for FP8, BF16, MXFP4, and microscaling formats. arXiv:2606.09686v2, 2026.
6. J. L. Gustafson, I. Yonemoto. Beating floating point at its own game: posit arithmetic. *Supercomputing Frontiers and Innovations*, 4(2), 2017. Стандарт Posit (2022) фиксирует $es=2$ на всех разрядностях.
7. C. Wolf. Yosys open synthesis suite. <https://yosyshq.net/yosys/>
8. D. Shah et al. nextpnr: a portable FPGA place-and-route tool. <https://github.com/YosysHQ/nextpnr>; цель Xilinx через Project X-Ray.
9. B. Hayes, «Third Base», *American Scientist*, т. 89, № 6, с. 490–494, 2001.
10. Radix economy, optimal radix choice. https://en.wikipedia.org/wiki/Radix_economy
11. Ternary27: A Balanced Ternary Floating Point Format, Version 3.1. <https://hackaday.io/project/183153>
12. D. Goldberg, «What Every Computer Scientist Should Know About Floating-Point Arithmetic», *ACM Computing Surveys*, т. 23, № 1, с. 5–48, 1991.
13. IBM hexadecimal floating-point. https://en.wikipedia.org/wiki/IBM_hexadecimal_floating-point
14. P. Elias, «Universal codeword sets and representations of the integers», *IEEE Trans. Inform. Theory*, т. 21, № 2, с. 194–203, 1975.
15. Closing the Gap Between Float and Posit Hardware Efficiency, arXiv:2603.01615.
16. Performance-Efficiency Trade-off of Low-Precision Numerical Formats in Deep Neural Networks, arXiv:1903.10584.
17. D. W. Boyd, «The maximal modulus of an algebraic integer», *Math. Comp.*, т. 45, № 171, с. 243–249, 1985.
18. Q. Wu, «The smallest Perron numbers», *Math. Comp.*, т. 79, № 272, с. 2387–2394, 2010.
19. C. Frougny, E. Pelantová, M. Svobodová, «Minimal digit sets for parallel addition in non-standard numeration systems», *J. Integer Seq.*, т. 16, ст. 13.2.17, 2013.
20. J. Légerský, «Construction of algorithms for parallel addition in expansions in an algebraic base», *Acta Polytechnica*, т. 58, № 5, с. 285–291, 2018.

21. C. Frougny, E. Pelantová, M. Svobodová, «Parallel addition in non-standard numeration systems», *Theoretical Computer Science*, т. 412, № 41, с. 5714–5727, 2011.
22. C. L. Siegel, «Algebraic integers whose conjugates lie in the unit circle», *Duke Math. J.*, т. 11, № 3, с. 597–602, 1944.
23. D. Berend, C. Frougny, «Computability by finite automata and Pisot bases», *Math. Systems Theory*, т. 27, с. 275–282, 1994.
24. S. Akiyama, «A family of non-sofic beta expansions», *Ergodic Theory Dynam. Systems*, т. 36, № 2, с. 343–354, 2016.
25. V. Egiazarian et al., «Bridging the Gap Between Promise and Performance for Microscaling FP4 Quantization», arXiv:2509.23202, 2025.
26. NVIDIA, «Introducing NVFP4 for Efficient and Accurate Low-Precision Inference», 2025; и «Pretraining Large Language Models with NVFP4», arXiv:2509.25149, 2025.
27. J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, S. Han, «AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration», в *Proc. MLSys*, 2024. arXiv:2306.00978.
28. X. Yang, Y. Wang, Y. Qin, J. Wang, S. Wei, Y. Hu, S. Yin, «FQP: A Fibonacci Quantization Processor with Multiplication-Free Computing and Topological-Order Routing», в *Proc. 61st ACM/IEEE Design Automation Conference (DAC)*, 2024. <https://doi.org/10.1145/3649329.3656502>
29. R. Fiandaca, M. D. Gomony, «Fibbinary-Based Compression and Quantization for Efficient Neural Radio Receivers», arXiv:2511.01921, 2025.
30. S. Ma et al., «The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits», arXiv:2402.17764, 2024.
31. ELiTeFormer: вычислительный элемент, устраняющий все умножения в троичных линейных проекциях битовым маскированием и полностью обходящийся без блоков DSP. arXiv:2607.03652, июль 2026. **Прецедент для всякого утверждения «ноль DSP» в этой статье.**
32. «Hardware Generation and Exploration of Lookup Table-Based Accelerators for 1.58-bit LLM Inference», arXiv:2604.25183, апрель 2026. Аналитическая модель стоимости и открытый генератор для пространства проектирования троичных LUT-ускорителей,

сверенные с синтезом TSMC 16 нм. **Прецедент для формализации пространства проектирования, которую делает golden_sieve.t27.**

Добавлено после первой сборки перевода

Ниже — разделы, вошедшие в статью после того, как перевод был собран. Все числа сохранены точно.

Что уже было известно, и что осталось

Конструкция, которую эта работа измеряет — двоично-радикальное число с плавающей точкой, чьё поле порядка несёт алфавит троичной мощности, плотно упакованный в $\lceil E_t \log_2 3 \rceil$ бит, — стоит достаточно близко к нескольким существующим направлениям, чтобы границы стоило провести явно, а не оставить читателю искать их самому.

Троичное поле порядка не ново. TERNAC был 48-тритным форматом с плавающей точкой, с 42-тритной мантиссой и **шеститритным порядком**, эмулированным на двоичном железе в SUNY Buffalo в 1972–73 годах. От формата здесь он отличается двумя существенными вещами — его семантика значений радика 3, а его триты представлены парой двоичных чисел, а не упакованы, — но *идея поля порядка, считаемого в тритах, старше этой работы на полвека*, и никакого притязания на первенство над ней нигде выше не делается и не подразумевается.

Упаковка при $k=5$ опубликована. Compilade опубликовал упаковку пяти тритов в восемь бит в июне 2024 года и привёл её как «99,06 % эффективности», что есть в точности $1 - w(5)$ в обозначениях теоремы о неукладываемости. В сопутствующем обсуждении отмечено также, что 17 тритов в 27 битах и 111 в 176 бьют 1,6 бита на трит — то есть *неравномерность $w(k)$ наблюдается публично*, пусть и не записана функцией. Раздел о неукладываемости добавляет замкнутую форму, доказательство положительности потери при каждом k и следствие для выбора рунга; само значение при $k=5$ принадлежит Compilade.

Упакованный декодер построен. TerEffic реализует ровно декодер упаковки 3^5 в восемь бит в LUT ПЛИС, для троичных весов, а не для поля порядка. Это один уровень абстракции от декодеров, измеренных в разделе о 54 семенах, и всякий читатель того раздела эту работу знает.

Двоичное кодирование сбалансированного троичного — учебный материал. Пархами и Маккеон разбирают двоично-кодированное сбалансированное троичное напрямую, по два бита на трит — это и есть база, которую плотная упаковка улучшает, и источник той платы за кодирование, которую эта статья измеряет количественно. Изложение Кнута и машины «Сетунь» устанавливают сбалансированное троичное как радикакс; *эта работа радикакс не меняет* и потому не наследует радикасно-экономического аргумента, который та литература несёт.

Что осталось. Отложив перечисленное, остаётся немного, и сформулировано оно узко: сочетание троичной **мощности** с двоичным **радикасом** и двоичным **хранением**, при котором избыток ёмкости $2^{\lceil k \cdot \log_2 3 \rceil} - 3^k$ рассматривается как расчётная величина, а не как накладные расходы — резервируется, охраняется и оценивается. Вокруг этого: замкнутая форма $w(k)$ и её следствия для выбора рунга, цена резервирования, измеренная, а не выведенная рассуждением, и разведённый по кристаллу кремний с исчерпывающей сверкой для каждого заявленного рунга. Каждый из этих вкладов — про **измерение и границу**, а не про первенство.

Следствие: потеря есть неопределённое кодовое пространство

Следствие (потеря есть неопределённое кодовое пространство). [доказано] Поскольку поля знака и мантиссы заполнены полностью — каждый из их $2^{(1+M)}$ образцов обозначает своё значение, — доля *хранимого слова*, оставленная спецификацией неопределённой, равна доле поля смещения, оставленной неопределённой:

$$\frac{2^{\text{ширина}} - 2 \cdot 3^{E_t} \cdot 2^M}{2^{\text{ширина}}} = 1 - \frac{3^{E_t}}{2^{\lceil E_t \log_2 3 \rceil}} = w(E_t).$$

Значит w — не абстрактная потеря на упаковке: она **есть** доля собственных кодов формата, не именующих ничего. При $E_t=4$ это 24 064 кода из 65 536; при $E_t=5$ — 3 328.

Оттого предложение о цене резервирования кусается ровно там, где w велико: коды, которые рунг обязан охранять, — это и есть коды, которые считает w , и третьего места для них нет.

Потеря решает, обходит ли защищённый рунг IEEE half

Следствие выше говорит, что $w(E_t)$ **есть** доля кодов рунга, не именующих ничего, а предложение о цене резервирования говорит, что эти коды обязаны охраняться. Вместе они предсказывают, что преимущество рунга над `binary16` должно переживать защиту тем лучше, чем меньше он теряет. Оба шестнадцатибитных рунга построены в защищённом и незащищённом виде и прогнаны на 54 семенах на общем стенде:

рунг 16 бит	$w(E_t)$	без защиты	с защитой	с защитой против <code>binary16</code>
TNF16a ($E_t=4$)	36,7 %	0,1214	0,1079	+0,10 %, $p = 0,90$
TNF16c ($E_t=5$)	5,1 %	—	0,1152	+6,92 % , $p_{\text{Holm}} = 6,9 \cdot 10^{-21}$

Защита обходится рунгу А в 11,10 % ($p = 9 \cdot 10^{-59}$) и сажает его на `binary16` в точности: 0,1079 против 0,1078, неотличимо. Рунг С, теряющий всемеро меньше, сохраняет +6,92 % после той же по природе платы.

Рунг, выигрывающий при равном хранении, есть рунг малой потери — и это тот самый, который следствие о хороших рунгах называет из одной лишь арифметики. Теория выбрала победителя до того, как кремний был построен, и это сильнейшая форма проверки, какую центральная величина этой статьи проходила.

Исправление: проценты защиты не были её ценой

Защищённая сборка рунга А измеряется на 15,1 % ниже своей незащищённой, а защищённый рунг С — лишь на 1,8 % ниже, **но эти проценты не есть цена сравнителя.** Синтезированная отдельно,

защита составляет 1 LUT при $E_t=5$, 2 при $E_t=4$ и 3 при $E_t=8$, против модулей примерно в пятьсот, — то есть на само сравнение приходится не более полупроцента площади.

Меняется другое: образ выхода декодера. Резервирование 47 смещений схлопывает 24 064 кода в один NaN, и всё ниже по течению подгоняется под наблюдаемый образ (см. предложение о срезании по образу выхода). **Разница между защищённой и незащищённой сборкой есть свойство всего датапути, а не цена, приписываемая защите**, — и эта статья излагала её вторым способом, пока состязательная проверка вывода не опровергла приписывание. На семнадцати битах та же разница целиком легла внутрь разброса семян.

ЭРРАТУМ 2026-08-13 — заглавное утверждение о пропускной способности отзывается

Внесено по итогам состязательной проверки (W652). Каждое число ниже пересчитано из таблицы самой статьи (строки 1216–1240); команда воспроизведения приведена.

Что не воспроизводится

в аннотации	в таблице	расхождение
«на 10.2% впереди следующего»	$0.1584/0.1429 = 1.1085$, то есть +10.85%	значения, дающего ровно 10.2%, в таблице нет (0.1437)
«в 6.1 × впереди posit32 »	$0.1584/0.0302 = 5.245$ ×	значения, дающего 6.1 ×, в таблице нет (0.0260)
«среди двадцати форматов»	24 строки данных	—
«восемь из двадцати наши»	12 строк выделены как наши	—

```
python3 -c "print(0.1584/0.1429, 0.1584/0.0302)" # 1.1085 5.2450
```

Что решает исход, и это хуже, чем арифметика

int8, измеренный на том же стенде, обыгрывает GFternary — и **исключён из ранжирования**. Строка 1369 этой статьи: 454 LUT, 78.83 МГц, **0.1736 MHz/LUT**. Против 0.1584 у GFternary это +9.6% **в пользу int8**.

Обоснование исключения — что **int8** нуждается во внешнем обучаемом масштабе — относится к *роли формата в системе*, а не к величине, которую меряет столбец. Столбец меряет МГц на LUT. По этой величине мы не первые.

И метрика спутана с шириной входа — по нашему же предложению

Предложение об усечении (строка ~1460) утверждает: декодер с n входными битами имеет не более 2^n различных выходов, логика ниже по течению специализируется под этот образ, и **«сравнение при разных n смешивает конструкцию формата с его шириной; только сравнение внутри одного n выделяет конструкцию»**.

GFternary — это $n=2$. Все конкуренты в ранжировании — от $n=8$ до $n=32$. Статья сама сообщает, что декодирование GFternary стоит 66 LUT там, где голый провод стоит 112, и признаёт: «формат не может быть дешевле провода; здесь меряется ширина его образа».

Ранжирование измеряет ширину входа, а не конструкцию формата, и статья это доказывает раньше, чем предъявляет ранжирование.

Что отзывается и что остаётся

Отзывается: утверждение о первом месте по пропускной способности на единицу площади, во всех трёх его формах — процент над следующим, кратность над **posit32**, и словосочетание «лидирует среди двадцати форматов».

Остаётся без изменений — ничто ниже не опирается на отозванное:

- теорема о **единственности золотого алфавита** ($r^2 = r + 1 \Rightarrow r = \varphi$);
- теорема о **точности пути без умножений** ($\mathbb{Z}[\varphi]$ — кольцо, линейный тракт точен), сверенная с кремнием на 16,000 векторах и 30 глубинах с нулём расхождений, половина проверена машиной в Coq;
- следствие о **замкнутости** и **теорема о пороге нормализации**;
- **закон точности** (теорема 1) и вся диагностика сужения, разрешающая каталог в четыре ступенчатые формы;
- **иерархия безумножительных масштабов** и семейство $r^d = r + 1$: тонкость шага стоит регистров, а не сумматоров;
- **28 LUT на вес и ноль DSP при любом фан-ине** на XC7A200T — это измерение датапути, а не ранжирование форматов.

Как это следовало бы сформулировать взамен. Не «GFTernary лидирует по пропускной способности на площадь», а: *среди форматов одинаковой ширины входа GFTernary — единственный, чья решётка замкнута относительно применения веса, и поэтому единственный, у которого линейный тракт точен без ступени нормализации.* Это утверждение о конструкции, оно проверяемо внутри одного n , и оно не опровергается `int8`.

Это одиннадцатое снятое утверждение этой работы, и первое, снятое не нами.

ИЗМЕРЕНИЯ НА FPGA, 2026-08-13 (W653–W655)

Внесено агентом по прямому запросу. Всё ниже измерено на этой машине; каждая цифра сопровождается командой, которой получена. Раздел не заменяет и не переписывает кремниевые таблицы статьи — он добавляет независимый набор чисел, полученный на том же семействе кристаллов и на полностью открытом флоу.

Что измерялось и на чём

Железо (измерено, не предположено): три платы **QMTech XC7A200T-FGG676**, `idcode 0x3636093`, `irlength 6`, каждая на своём кабеле Digilent FTDI `0403:6014`. **Все три кабеля сообщают один и тот же серийный номер 210512180081**, поэтому `--ftdi-serial` совпадает со всеми тремя и молча берёт первый попавшийся; различает их только `--busdev-num` либо `locationID` из `ioreg`. Флеш — Micron N25Q128 3V, JEDEC `0x20ba18`.

Тулчейн: Yosys 0.63, nextpnr-xilinx `e4a261c` (собран локально, `-DUSE_OPENMP=OFF`; Apple clang отвергает `-fopenmp`), Icarus Verilog 13.0, Python 3.14.6. Статья указывает Yosys 0.65 и nextpnr-xilinx `1743d0f`; расхождение версий названо здесь, а не сглажено.

Битстрим собран локально и загружен: `9 730 896` байт, SHA-256 `4272dd6f...`, не совпадающий ни с одним из 16 закоммиченных в дереве. На всех трёх платах получен **переход Done 0x0 → 0x1**, полученный тем, что плата сначала намеренно приводилась в состояние `Done = 0` битстримом от другого чипа. *Сам по себе done 1 ничего не доказывает: платы грузятся из Master-SPI флеша и поднимают DONE без нашего участия, а заведомо испорченный битстрим — 4 КБ полезной нагрузки, инвертированные посередине, — даёт неотличимые от успеха сигналы. Тракт конфигурации проверяет конверт, а не содержимое.*

Синтез: четыре спецификации, ни одного DSP

Все четыре написаны на `.t27`, выведены в Verilog компилятором репозитория и синтезированы `synth_xilinx -family xc7 -flatten` с определённым `SIMULATION` (то есть тестовые блоки исключены).

артефакт	LUT	прочее	DSP
3B2T троичный линейный кодер (specs/fpga/ ternary_link.t27)		7 LUT6 11 IBUF, 9 OBUF	0
TNF17 , отрицание (specs/numeric/ tnf17.t27)		0 35 IBUF, 33 OBUF	0
Применение веса в Z[φ] (specs/igla/ race/ phi_weights.t27)		3 LUT6 11 IBUF, 33 OBUF	0
Составной троичный узел (specs/igla/ race/ ternary_node.t27)	32 LUT2, 32 LUT5, 2 LUT6	24 CARRY4 , 115 IBUF, 33 OBUF	0

Три числа стоит выделить.

Отрицание в TNF стоит ноль логики. Знак — единственное, что TNF17e добавляет к величине GF-T16, и в кремнии это переворот одного бита на позиции 16, то есть **чистая проводка**. Статья утверждает это как свойство раскладки; здесь оно размещено и разведено.

Мост между каналом и алфавитом весов стоит ноль LUT. Проволочная кодировка линейного кода — `2'b01 / 2'b10 / 2'b00` — **побитово совпадает** с алфавитом `GFT_POS / GFT_NEG / GFT_ZERO`. Функция преобразования синтезируется в ничто, и модуль занимает те же 7 LUT6 до и после её добавления. **Преобразование, которое ничего не стоит, — это преобразование, которого нет:** канал переносит веса, а не их сериализацию.

В составном узле 24 CARRY4 — это сумматоры, и умножителя в нём нет нигде. Полный шаг: приём двумя компараторами → символ → вес → применение → накопление. Опровержимое содержание цифры «ноль DSP» состоит в том, что узел на единичном алфавите `{-1,0,+1}` её достичь не может: усиление слоя равно 1 и не несёт информации, поэтому нужен обучаемый вещественный α_l , а его применение есть умножение.

Чего этот раздел НЕ измеряет. Проверен один MAC-шаг. Фан-ин, глубина и ширина аккумулятора, требуемая чтобы пара `(a,b)` осталась точной на многих шагах, здесь не измерены — статья сообщает логарифмический рост компонент и восемь бит при фан-ине 512, и это

утверждение данными раздела не подтверждено. Сравнение с узлом на единичном алфавите **не проводилось**, поэтому «ноль DSP» — измерение одной стороны.

И ноль относится к троичному ядру, а не к целому конвейеру. Это вторая половина, без которой цифра читается шире, чем она есть. Тракт, несущий блочный масштаб в плавающей точке — скажем, один множитель FP16 на 128 весов, — стоит порядка $13,4 \cdot 10^6$ умножений на токен, то есть примерно одного DSP48E1 из 740 при единицах токенов в секунду. Ноль верен для троичного ядра и неверен для конвейера с блочными масштабами, и обе половины принадлежат этой статье.

И первенства на нём нет. ELiTeFormer (arXiv:2607.03652, июль 2026) публикует вычислительный элемент, «устраняющий все умножения в троичных линейных проекциях ... полностью обходясь без выделенных блоков DSP», — то есть ровно ту величину, которую меряет столбец DSP выше, и раньше. Наше измерение независимо и остаётся измерением; утверждение о новизне снимается. Опровержимое содержание, названное двумя абзацами выше, — про алфавит, а не про приоритет, и оно этим не затрагивается.

Проверка в двух бэкендах

спецификация	Zig	Icarus + vvp
numeric/tnf17.t27	34/34	34 PASSED, 0 ошибок компиляции
fpga/ternary_link.t27	46/46	46 PASSED, 0 ошибок
igla/race/phi_weights.t27	29/29	29 PASSED, 0 ошибок
igla/race/ternary_node.t27	26/26	26 PASSED, 0 ошибок

Эти числа стали осмысленными только в ходе той же кампании. До неё каждый сгенерированный Verilog-тест печатал PASSED независимо от исхода: вердикт печатался безусловно после ветки отказа, привязки given эмитировались без объявлений, а временная переменная вызова объявлялась и никогда не присваивалась — так что всякая проверка сравнивалась с x, а if (!(x)) в Verilog ложно. Согласие двух бэкендов есть свидетельство только тогда, когда **оба способны разойтись**; до починки половина на Verilog была безусловной и не несла информации.

Арифметическое замечание к разделу о формате

Раздел «Формат» пишет $TNF16 = [s \mid E=4 \text{ трита} \mid M=11 \text{ бит}]$, тогда как таблица точности и лестница той же статьи используют $M = 9$ для рунга 16, и $2^{-(M+1)} = 9.77 \times 10^{-4} = 2^{-10}$ это подтверждает. Одновременно верны они быть не могут: при $M = 11$ значащая часть $1 + M/2^9$ пробегает $[1, 5)$, что не нормализовано. Правило ширины $1+E+M=N$ считает **позиции**, а трит — не бит: на двоичной фабрике четыре трита стоят семь бит, и $1+7+9 = 17$. Поэтому реализован однозначный семнадцатитритный рунг, который кремниевая таблица статьи и называет **TNF17e**; что именно выменивает TNF16с, остаётся открытым.

Тождество избытка-1, замеченное при реализации

Смещение 40 — не соглашение. Оно равно $1 + 3 + 9 + 27 = (3^4 - 1)/2$, то есть **репьюниту по основанию 3**, и именно поэтому вычитание смещения вычитает ровно единицу из каждой троичной цифры:

$$\text{trit}_i(e) = \text{digit}_i(\text{offset}) - 1, \text{offset} = e + 40 \in [0, 80]$$

Сбалансированные триты извлекаются из поля, которое формат уже держит, **без знакового деления и без знакового остатка**. Сверено с эталонным преобразованием на смещениях 0, 33, 40, 53, 80. Обобщение: смещение сбалансированного поля из d цифр по основанию r есть репьюнит $(r^d - 1)/(r - 1)$, и **потому, что это репьюнит**, разсмещение — подигитный декремент, а не вычитание с заёмом.

Теоретико-числовое замечание к троичному каналу

Код $nBmT$ несёт n бит в m троичных символах и оставляет $k = 3^m - 2^n$ неиспользованных слов. Разделитель кадра недостижим из данных при $k \geq 1$ и стоит минимума ёмкости при $k = 1$. Уравнение $3^m - 2^n = 1$ имеет **ровно два** решения в натуральных числах — $(1, 1)$ и

$(2,3)$ — поскольку при $m, n > 1$ это **теорема Михайлеску** (2002; гипотеза Каталана, 1844). Случай $(1,1)$ вырожден: один бит на троичный символ, КПД $1/\log_2 3 = 63.09\%$.

Значит **3B2T** — единственный невырожденный троичный линейный код, оставляющий ровно одно резервное слово, при КПД $1.5/\log_2 3 = 94.64\%$. Это даёт выбору IEEE 802.3bp-2016 второе, теоретико-числовое обоснование помимо скорости и спектра. Проверено исчерпывающим целочисленным перебором при $m \leq 199$, $n \leq 319$; перебор проверяет формулировку, доказательство принадлежит Михайлеску.

MVP: ПОЛНАЯ ТРОИЧНАЯ СЕТЬ НА КРЕМНИИ, 2026-08-14

*Раздел добавлен агентом. Всё измерено на трёх физических XC7A200T-FGG676; каждая цифра сопровождается командой. Раздел не заменяет кремниевые таблицы статьи — он предъявляет первую **сеть**, а не первое ядро.*

Что положено на кристалл

Однослойный троичный классификатор: **8 двоичных входов** $\rightarrow$ **3 класса** $\rightarrow$ **argmax**, **24 троичных веса** в алфавите $\{-\phi, 0, +\phi\}$, пятнадцать из них ненулевые. Каждый класс — шаблон над входом, отклик — корреляция, класс — наибольший отклик. **Это и есть слой троичной нейросети** — та же операция, что выполняет слой BitNet, в размере, где арифметика уместается в одну строку.

ресурс	количество
LUT (сеть)	83 — 4 LUT2, 4 LUT3, 49 LUT4, 2 LUT5, 24 LUT6
CARRY4	37
LUT (с обвязкой верхнего уровня)	113
DSP48E1	0
доля XC7A200T	0.06 %

Проверено в двух бэкендах, оба способны провалиться: `zig test 31/31`; `iverilog + vvp 31 PASSED`, ноль ошибок компиляции. P&R: **28 warnings, 0 errors**, 5174 строки FASM, 20 230 фреймов.

Загружено на все три платы битстримом `2b2ace34...` (9 730 896 байт, собран целиком локально):

```
0:4    Done 0x0 -> Done 0x1, No ID error
0:7    Done 0x0 -> Done 0x1, No ID error
0:10   Done 0x0 -> Done 0x1, No ID error
```

Переход получен **принудительно**: плата сперва уронена в `Done = 0` битстримом от другого чипа, и только потом загружен наш. `done 1` сам по себе ничего не доказывает — тракт конфигурации проверяет конверт, а не содержимое.

Каждое ожидаемое значение посчитано независимо и записано в шапку спецификации ДО того, как код был написан. Правило ничьей — «побеждает первый» — тоже часть спецификации и проверено при `x = 0` и `x = 255`, где все три отклика равны: неявное правило ничьей есть место, где эталон и реализация расходятся, оба выглядя правильными.

Ёмкость: сколько модели держит один кристалл

Ресурсы XC7A200T: **134 600 LUT6, 365 BRAM36** = 13 455 360 бит = **1.60 МБ**, 740 DSP48E1.

формат	весов на кристалле
GF Ternary, 2 бита	6.73 М
4 бита	3.36 М
int8	1.68 М
GF-T16 / TNF16	0.84 М

Троичность даёт множитель ×8 по ёмкости против int8 и ×2 против четырёх бит. Это, а не площадь декодера, — практический аргумент за двухбитный алфавит: на одном кристалле помещается 6.73 миллиона весов, и только так.

По измеренному составному узлу (66 LUT на MAC) на плату помещается **2039 параллельных MAC**, что при 100 МГц даёт **204 GMAC/s** без единого DSP.

Сколько плат нужно, и почему ответ не про ёмкость

модель	параметров	плат @2 бита
MobileNetV2	3.4 М	1
EfficientNet-B0	5.3 М	1
ResNet-18	11.7 М	2
TinyBERT-4L	14.5 М	3
DistilBERT	66 М	10
SmolLM2-135M	135 М	21
BitNet b1.58 2B	2.0 B	298

SmolLM2-135M — модель, на которой измерена вся перплексия этой статьи; по ёмкости она требует **21 платы**.

Но ёмкость — не узкое место. Троичный канал 3B2T на одном проводе LVCMOS33 даёт 150 Мбит/с при 100 МСимв/с (1.5 бита на символ). Сравнение с вычислением на той же плате:

слой 576×576 = 331 776 MAC на 2039 узлах @100 МГц	=	1.6 мкс
передача активаций, seq=128, один провод	=	3932 мкс
32 провода	=	123 мкс

Связь медленнее счёта в 77 раз даже на тридцати двух проводах. Плата считает слой за 1.6 мкс и затем 123 мкс ждёт. Разбиение модели по слоям между платами Artix-7 оставляет фабрику простаивающей 99 % времени.

Режим, в котором разбиение окупается, ровно один: генерация по одному токenu. При **seq = 1** передаётся 576 активаций — 30.7 мкс на одном проводе и **1.0 мкс на тридцати двух**, и связь перестаёт быть узким местом.

Следствие для архитектуры сети. [источник в тексте] Множество FPGA осмысленно не как один разрезанный слой, а как множество узлов, **каждый со своей моделью до 6.7 М**

весов, обменивающихся редко. Это mesh, а не кластер, — и потому троичный линейный код с недостижимым из данных разделителем кадра является правильным примитивом связи, а не украшением.

Чего этот раздел не показывает

- **Что вычисляет плата сейчас — не наблюдалось.** Done 0x1 одинаково до и после любой загрузки; обратное чтение конфигурации не выполнялось. Доказана **конфигурация**, не **функция**.
- **Три платы несут одну и ту же сеть.** Это репликация, а не распределение: межплатной связи не существует, и ни одной строки документации о ней найдено не было.
- **Карта выводов расширительной гребёнки отсутствует.** Единственная в репозитории — от **другой платы** (XC7A100T-CSG324 против нашего XC7A200T-FGG676) и противоречит соседнему файлу: T14 / T15 названы UART в одном и JTAG в другом. По ней подключать нельзя.
- **UART каждой платы не проверен на разводку.** Три моста CP2102 перечислены системой (/dev/cu.usbserial-130 / -1230 / -1240), но чтение и запись дали тишину — что доказывает лишь, что **загруженный сейчас проект не умеет говорить**, и ничего не говорит о проводах.
- **Точность не измерялась.** Ни здесь, ни в сравнении ф-алфавита с единичным.

$$\varphi^2 + \varphi^{-2} = 3 \mid \text{TRINITY}$$

Что устояло на трёх задачах (W801–W804)

Раздел выше об алфавите написан до того, как программа перемерила себя. Ниже — то, что осталось после трёх датасетов, восьми семян и разделения по знаку эффекта. **Читать это раньше разделов об алфавите.**

Три заявки, значимые на трёх задачах

вмешательство	UNSW	Fashion	MNIST	
нормировка (последний BN)	+17.85	+6.36	+20.48	знак ×3, значимо ×3
fan-in 6 против 3	+1.73	+0.91	+4.51	знак ×3, значимо ×3
троичные активации	-2.37	-0.35	-1.83	стоят денег на всех трёх
сбалансированное покрытие	-0.77	+0.54	+0.01	три разных ответа
глубина L=5	-0.27	+0.46	+2.80	три разных ответа

Только первые три можно называть свойствами. Два последних — наблюдения о задаче: пятислойная сеть **ни на одной задаче не была хуже** трёхслойной, она просто ничего не давала на двух из трёх. **[измерено]**

Величины различаются вчетверо и не подлежат усреднению. UNSW труднее, и там всякий эффект крупнее; объединённое число не описало бы ни одну задачу.

Закон fan-in: предсказан до измерения

Средняя взаимная информация признака предсказывает выигрыш от fan-in 6 **до обучения**. Два датасета выбраны по MI заранее, прогноз зарегистрирован до первой эпохи, оба попали:

датасет	средняя MI	выигрыш	
0v1	0.0823	+0.14	предсказано
Fashion	0.0561	+0.91	подогнано
UNSW	0.0269	+1.73	подогнано
4v9	0.0096	+2.77	предсказано
MNIST	0.0058	+4.51	подогнано

Моноotonно по пяти датасетам. Механизм: fan-in платит тем больше, чем беднее средний признак — когда каждый вход несёт мало, нейрон из трёх часто не ловит ничего. **[измерено]**; функциональная форма связи **не измерена**, установлен только порядок.

Это первая предиктивная закономерность в алфавитной линии. Всё прежнее — топ оснований, эффект мощности, потолок девяти ступеней, junta degree — было подогнано к уже увиденным данным, и три из четырёх позже отозваны или пересужены.

Что это делает с правилом шести бит

S4 запрещает fan-in 6 по доводу о площади: post-route $3.75\times$ на слое (не $20\times$, как следовало из подушевых цифр). T482 говорит, **что эта площадь покупает, по задачам**, из статистики, считаемой за минуту без обучения: ничего на 0v1, +2.77 пп на 4v9. **Фильтр, запрещающий рычаг, ценность которого теперь предсказуема, должен брать предсказание на вход, а не запрещать рычаг. [открытая гипотеза]** — переписывание S4 не сделано.

Кремний: пять спецификаций ответили (W796–W799)

Прежние разделы о железе описывают отдельные сборки. Ниже — то, что измерено целиком: **спецификация .t27 доводится до кристалла и возвращает вердикт**, причём каждая загрузка взята в скобки чужим битстримом, который обязан сначала уронить Done в ноль. **done 1 сам по себе не считается результатом.**

спецификация	LUT	CARRY4	DSP48E1 / SRL16E	цепочка BSCAN	вердикт
specs/fpga/ternary_link.t27	118	16	0 / 0	4	ok=1, 3/3 плат
specs/numeric/e8m0.t27	98	56	0 / 0	1	ok=1, 3/3
specs/numeric/tnf17.t27	86	16	0 / 0	4	ok=1, 3/3
specs/igla/race/phi_weights.t27	86	16	0 / 0	3	ok=1, 3/3
specs/igla/race/ternary_node.t27	92	16	0 / 0	3	ok=1, 3/3

Пятнадцать вердиктов «спека → кристалл», ноль DSP48E1 и ноль SRL16E в каждой — значит ни одна не подвержена дефектам openXC7 из T246/T250 и T342, где верный нетлист даёт неверный битстрим. **[измерено]**

Ни одна проверка не использует таблицу золотых констант. Таблица, выведенная из спецификации, сверяла бы спецификацию с собой; вместо этого проверяется алгебраическое свойство, которое спека утверждает сама: инволюция отрицания, антисимметрия весов, аннигиляция вне алфавита, точная аддитивность аккумулятора. У каждой — **своя оговорка о нетривиальности**, потому что мёртвый

ответ у каждой свой: инволюцию проходит провод, антисимметрию — модуль, возвращающий ноль, аддитивность — модуль, возвращающий асс нетронутым. **[доказано]** для свойств, **[измерено]** для их проверки на кремнии.

Чего это не утверждает. Что схемы вычисляют всё, что записано в спецификациях: проверено ровно то, что перечислено выше. И тайминг чтения не охарактеризован — вердикт читается до трёх раз, и на четырёх конструкциях из пяти однократное чтение объявило бы одну из плат отказом. Причина не названа.

golden_sieve.t27 до кристалла дойти не может и это верно: файл целиком состоит из предикатов и комптайм-инвариантов, восемь доказаны при компиляции, и через границу двигать нечего. Любая доля корпуса «на кремнии» обязана исключать доказательственные спеки из знаменателя.

ERRATA W778–W788 (2026-08-17)

Статья написана до волн W778–W788. Ниже — что эти волны изменили, по пунктам. Проза выше **не переписана**; каждый пункт называет затронутое утверждение и теорему, которая его меняет. Полный разбор: `docs/reports/TNF-ARTICLE-RECONCILIATION.md`. Теоремы: `docs/theory/IGLA-FORMAL-RESULTS.md`.

Что не изменилось

Замкнутость $\mathbb{Z}[\varphi]$ через $\varphi^2 = \varphi + 1$; перечисление безумножительных шкал (целое алгебраическое число, у сопровождающей матрицы элементы в $\{0, \pm 1\}$; вторая степень допускает φ единственно); «аккумулятор — единственное место, где нужен float»; закон ширины. Это алгебра и перечисление — измерения их не трогают. **[доказано]**

Что пересужено

1. **Мощность алфавита как рычаг точности.** Опубликованный эффект +0,844 пп измерялся на стенде **без нормировки**. На нормированном стенде переход 3→9 уровней стоит +0,26 пп (UNSW) / +0,28 (Fashion) — около 70 % прежней величины было

артефактом фиксированного порога, и порог штрафвал именно **узкий алфавит** (T447). **Закон девяти ступеней понижен с закона до измерения**: держится на Fashion, ломается на UNSW на 13 уровнях (+0,14 пп). **[измерено]**

2. **«Меньше алфавит — дешевле в кремнии»**. Неверно для табличного тракта. Post-route: троичный $\{0, \pm 1\}$ — **137 SLICE_LUTX**, дуадический на девяти уровнях — **137**, линейный на девяти — **128**. Таблица занимает $2^{\text{fanin} \times \text{bits}}$ строк при **любом** алфавите; мощность входит в цену только в **сумматорном** тракте (T448, и T398: 203 против 103). **[измерено]**
3. **Оптимум лестницы**. Ни одна целая лестница из двух и более величин не проходит фильтр недоминирования, **ни при каком размере**: для $A = \{0\} \cup \{\pm b^i\}$, $b \in \mathbb{Z}$, $b \geq 2$, $k \geq 2$ верно $\sum_{i < k-1} b^i < b^{k-1}$ (T442, исчерпывающе по $b \in [2, 12]$, $k \in [2, 12]$). Утверждение об оптимуме вдоль лестницы остаётся **представленческим**; в тракте с fan-in 3 ранжирование иное и перечислено: **linear 9 = $\{0, \pm 1, \pm 2, \pm 3, \pm 4\}$ — ранг 1 из 1156 допустимых девятиуровневых целых алфавитов** (T444). **[доказано] / [измерено]**
4. **φ как носитель межслойного масштаба**. Преимущество реально для **одной величины** — $\{0, \pm \varphi\}$ односторонний, потому что φ сокращается как общий масштаб (T406) — и **разворачивается для лестницы**, где смещение степеней φ требует двухполосного разрешения ценой 8 DSP48E1 или ~2750 LUT (T293). **[измерено]**

Что добавить при переработке

- **T455**: junta degree → площадь, наклон **+151 LUT на единицу**, 95 % ДИ **[+139, +189]**, Spearman **+1,000**, $n=5$, **при L=8**; при L=4 связь не разрешается. Корреляции r в качестве доказательства не приводить (T424).
- **T413с / T456**: два независимых дефекта тренера, каждый **переупорядочивал** плечи, а не опускал их — фиксированный порог (r перевернулся $-0,971 \rightarrow +0,956$) и одноклассовый валидационный сплит (порядок связности обратился).
- **T454**: двенадцать входных бит стоят **3,75×** шести post-route на слое, а не 20–27×, как следует из подушевых цифр.

- **T456b**: паритет с полем достигнут — **4,76 пп против 4,79** у SparseLUT при сопоставимой конфигурации — **устранением дефектов, а не архитектурой**.

Что придержать

Всё, что опирается на ветку связности (T430, T431, T432, T439, T443, T446): измерялось под одноклассовым валидационным сплитом, и порядок **обращается** при его починке (T456). Перемер идёт.

Приоритет, который надо процитировать

LogicNets (arXiv:2004.03021) — правило шести бит как модель стоимости, и их же конфигурации идут на 14 битах. Servedio (arXiv:0902.3757) — critical-index разложение пороговых функций. OEIS A058265 — константа трибоначчи. APoT (arXiv:1909.13144) — «жёсткое разрешение» PoT. FINN §4.2.2 и TWN (arXiv:1605.04711) — свёртка BatchNorm в порог. Logic Shrinkage (10.1145/3583075) — обучение числа входов на LUT ради площади.